

MMO PROJECT

MMO Nexus Server Engine

C++ IOCP 기반 다중 서버 구조와 Unity 클라이언트를 함께 구현한 MMORPG 서버 프로젝트



프로젝트 유형

개인 MMORPG 서버 프로젝트
(클라이언트 통합 구현)



개발 기간

2025.06 ~ 2026.03



사용 기술 스택

C++, C#(Unity), Windows
IOCP, Redis, MSSQL,
Visual Studio 2022



[GITHUB 바로가기](#)

전체 소스 코드와 구현 기록



[게임 시연 영상 바로가기](#)

플레이 영상 또는 시연 링크 자리

MMO Nexus Server Engine

C++ IOCP 기반 다중 서버 구조 MMORPG 프로젝트

한 줄 요약

LoginServer, GameServer, DBAgent, Unity 클라이언트를 분리한 C++ IOCP 기반 MMORPG 프로젝트

프로젝트 설명

네트워크 처리, 서버 분리, 서버 기준 판정, Redis Write-Back 기반 저장 처리까지 실제 플레이 흐름 전체를 직접 연결해 구현한 MMORPG 프로젝트다.

핵심 목표

클라이언트 입력을 그대로 믿지 않고 서버가 최종 결과를 결정하도록 만들었으며, Actor/JobQueue 기반 순차 처리로 동시성 문제를 줄이고, Redis Write-Back과 DBAgent 분리 구조로 실시간 처리와 저장 처리를 안정적으로 나누는 것을 목표로 했다.

기술 구성

사용 언어

C++, C#(Unity 클라이언트)

네트워크

Windows IOCP

저장소

Redis, MSSQL

개발 환경

Windows, Visual Studio 2022, Unity 2022.3.62f3

설계 기준

✓ 서버가 최종 상태를 결정

✓ 상태 변경은 순차 경로에서 처리

✓ 실시간 처리와 저장 처리를 분리

✓ 실패는 같은 흐름으로 정리

프로젝트 범위

네트워크 코어

IOCP 기반 Session, PacketSession, 송수신 완료 처리와 종료 정리 경로 구현

서버 구성

LoginServer, GameServer, DBAgent 분리와 서버 간 통신 구조 구성

게임 기능

이동, 전투, AOI, 거래, 파티, 맵 전이, 인스턴스 플레이 흐름 구현

저장 처리

Redis Write-Back, AutoCommit, DBAgent 저장, 종료 시 Flush 처리

클라이언트

Unity 클라이언트, 패킷 연동, 로그인, 입장, 인게임 화면 구성

CONTENTS

1

Section 1. 개요 & 범위 정의

프로젝트 한 페이지 요약
시스템 아키텍처
End-to-End 데이터 흐름

4

Section 4. 인증 처리와 저장 구조

로그인 인증과 입장 안정화
저장 구조와 데이터 정합성

2

Section 2. 핵심 게임 기능과 상태 처리

서버 권위 이동 검증 및 이동 동기화
AOI와 스냅샷 배치
스킬·투사체 판정과 피해 확정 경로
맵 전이·파티·인스턴스 던전 정합성 처리
거래 원자성 보장과 실패 수렴 검증

5

Section 5. 운영 관측 & 성능 검증

메모리/송신 가드레일
Prometheus + Grafana
DummyClient 부하 검증

3

Section 3. 서버 실행 구조와 처리 흐름

실행 구조와 동시성 제어
패킷/IO 신뢰 경로와 완료 처리
입장 게이트와 로딩 동기화
맵 전이 검증과 종료 수렴

6

Section 6. 신뢰성 사례 & 마무리

안정성 도구 + 트러블슈팅
역량 요약 & 참고 링크

Section1 [프로젝트 흐름 요약]

- IOCP 기반 C++ MMORPG 서버 전반을 단독 구현한 개인 프로젝트
- 로그인, 입장, 이동, 전투, 거래, 저장까지 핵심 서버 경로 포함
- 서버 권위 판정, 동시성 제어, 영속화 구조, 실패 수렴 기준 중심 정리

이번 파트에서 확인할 내용

✔ 1. 로그인부터 종료까지의 서버 흐름

로그인, 입장, 플레이, 저장, 종료가 어떤 확인 절차를 거쳐 이어지는지 단계별로 정리해 전체 흐름을 보여준다.

✔ 2. 아키텍처와 주요 구현 포인트

LoginServer, GameServer, Redis, DBAgent가 맡은 역할과 게임 서버에서 중요하게 본 주요 구현 포인트를 함께 정리한다.

로그인부터 종료까지 이어지는 처리 단계

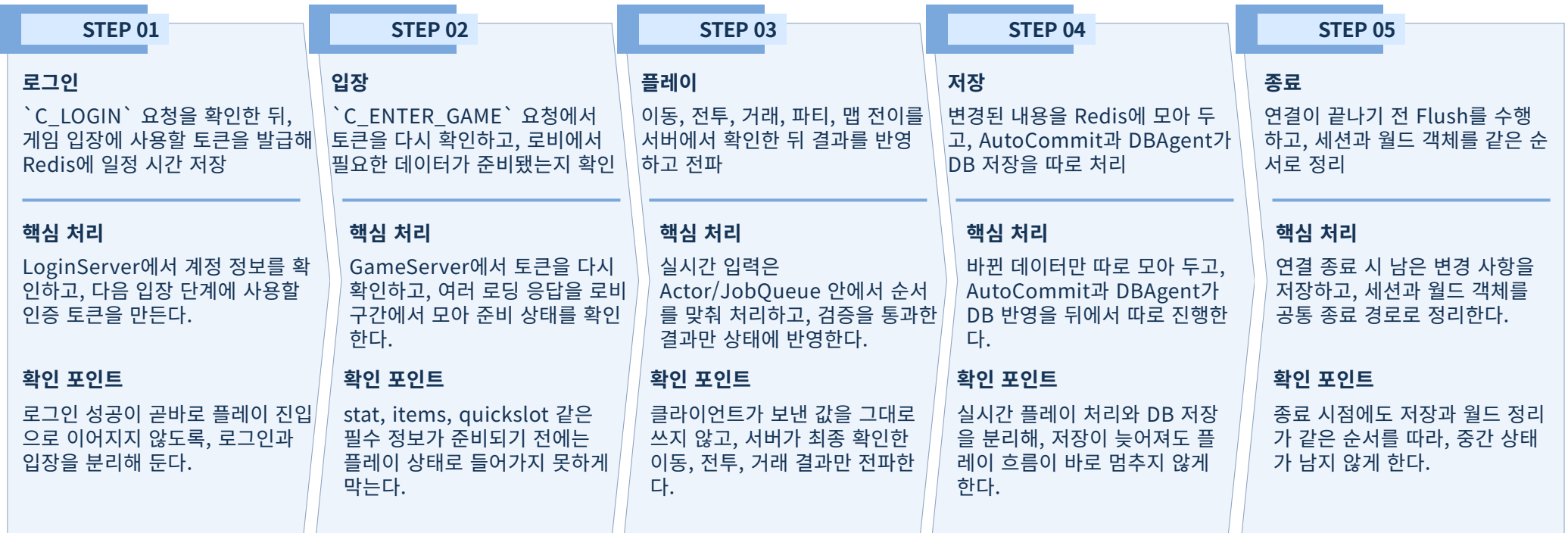
1. 로그인부터 종료까지의 서버 흐름

- ▶ 로그인부터 종료까지 각 단계가 어떤 확인 절차를 거쳐 다음 단계로 넘어가는지 정리한 흐름이다.
준비되지 않았거나 검증되지 않은 상태는 다음 단계로 넘기지 않도록 구성했다.

이 흐름의 핵심

로그인과 입장 분리 로그인 성공이 바로 플레이 진입으로 이어지지 않도록, 입장 단계에서 한 번 더 확인하도록 나눴다.	입장 전 데이터 확인 캐릭터 정보와 인벤토리 같은 필수 데이터가 준비되기 전에는 플레이 상태에 들어가지 못하게 했다.
플레이 중 서버 검증 이동, 전투, 거래, 전이는 클라이언트 입력을 그대로 쓰지 않고 서버에서 확인한 뒤 반영한다.	저장과 종료 정리 플레이 중 저장과 종료 직전 정리를 나눠, 데이터 누락이나 중복 저장 가능성을 줄였다.

단계별 처리 흐름

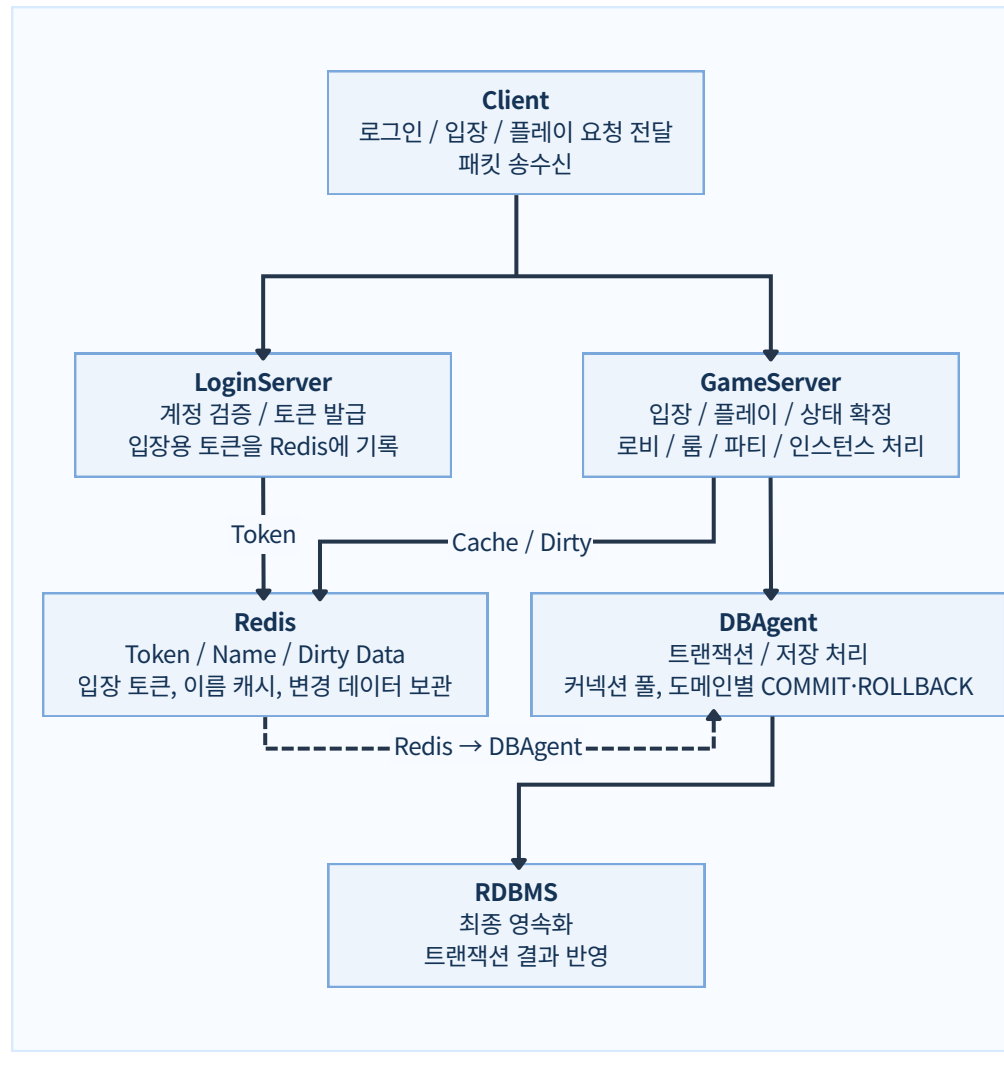


2. 아키텍처와 주요 구현 포인트

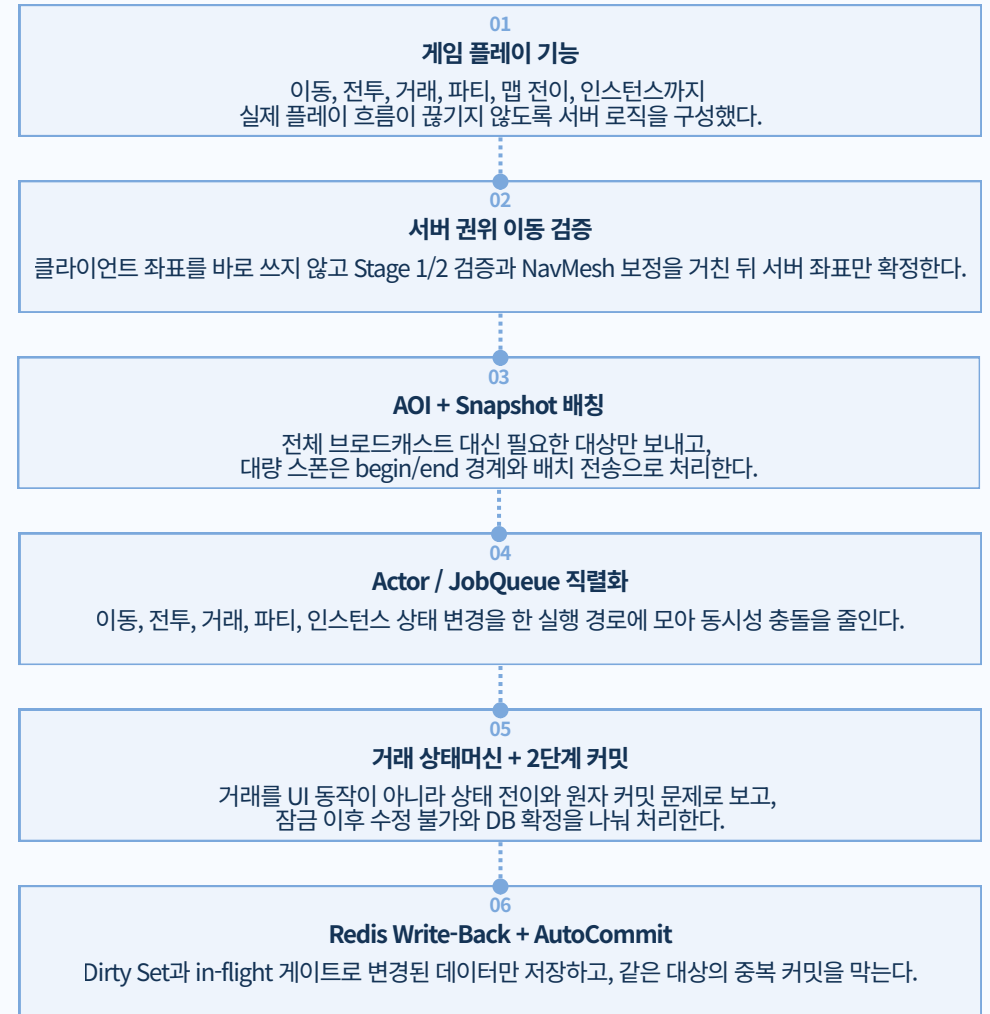
컴포넌트 역할과 주요 데이터 흐름

▶ LoginServer, GameServer, Redis, DBAgent, RDBMS가 나뉘어 맞물리는 전체 흐름을 먼저 정리했다.

시스템 아키텍처



주요 구현 포인트



Section2 Part A. [서버 권위 이동 검증 및 이동 동기화]

- 클라이언트 입력은 제안으로만 받고, 좌표 확정은 서버에서 수행
- Stage 1에서 후보를 정리하고 Stage 2에서 최종 반영 좌표를 판정
- 충돌 무시, 비정상 거리, 역전 시퀀스 요청을 공통 규칙으로 차단

이번 파트에서 확인할 내용

✔ 1. 클라이언트 입력 신뢰의 리스크와 서버 이동 검증 기준

클라이언트 신뢰 시 발생하는 속도해/텔레포트, 벽 통과/지형 침범, 시퀀스 역전/재전송을 리스크로 정의하고, 속도 상한 검증
· `ValidateMove` · `IsSeqNewer` 기반 대응을 함께 제시한다. 결론은 검증을 통과한 좌표만 commit/전파해 월드 정합성과 전
투 공정성을 유지하는 것이다.

✔ 2. 이동 검증 2단계 체계 - 입력 정규화부터 좌표 확정·전파 결정까지

Stage 1에서는 요청 정규화와 후보군 수집으로 불필요한 연산을 먼저 줄이고, Stage 2에서는 `ValidateMove`를 통과한 좌표만
최종 반영 대상으로 확정한다. 이 2단계 분리 구조로 초기 필터링 속도와 최종 판정 정확도를 함께 확보한다.

✔ 3. 서버 이동 검증의 차단·보정 동작 증명

역순·NaN/Inf/CrazyPos는 drop, OOB·ValidateMove reject는 오너 재동기화, 속도 초과·충돌은 clamp/slide 후 commit되
는지 확인해 hard drop / owner correction / 보정 commit 흐름을 검증한다.

1. 클라이언트 입력 신뢰의 리스크와 서버 이동 검증 기준

클라이언트 입력은 제안, 위치 확정은 서버 책임

▶ 왜 클라이언트 좌표를 그대로 반영하면 안 되는지와, 이를 막기 위한 서버 검증 기준을 설명한다.

클라이언트 신뢰 시 실제 문제

리스크 시나리오	발생 조건	영향
속도해/텔레포트	과도한 delta/거리 요청	전투 공정성 붕괴
벽 통과/지형 침범	충돌 무시 좌표 제출	규칙 밖 좌표 commit
시퀀스 역전/재전송	과거 seq 재주입	현재 상태 롤백/Desync

결과: 월드 상태 오염 -> 전투 불공정 -> 클라이언트 간 위치 불일치

서버 권위 기반 해결 방법

리스크 시나리오	서버 대응	효과
속도해/텔레포트	과도한 delta/거리 요청속도 상한 검증 + 초과 거리 클램프	비정상 이동 차단, 전투 공정성 유지
벽 통과/지형 침범	NavMesh `ValidateMove` + slide/correct 보정	지형 규칙 보존, 불법 좌표 반영 방지
시퀀스 역전/재전송	`IsSeqNewer` 기반 역순 입력 즉시 drop	상태 롤백 방지, 동기화 일관성 유지

결론: 검증 통과 좌표만 커밋/전파해 월드 정합성을 안정적으로 유지한다.

위협-대응 구조 다이어그램

Risk Path (Client Trust)



Authoritative Validation Path



서버 권위 설계 원칙

- **순차 검증:** `seq`, `dt`, `speed`, `navmesh`를 고정 순서로 처리
- **예외 수렴:** `hard drop`, `owner correction`, `clamp/slide`를 상황에 맞게 분기
- **상태 반영:** 검증 통과 좌표만 `commit`
- **동기화:** 확정 결과만 AOI 대상에 `S\_MOVE` 전파

핵심 보장 원칙

1. 검증 실패 입력은 월드 상태를 변경하지 않는다.
2. 이동 상태 커밋은 단일 경로에서만 수행한다.
3. 브로드캐스트는 서버 확정 좌표만 전송한다.
4. 다음 검증 기준(마지막 seq/time)은 커밋 시점 기준으로만 전진한다.

서버 이동 검증 기준

- **입력 안정성:** NaN/Inf/Crazy Position 즉시 거부
- **순서/시간:** 역순 seq drop, dt 최소/최대 clamp
- **속도:** 허용 이동 거리 초과 시 보정 후 전달
- **지형:** `ValidateMove` 통과 좌표만 확정

2-1. 이동 입력 정규화와 1차 판정 기준 (Stage 1)

요청 수신 -> 입력 적합성 확인 -> 순서/시간/속도 1차 판정

▶ 서버 이동 검증을 Stage 1과 Stage 2의 2단계로 나눠, Stage 1에서는 입력 정규화와 1차 판정을 수행해 유효 후보만 다음 단계로 전달한다.

Stage 1·2 분리 설계 이유

처리 효율

입력 이상·역순 요청을 1차에서 먼저 걸러 불필요한 후속 연산을 줄인다.

확정 안정성

좌표 커밋과 전파는 Stage2에서만 수행해 상태 반영 경로를 하나로 만든다.

실패 수렴

Stage 1 실패는 hard drop 또는 owner correction으로, Stage 2 reject는 no commit + owner correction으로 종료한다.

Stage 1·2 역할 구분

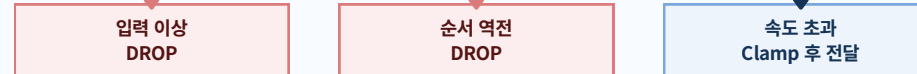
비교 항목	Stage 1	Stage 2
주요 목적	입력 정규화·1차 판정	좌표 확정·전파
입력 데이터	클라이언트 원본 요청(`reqRaw`)	정규화 후보 좌표(`reqClamped`)
주요 산출물	Stage 2 전달용 후보 좌표(`reqClamped`)	서버 확정 좌표(`fixed`) + 전파 이벤트
월드 상태 변경	없음(미확정)	있음(커밋)
실패 시 수렴	hard drop 또는 owner correction 후 종료	no commit + owner correction(필요 시)

Stage 1 처리 흐름 (입력 정규화)

Stage 1 Validation Path



Authoritative Validation Path



입력 상태별 판정 기준표

입력 상태	판정 기준	서버 처리	서버 처리
입력 이상	NaN/Inf/CrazyPos	즉시 중단(drop)	X
순서 역전	`IsSeqNewer == false`	즉시 중단(drop)	X
속도 초과	`CheckSpeed2D == over`	clamp 보정 적용	O(`reqClamped`)
경계 이탈	`_grid.GetZoneIndex(reqRaw) < 0`	keepPos + owner correction + stamp 갱신	X
정상 입력	모든 검사 통과	정규화 좌표 유지	O(`reqRaw/reqClamped`)

Stage 1 출력 계약

- 성공 경로 출력: `reqClamped` (Stage 2 입력)
- 실패 경로 출력: Stage 2 전달 없음, 필요 시 owner `S\_MOVE`로 현재 권위 좌표 재동기화
- Stage 1 종료 시점에는 월드 좌표를 확정하지 않음

Stage 1 출력 계약

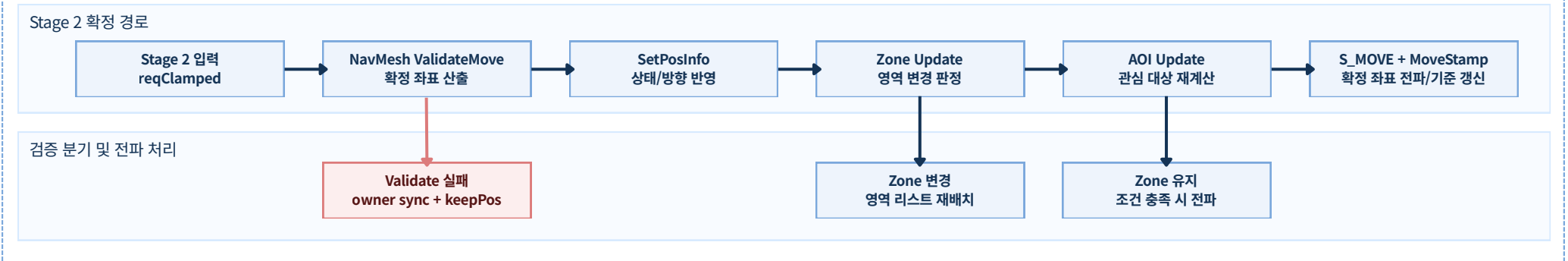
- `C\_MOVE` 수신 후 RoomActor Queue에 직렬화
- NaN/Inf/CrazyPos는 hard drop, Out-of-Bounds는 keepPos + owner correction
- `IsSeqNewer`, `ComputeDtSec`로 순서/시간 기준 확인
- `CheckSpeed2D`로 속도 기준 확인 후 `reqClamped` 산출

2-2. 좌표 확정 및 전파 기준 (Stage 2)

NavMesh 검증 결과를 기준으로 서버 좌표를 확정하고 AOI 대상에 전파

▶ Stage 2는 Stage 1에서 정리된 입력을 최종 검증해 서버 좌표로 확정하고, 확정 좌표만 주변 대상에 전파해 동기화 일관성을 유지한다.

Stage 2 처리 흐름 (좌표 확정 및 전파)



확정/전파 판단 기준표

검증 결과	서버 처리	상태 반영	전파
`ValidateMove` 실패	현재 권위 좌표 유지 + owner correction + stamp 갱신	X	오너만 O (`S_MOVE`)
Zone 변경	Zone 리스트 이동 + AOI 갱신	O	O (`S_MOVE`)
Zone 유지	AOI 조건 충족 시 전파 대상 재계산	O	O (`S_MOVE`)

Stage 2 처리 단계

- `ValidateMove` 통과 좌표만 `fixed`로 확정
- `SetPosInfo`로 위치/상태를 서버 기준으로 갱신
- Zone/AOI 변경을 반영해 전파 대상을 재계산
- 성공 시 AOI에 `S\_MOVE` 전파, reject 시 owner correction 후 `SetMoveStamp` 갱신

Stage 1 출력 계약

- 서버 상태에는 `ValidateMove` 통과 좌표만 반영한다
- 반영된 동일 좌표만 주변 클라이언트에 전파한다
- `MoveStamp`는 `fixed` 또는 `keepPos`처럼 서버가 유지하기로 확정한 권위 좌표 기준으로 갱신한다
- 검증 reject 요청은 월드 상태를 바꾸지 않되, 필요 시 owner correction으로 종료한다

Stage 1·2 분리 적용 효과

평가 관점	분리 전 리스크	분리 후 효과
처리 효율	모든 요청이 동일 경로로 유입되어 비정상 입력도 후속 연산을 점유	Stage 1 조기 차단으로 Stage 2가 유효 입력 처리에 집중
상태 반영 경계	정규화와 좌표 확정이 혼재되어 반영 시점 해석이 어려움	좌표 확정/전파를 Stage 2로 고정해 반영 기준을 단일화
장애 분석	실패 지점이 한 경로에 섞여 원인 추적이 지연	Stage별 실패 지점 분리로 원인 파악과 대응 속도 개선
동기화 신뢰성	검증 전 좌표 전파 위험으로 클라이언트 간 상태 차이가 발생	검증 통과 좌표만 전파해 결과 일관성과 재현성을 확보

3. 서버 이동 검증의 차단·보정 동작 증명

로그와 좌표 결과로 차단·보정 동작을 검증

▶ 서버 이동 검증 2단계에서 차단·보정 분기가 의도한 기준으로 동작하는지 케이스별 결과로 검증한다.

실패 케이스 · MOVE_SEQ_REWIND_DROP

DROP CASE

요청 과거 seq 값을 재전송
서버 처리 최신 stamp 대비 역순 입력으로 즉시 drop
결과 플레이어 위치/상태 미변경, 브로드캐스트 없음

LOG EVIDENCE

[MOVE] player=1102 seq=174 lastSeq=176 -> DROP(reason=seq_rewind)
 [STATE] keepPos=(124.60, 0.00, 55.80) no_commit no_broadcast

보정 케이스 · SPEED_EXCEEDED_CLAMP

CLAMP CASE

요청 허용 이동거리 초과 좌표 요청
서버 처리 clamp 적용 후 NavMesh ValidateMove 재검증
결과 보정 좌표만 commit하고 S_MOVE 전파

LOG EVIDENCE

[MOVE] player=1102 reqDist2D=9.82 maxDist=3.60 -> CLAMP
 [MOVE] ValidateMove pass fixed=(128.40, 0.00, 57.12) -> COMMIT + S_MOVE

실패 케이스 · INPUT_SANITY_DROP

REJECT + SYNC

요청 맵 밖 좌표 또는 ValidateMove 최종 reject 요청
서버 처리 월드 상태는 유지하고 MOVE_OWNER_CORRECTION(reason=VALIDATION_REJECT) 로그 후
 오너에게 현재 권위 좌표를 S_MOVE로 재전송
결과 no commit, 주변 브로드캐스트 없음, owner sync + MoveStamp 갱신

LOG EVIDENCE

[MOVE] player=1102 pos=(nan, 0.00, inf) -> DROP(reason=input_invalid)
 [[MOVE_OWNER_CORRECTION] reason=VALIDATION_REJECT ID: 1102 req=
 (140.20,0.00,74.00, yaw=90) srv=(128.40,0.00,57.12, yaw=90)

보정 케이스 · NAVMESH_COLLISION_SLIDE_CORRECT

CORRECT CASE

요청 벽 관통/비정상 지형 진입 좌표 요청
서버 처리 ValidateMove에서 충돌면 기준 slide/correct 보정
결과 보정 좌표만 commit 후 S_MOVE 브로드캐스트

LOG EVIDENCE

[MOVE] player=1102 req=(132.90, 0.00, 61.70) hit=wall -> CORRECT(slide)
 [MOVE] fixed=(131.28, 0.00, 59.94) -> COMMIT + S_MOVE

요청 좌표 vs 서버 확정 좌표 비교

시나리오	reqDist2D	maxDist	서버 처리	최종 좌표
역순 seq	-	-	drop	변경 없음
과속 이동	9.82m	3.60m	clamp + validate (+ owner sync)	(128.40, 0.00, 57.12)
검증 reject(OOB/실패)	-	-	keepPos + owner correction	현재 권위 좌표 재통지
벽 관통/지형 충돌	3.10m	3.60m	slide/correct + validate	(131.28, 0.00, 59.94)

Section2 Part B. [AOI와 스냅샷 배칭]

- 전체 전송 대신, AOI 조건을 만족한 대상만 선별해 전파
- 후보군 축소와 연결성 판정으로 불필요한 전송을 구조적으로 차단
- 스냅샷 배칭과 완료 경계를 함께 적용해 동기화 완료 시점을 명확히 관리

이번 파트에서 확인할 내용

✓ 1. 전체 브로드캐스트 한계와 AOI 설계 근거 및 적용 방식

플레이어 밀집 구간의 전송 폭증, 입장·재접속 시점의 순간 버스트, 거리 기준 단독 판정에서 발생하는 불필요 전송과 시야 판정 오류를 먼저 짚고, 패킷 경량화만으로는 부하 상한을 제어하기 어렵다는 점을 근거로 AOI 채택 배경을 제시한다. 이후 후보군 축소, 필터 확정, 배치 전송으로 이어지는 적용 방식과 적용 전/후 차이를 함께 보여준다.

✓ 2. AOI 후보군 수집과 연결성 기반 필터링 기준

갱신 조건을 먼저 확인한 뒤 SpatialGrid 인접 존에서 후보군을 수집하고, 거리 판정과 연결성 판정을 순차 적용해 실제 전파 대상만 남기는 절차를 다룬다. 또한 기존/신규 집합 비교를 통해 출현·이탈 변경점을 계산하고 관찰자 목록과 클라이언트 상태를 일관되게 맞추는 기준을 제시한다.

✓ 3. Snapshot 경계 배칭과 초기 동기화 완료 기준

초기 진입과 맵 이동 경로에서 `snapshot\_id`를 발급하고 begin/end 경계 플래그를 강제해, 다량 전송 구간의 시작과 종료를 서버가 명확히 관리하는 방식을 설명한다. 트리거 경로, 배치 단위, 빈 배치 처리, 완료 판정 순서를 함께 적용해 초기 동기화 완료 시점을 서버 기준으로 고정하는 기준을 제시한다.

1. 전체 브로드캐스트 한계와 AOI 설계 근거 및 적용 방식

패킷 경량화 이전에 수신자 집합부터 구조적으로 축소한다

▶ 부하를 안정적으로 줄이기 위해서는 패킷 경량화보다 전송 범위 축소가 먼저 필요했다. AOI는 그 판단에서 출발한 구조적 개선안이었다.

1) 전체 브로드캐스트의 한계

플레이어 수가 늘어나는 구간에서는 "누구에게 보낼지"가 먼저 병목이 된다.
수신 대상 수를 줄이지 않으면 직렬화 최적화만으로는 송신 부담 상한을 통제하기 어렵다.

문제 상황	발생 원인	운영 영향
엔티티 증가 구간	송신자마자 다수 수신자 전송이 동시에 증가	전송 루프/직렬화 비용 급증
입장/재접속 동기화	대량 스폰 단발 전송	순간 버스트로 지연/완료 시점 불명확
거리 기준 단독 판정	지형/연결성 분리가 반영되지 않음	오전송 및 시야 오판정 발생

2) AOI를 채택한 이유

- AOI(Area of Interest)는 플레이어 주변에서 실제로 전송이 필요한 대상만 선별하는 방식이다.
- AOI는 관심 대상만 남겨 송신자 1명당 평균 수신자 수 (`avg\_receivers\_per\_sender`)를 구조적으로 낮춘다.
- 후보군 축소를 먼저 적용하면 CPU(직렬화)와 네트워크(송신 큐)를 동시에 제어할 수 있다.
- 필터 기준을 고정해 이후 배치/동기화 규칙까지 같은 기준으로 연결할 수 있다.

3) 왜 AOI인가

대안 방식	한계	AOI 선택 이유
전체 브로드캐스트 + 패킷 압축	패킷 크기만 줄고 수신 대상 수는 그대로 유지	대상 수 상한 자체를 줄이기 위해 AOI 필요
전송 주기 완화 (저주기/샘플링)	부하는 낮아지지만 위치 최신성 저하	응답성을 유지한 채 불필요 대상만 제거 가능
거리 필터 단독 적용	지형 단절/연결성 정보 미반영으로 오전송 발생	AOI에서 거리 + Connectivity로 정확도 보완

4) AOI 적용으로 해결되는 방식

기존 문제	AOI 적용 방식	해결 결과
전체 대상 전송으로 루프 확대	`SpatialGrid`로 후보군 1차 축소	전송 대상 수 상한 축소
거리 기준 단독 판정으로 오전송	거리 + Connectivity 2차 필터	실제 가시 집합 중심 전송
입장 시 대량 전송 버스트	snapshot begin/end + 배치 전송	초기 동기화 완료 시점 명확화

5) 적용 전/후 비교

구분	기존	개선
대상 선정	전체 브로드캐스트	AOI 후보군 한정
판정 기준	거리 위주	거리 + 연결성
입장 동기화	대량 스폰 단발 전송	snapshot begin/end + 배치 전송
완료 시점 판정	클라이언트 측 추정 처리	경계 패킷으로 완료 시점 명확화
결과	불필요 전송 다수	우호 대상 중심 전송

결론: AOI는 브로드캐스트를 "최적화"하는 수준이 아니라, 전송 대상을 먼저 통제하는 구조적 해법이다.

6) 설계 판단 기준

- ✓ `avg\_receivers\_per\_sender`를 `visible\_set\_size` 수준으로 제한 가능한가?
- ✓ 후보군 축소 → 필터 확정 → 배치 전송 순서가 단일 기준으로 이어지는가?
- ✓ 빈 집합 포함 전 경로에서 `snapshot begin/end` 경계가 유지되는가?
- ✓ 송신량/지연/오류율 지표로 개선을 재현 검증할 수 있는가?

2. AOI 후보군 수집과 연결성 기반 필터링 기준

가까워도 연결되지 않으면 AOI 대상에 포함하지 않는다

▶ AOI 후보군을 수집한 뒤 거리·연결성 판정으로 유효 대상을 확정하고, 기존 집합과의 비교를 통해 출현·이탈 변경점을 계산해 동기화 기준으로 반영한다.

AOI 필터링 구조

AOI 필터 파이프라인은 "주변에 있는 것"이 아니라 "현재 플레이어에게 실제로 보여야 하는 대상"만 남기기 위한 서버 판정 절차다.

1단계 · 후보군 수집

GetNearbyZones로 인접 존 후보를 모은 뒤, 동일 변경 기준으로 플레이어·몬스터·투사체만 수집해 처리량을 고정한다.

2단계 · 정밀 판정

거리(`PassDistance2D`)와 연결성(`Connectivity`)을 순차 적용해 신규 AOI 집합(`NewVis`)을 확정한다.

3단계 · 집합 비교

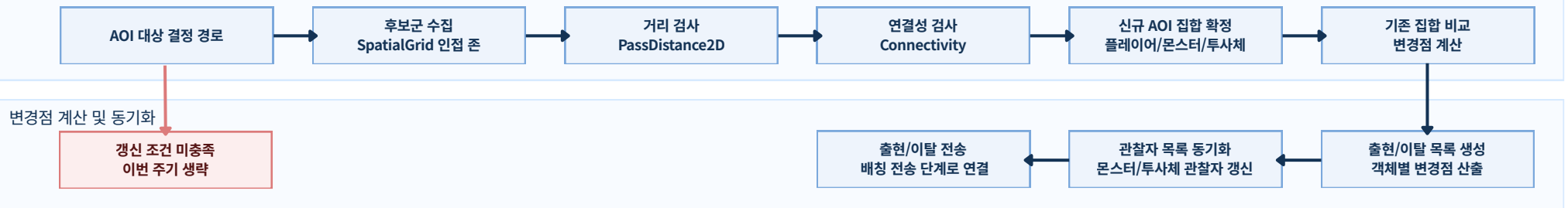
기존 AOI 집합(`OldVis`)과 신규 AOI 집합(`NewVis`)을 비교해 생성 대상(`toSpawn`)·제거 대상(`toDespawn`) 변경점만 산출한다.

4단계 · 변경점 전송

산출된 변경점만 출현/이탈 이벤트로 전송해 송신량을 줄이고 AOI 상태를 일관되게 유지한다.

AOI 필터 파이프라인

Stage 2 확정 경로



1) 갱신 트리거와 입력 기준

1. `ValidateMove` 통과 좌표만 `fixed`로 확정
2. `SetPosInfo`로 위치/상태를 서버 기준으로 갱신
3. Zone/AOI 변경을 반영해 전파 대상을 재계산
4. `S\_MOVE` 전파 후 `SetMoveStamp`로 다음 검증 기준 갱신

2) 변경점 계산 및 동기화 기준

- 서버 상태에는 `ValidateMove` 통과 좌표만 반영한다
- 반영된 동일 좌표만 주변 클라이언트에 전파한다
- `MoveStamp` 갱신 시점을 다음 요청의 검증 기준으로 사용한다
- 검증 실패 요청은 상태 변경과 전파 없이 종료한다

3. Snapshot 경계 배칭과 초기 동기화 완료 기준

▶ AOI로 대상이 확정된 이후, 초기 동기화를 어떤 경계로 전송하고 언제 완료로 확정할지에 대한 서버 기준을 정리한다.

입장·맵 이동 구간에서도 전송 경계를 일관되게 유지해 완료 판단을 명확히 만든다

스냅샷 배칭 개요

스냅샷 배칭이란?

초기 동기화 데이터를 한 번의 큰 패킷으로 보내지 않고, 여러 S_SPAWN 패킷 묶음으로 나눠 전송하는 방식이다. 핵심은 초기 동기화 데이터를 여러 묶음으로 나눠, 한 패킷에 데이터가 과도하게 몰리지 않도록 하고 전송 부하를 고르게 분산하는 데 있다.

왜 이 방식을 적용했는가?

스냅샷 경계는 서버가 같은 묶음에 동일한 snapshot_id를 부여하고, 첫 패킷과 마지막 패킷에 begin/end를 지정해 유지한다.

배치 결과가 비어도 begin=true, end=true 단일 패킷을 보내 경계 누락 없이 완료를 확정한다.

왜 이 방식을 적용했는가?

이 방식을 적용한 이유는 입장/맵 전이 구간에서 전송 대상이 많아 단일 패킷 전송으로 처리하기 어려웠고, 완료 시점을 클라이언트 추정에 맡기면 중간 상태를 완료로 오판할 수 있었기 때문이다.

배치 경계를 서버 기준으로 고정해 부분 반영·중복 반영·로딩 해제 타이밍 오류를 줄였다.

어떤 단위로 나눠 전송하나?

전송은 Player/Monster/Projectile을 각각 별도 배치로 나눠 순차 처리한다. 각 타입은 한 패킷에 담는 최대 개수를 고정해 과도한 단건 패킷 크기와 송신 버스트를 함께 제한한다.

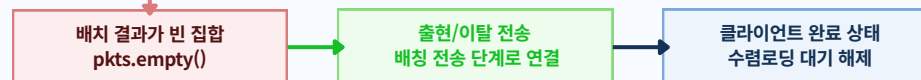
forceFullSnapshot 경로에서는 이 배치 규칙을 강제하고, S_DESPAWN은 별도 경로로 분리해 처리한다.

스냅샷 전송 경로

정상 스냅샷 경로



예외 수렴 경로



3) 트리거 경로

진입 경로	호출	적용 목적
EnterGame	UpdateAOI(..., true)	입장 직후 초기 동기화 기준 고정
EnterMapChange	UpdateAOI(..., true)	맵 전이 직후 동일 경계 규칙 적용

4) 완료 판정 시퀀스

순서	패킷 플래그	의미
1	snapshot_id=K, begin=true, end=false	스냅샷 경계 시작
2	snapshot_id=K, begin=false, end=false	중간 배치 누적 전송
3	snapshot_id=K, begin=false, end=true	스냅샷 종료 및 완료 확정

Section2 Part C. [스킬·투사체 판정과 피해 확정 경로]

- 스킬 요청은 서버 가드와 타입별 판정 규칙을 통과한 경우에만 피해 처리로 연결
- 투사체는 Tick 경로에서 이동·충돌·소멸을 순차 확정해 고속 구간의 누락을 축소
- 쿨타임/중복 타격 차단과 피해 이후 후속 처리 수렴으로 전투 결과 일관성을 유지

이번 파트에서 확인할 내용

✔ 스킬 요청 검증과 타입별 서버 판정 경로

즉발·범위·부채꼴 스킬 요청은 서버 진입 가드(룸 소속, 스킬 유효성, 쿨타임)와 판정 파라미터 보정을 거쳐 타입별 판정 경로로 분기된다. 이 과정에서 실패 요청은 전파 없이 즉시 종료되고, 판정을 통과한 요청만 피해 결과 전파 단계로 넘어간다.

✔ 투사체 Tick 기반 이동·충돌·소멸 확정 경로

투사체는 서버 Tick마다 이동 업데이트와 벽 충돌 보정을 먼저 수행한 뒤, 후보군 필터링과 충돌 순서 고정으로 피해 적용 대상을 확정한다. 마지막으로 종료 조건(벽 충돌, 최대 타격 수, 수명 만료)을 동일 기준으로 수렴시켜 판정 누락과 상태 불일치를 함께 줄인다.

✔ 쿨타임 검증부터 중복 타격 차단까지 서버 실행 경로

연속 입력은 서버 시간 기준 쿨타임 검증으로 선차단하고, 투사체 충돌은 타격 이력(HasAlreadyHit)으로 중복 적용을 막는다. 요청 수신부터 피해 반영 직전까지 동일한 서버 기준을 유지해, 같은 스킬 입력이 반복돼도 피해 결과는 1회만 확정되도록 고정한다.

✔ 전투 결과 확정 이후 서버 수렴 처리 체계

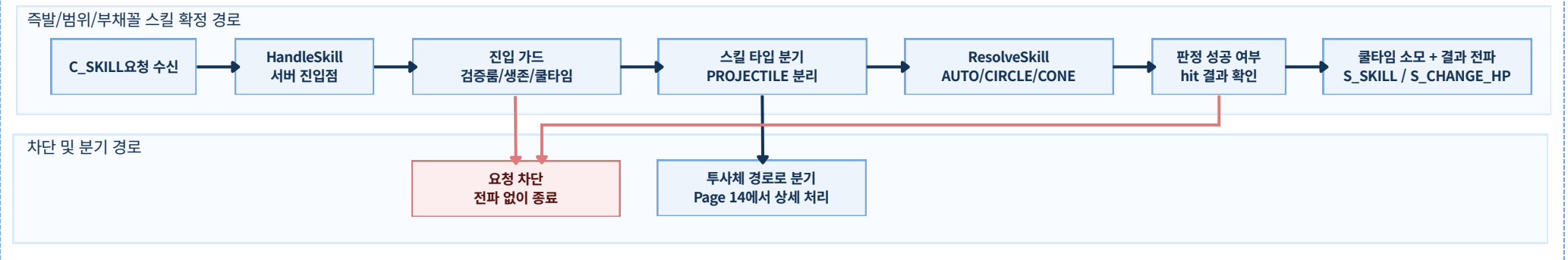
피해가 확정된 뒤에는 AI 상태 전이, 사망 처리, 보상 반영, 리스폰 예약을 Room JobQueue 직렬 경로로 연결해 후처리 순서를 고정한다. 중복 사망 이벤트, 비플레이어 처치, 과스폰 같은 예외도 동일한 서버 가드 기준으로 차단해 최종 상태를 일관되게 수렴시킨다.

1. 스킬 요청 검증과 타입별 서버 판정 경로

즉발/범위/부채꼴 스킬은 공통 가드를 통과한 요청만 피해 반영으로 이어진다

▶ 스킬 요청은 준비, 판정, 전달의 3단계를 통과한 경우에만 전투 결과로 반영되며, 실패 요청은 즉시 종료된다.

요청 -> 판정 -> 피해 반영 흐름



준비

요청을 서버가 판정 가능한 입력으로 정리한다.

진입 가드

- 요청 주체의 현재 룸 소속 여부를 먼저 확인한다.
- 스킬 템플릿 유효성과 공격자 생존 상태를 확인한다.
- 쿨타임 사용 가능 여부를 검사하고 실패 요청은 즉시 종료한다.

판정 파라미터 보정

- singleRange는 range 우선, 없으면 radius를 사용한다.
- circleRadius/coneRange는 radius 우선, 없으면 range를 사용한다.
- coneAngle <= 0이면 기본값 90으로 고정한다.

후보군 수집

- 공격자 주변 Zone 후보군만 조회해 판정 범위를 제한한다.
- 플레이어 공격은 몬스터 후보군, 몬스터 공격은 플레이어 후보군을 사용한다.
- 전체 맵 순회 대신 주변 후보군 기반으로 판정 비용 상한을 유지한다.

판정

스킬 타입별 규칙으로 서버 판정을 확정한다.

>ResolveSkill에서 스킬 타입을 분기해 동일 기준으로 hit 여부를 확정한다.

기본 공격 (SKILL_AUTO)

CheckCircle(singleRange)를 적용해 가장 먼저 적중한 대상 1명만 확정한다.

원형 범위 공격 (SKILL_AREA_CIRCLE)

CheckCircle(circleRadius)를 기준으로 반경 안 대상들을 동시에 판정한다.

부채꼴 범위 공격 (SKILL_AREA_CONE)

CheckFan(coneRange, coneAngle)로 castYaw 방향의 전방 부채꼴 범위를 판정한다.

투사체 공격 (PROJECTILE)

즉시 분기해 투사체 Tick 경로로 넘기고, 이동/충돌/소멸 판정은 다음 페이지에서 처리한다.

전달

성공 결과만 전파하고 실패 요청은 서버에서 종료한다.

성공 전파 시작

판정 성공 시에만 StartSkillCooldown을 수행하고, 스킬 사용 사실을 S_SKILL로 먼저 전달한다.

피해 결과 전달

적중 대상별 처리 결과를 S_CHANGE_HP로 전달해 클라이언트가 최종 피해 상태를 동일하게 확인하도록 맞춘다.

차단 종료

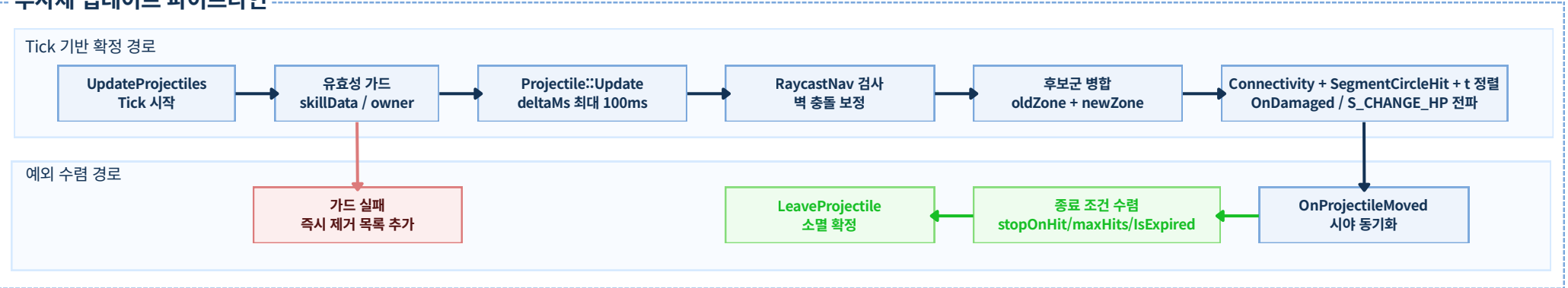
진입 가드 실패, 판정 실패, hit 없음 요청은 전파 없이 서버에서 즉시 종료해 잘못된 전투 결과 반영을 막는다.

2. 투사체 Tick 기반 이동·충돌·소멸 확정 경로

이동, 충돌, 소멸 기준을 한 루프에서 처리해 고속 이동 구간의 판정 누락을 줄인다

▶ 투사체 처리는 서버 Tick마다 이동 보정(RaycastNav), 충돌 판정, 종료 조건을 순차 확정해 고속 이동 구간의 판정 누락과 중복 반응을 줄인다.

투사체 업데이트 파이프라인



투사체 Tick 처리 구조

투사체 처리의 핵심은 "한 번의 Tick에서 이동 갱신, 충돌 판정, 소멸 결정을 함께 확정"해 고속 구간에서도 누락 없는 결과를 유지하는 것이다.

1단계 · Tick 입력 정규화

Tick 입력을 서버 기준으로 고정해 비정상 경로를 초기에 배제한다.



입력 상한·유효성 가드

- deltaMs를 100ms 상한으로 제한해 과도 이동을 차단한다.
- skillData 또는 owner가 없으면 즉시 제거 목록에 추가한다.

2단계 · 이동 보정

투사체 위치를 갱신하고 벽 충돌을 반영해 판정 입력 좌표를 확정한다.



이동/벽 충돌 처리

- oldPos -> newPos 구간을 RaycastNav로 검사한다.
- Tick 종료 시 shouldDespawn || IsExpired()로 제거 여부를 확정한다

- 벽 충돌 시 충돌 지점으로 좌표를 보정하고 소멸 플래그를 설정한다.

3단계 · 충돌 판정

후보군을 정제하고 충돌 순서를 고정해 피해 적용 순서를 단일화해 만든다.



충돌 후보 필터 기준표

- 생존 상태: hp > 0 대상만 검사해 이미 사망한 대상 재타격을 막는다.
- 연결성 일치: GetConnectivityId 동일 구간만 통과시켜 벽 너머 판정을 차단한다.
- 이미 타격한 대상: HasAlreadyHit(victimId) 대상은 제외해 동일 투사체 재충돌 중복 처리를 방지한다.
- 공격자 기준 대상군: 플레이어 발사체는 몬스터, 몬스터 발사체는 플레이어만 검사해 아군/비대상군 오 판정을 줄인다.

충돌 후보 필터 기준표

- oldZone + newZone 후보군을 병합해 충돌 대상을 수집한다.
- 생존 상태와 연결성이 일치한 대상만 충돌 검사를 진행한다.
- SegmentCircleHitXZ로 충돌 시점 t를 계산해 오름차순으로 처리한다.
- stopOnHit 또는 maxHits 도달 시 즉시 소멸 경로로 전환한다.

4단계 · 종료 수렴

소멸 조건을 동일 기준으로 판정해 Tick 종료 시점 처리를 단일화한다.



종료/수렴 기준

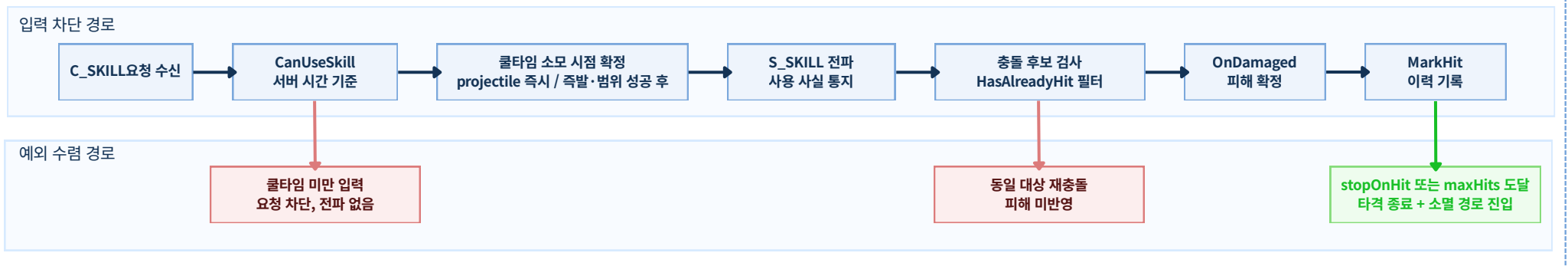
- 가드 실패(skillData/owner 없음): 즉시 제거 목록에 추가해 이후 판정을 중단한다.
- 벽 충돌 또는 타격 종료 조건 충족: shouldDespawn=true로 소멸 경로에 진입한다.
- 수명/사거리 만료(IsExpired): Tick 종료 시 LeaveProjectile을 호출해 소멸을 확정한다.

3. 쿨타임 검증부터 중복 타격 차단까지 서버 실행 경로

쿨타임은 서버 시간, 타격 기록은 서버 집합 기준으로 검증한다

▶ 반복 입력이 들어와도 서버는 쿨타임 검증과 타격 이력 차단을 함께 적용해, 피해를 검증 통과 1회 결과로만 확정한다.

쿨타임/중복 타격 차단 흐름



서버 시간 기반 쿨타임 검증

사용 가능 여부 검사

CanUseSkill은 스킬 템플릿 유효성과 만료 시각을 함께 확인한다.

시간 기준 고정

판정 시각은 GetTickCount64로 고정하며, 클라이언트 시각은 기준으로 사용하지 않는다.

만료 시각 갱신

StartSkillCooldown에서 now + cooldownMs로 다음 사용 가능 시점을 기록한다.

소모 시점 분리 적용

쿨타임 소모 시점은 투사체 즉시, 즉발/범위는 판정 성공 후로 구분해 적용한다.

타격 이력 기반 중복 타격 차단

이력 상태 유지

투사체마다 _hitVictims, _hitCount를 런타임 상태로 유지한다.

후보 단계 즉시 제외

충돌 후보 단계에서 HasAlreadyHit(victimId)면 즉시 제외한다.

피해 확정 직후 기록

피해 확정 직후 MarkHit(victimId)로 타격 이력을 기록한다.

풀 재사용 초기화

오브젝트 풀 재사용 시 ResetHitState로 이전 이력을 초기화한다.

타격 상한 수렴

stopOnHit 또는 maxHits 도달 시 추가 타격 없이 소멸 경로로 이동한다.

차단 시나리오별 처리 기준

쿨타임 미만 연속 입력

CanUseSkill == false로 판정해 요청을 차단하고 전파하지 않는다.

동일 대상 재충돌

HasAlreadyHit == true면 후보에서 제외해 피해를 반영하지 않는다.

최대 타격 수 초과

HitCount >= maxHits에서 추가 타격을 중단하고 소멸 경로로 진입한다.

단발 투사체 설정

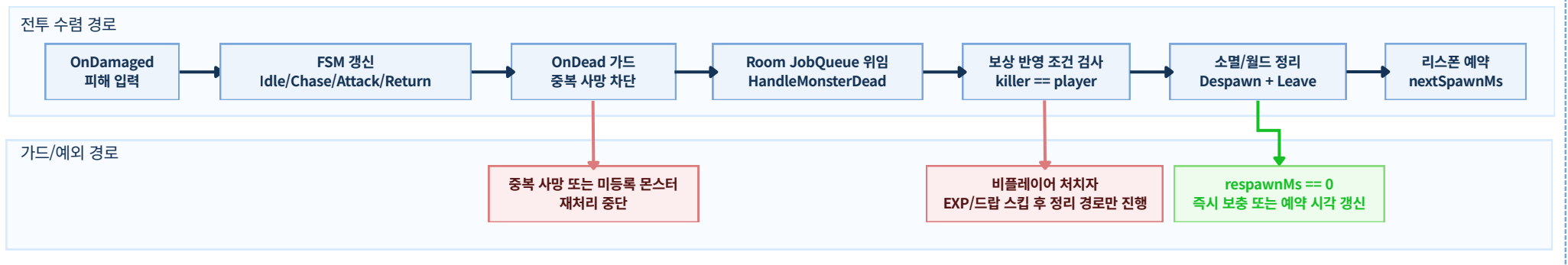
stopOnHit == true 조건에서 첫 타격 후 즉시 소멸 처리한다.

4. 전투 결과 확정 이후 서버 수렴 처리 체계

AI 반응 -> 사망 처리 -> 보상 반영 -> 리스폰 스케줄

▶ 전투 결과가 확정된 뒤에는 AI 상태 전이, 사망 처리, 보상 반영, 리스폰 예약을 같은 서버 기준으로 연결해 처리 일관성을 유지한다.

피해 확정 이후 수렴 흐름



AI 상태 전이 규칙

피해 입력 이후 몬스터가 어떤 상태 전이를 거쳐 행동을 결정하는지 서버 기준으로 정리한다.

피해 입력 반영

OnDamaged에서 공격자를 타겟으로 설정하고 현재 상태가 Idle/Return이면 즉시 추적으로 전환한다.

상태 전이 기준

Tick 루프에서 Idle -> Chase -> Attack -> Return 경로를 유지해 전이 순서를 고정한다.

타겟 유효성 가드

타겟 사망/이탈은 GetTarget 검증에서 차단하고 Return 경로로 수렴시킨다.

공격 실행 조건

공격 상태는 내부 공격 간격 조건을 만족한 경우에만 피해 처리 경로를 호출한다.

차단/가드 처리 기준

중복 처리와 비정상 조건을 어디서 차단하고 어떤 종료 경로로 수렴시키는지 명확히 고정한다.

중복 사망 이벤트

ACTION_DEAD 상태이거나 _monsters에 엔티티가 없으면 재처리를 중단한다.

처치 주체가 비플레이어

killer == nullptr면 EXP/드랍은 건너뛰고 월드 정리와 리스폰 경로만 진행한다.

과스폰 위험

aliveCount >= maxAlive 조건이면 추가 스폰을 차단해 개체 수 상한을 유지한다.

즉시 리스폰 설정

respawnMs == 0인 경우 nextSpawnMs = now로 갱신해 즉시 보충 경로로 수렴한다.

사망 처리/보상 확정 단계

사망 이벤트 이후 보상 반영, 월드 정리, 리스폰 예약까지의 확정 순서를 단일 경로로 관리한다.

중복 사망 차단

OnDead는 ACTION_DEAD 가드로 중복 진입을 막아 사망 처리 1회성을 보장한다.

룸/엔티티 재검증

HandleMonsterDead에서 룸 소속과 _monsters 존재 여부를 다시 확인해 잘못된 정리를 막는다.

보상 반영·동기화

EXP/스탯은 UpdatePlayerCore 저장과 S_CHANGE_STAT 전파를 함께 수행해 상태를 맞춘다.

직렬 처리 위임

사망 후 처리는 Room JobQueue로 위임해 동시 입력 상황에서도 단일 순서로 실행한다.

보상 지급 조건

보상은 killer == player일 때만 적용하고 비플레이어 처치자는 보상 경로를 건너뀀다.

월드 정리·리스폰 예약

OnMonsterDespawnd_ActorOnly -> LeaveMonster 순서로 정리하고 nextSpawnMs를 갱신한다.

Section2 Part D. [맵 전이·파티·인스턴스 던전 정합성 처리]

- 맵 전이는 세션 상태 전이와 BEGIN/ACK 검증을 함께 적용해 실행 순서를 고정
- 파티 변경은 단일 액터 직렬화와 예외 차단으로 중간 상태를 남기지 않도록 정리
- 인스턴스 던전은 생성·입장·종료·최종 정리를 상태 전이 기준으로 일관되게 확정

이번 파트에서 확인할 내용

✔ 맵 변경 요청 확정과 실패 차단·상태 복구 처리

요청 승인 시 서버는 세션에 pending 정보와 BEGIN 토큰을 기록하고, ACK 수신 시 토큰 일치와 상태 전이(WAITING_ACK -> SWITCHING -> NONE)를 함께 검증해 통과 요청만 Room 전이를 실행한다. 중복 요청, 위조/오래된 ACK, 세션·룸 불일치, 목적지 준비 실패는 단계별 가드에서 즉시 차단하며, 실패 요청은 CancelMapChange 경로로 수렴시켜 상태를 NONE으로 복구한다.

✔ 파티 변경 직렬화와 예외 차단으로 정합성을 유지하는 서버 처리

초대/수락/탈퇴/강퇴/해산 요청은 PartyActor 단일 큐에서 직렬 처리해 동시 요청 간 순서 충돌을 막고, 변경 성공 시 version을 증가시켜 동일 기준으로 결과를 전파한다. 던전 전이 중 요청, 권한 불일치, 오프라인 대상 같은 예외는 가드 단계에서 차단하거나 ForceReturn 후속 복구 경로로 묶어 파티 상태와 클라이언트 표시 상태가 어긋나지 않도록 유지한다.

✔ 입장 승인부터 종료 정리까지 인스턴스 던전 수명주기 확정 경로

입장 요청은 전이 잠금 이후 던전 생성 또는 재사용을 확정하고, 파티 상태와 멤버 전이를 함께 갱신해 IN_DUNGEON 상태를 고정한다. 종료 구간은 종료 잠금 후 파티/플레이어 매핑과 던전 상태를 같은 순서로 정리하며, 빈 방·유예 시간 조건을 모두 충족한 경우에만 최종 정리를 수행해 유령 인스턴스와 중복 정리를 방지한다.

1-1. 세션 상태 전이와 ACK 검증으로 확정하는 맵 변경 요청 처리

요청 승인(BEGIN) -> ACK 검증 -> Room 전이 -> 완료(END)

▶ 맵 변경 요청은 pending 정보와 BEGIN 토큰을 기록한 뒤, ACK 검증과 상태 전이를 통과한 경우에만 Room 전이로 확정된다.

맵 이동 방식을 이렇게 설계한 이유

초기 접근	운영에서 드러난 문제	선택한 처리 방식
ACK 없이 즉시 전이	클라이언트/세션 상태가 엇갈리면 완료 시점이 불명확해진다.	BEGIN 토큰 발급 후 ACK 일치 요청만 전이 실행 대상으로 확정한다.
세션·룸 전이를 분리 호출	중간 실패 시 유령 세션, 중복 입장 같은 잔여 상태가 남을 수 있다.	SessionActor -> RoomActor 직렬 체인으로 Leave -> Enter 순서를 고정한다.

전이 착수/완료 조건 체크리스트

착수 조건

- 맵 ID/채널이 유효하고 전이 중복 상태가 아니어야 한다.
- TryBeginMapChange에서 pending과 BEGIN 토큰 발급이 성공해야 한다.
- 요청 세션이 룸 경계 검증을 통과해야 실행 경로에 진입한다.

완료 조건

- TryConsumeMapChangeAck에서 토큰 일치 ACK를 소비해야 한다.
- Leave -> EnterMapChange 실행 성공 후 S_MAP_CHANGE_END를 전파한다.
- EndMapChange 호출로 세션 상태가 NONE에 복귀해야 완료로 본다.

승인·실행 단계별 처리

1 요청 승인

맵 유효성 확인 후 TryBeginMapChange로 pending과 토큰을 고정한다.

2 ACK소비

대기 상태와 토큰이 모두 일치한 요청만 TryConsumeMapChangeAck를 통과시킨다.

3 전이 실행

TransferMapChangeById에서 Leave -> EnterMapChange를 직렬 순서로 실행한다.

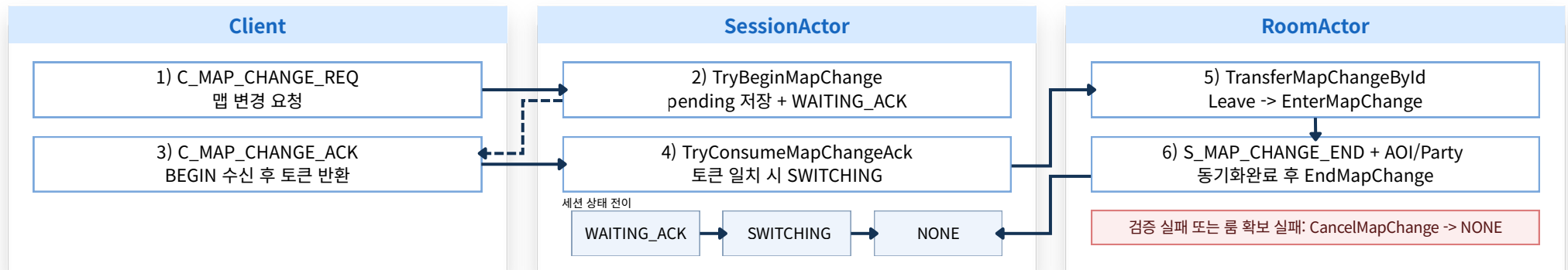
4 예외 정리

룸/세션 불일치 또는 목적지 준비 실패는 CancelMapChange로 즉시 종료한다.

5 완료

S_MAP_CHANGE_END 전파 후 EndMapChange로 상태를 NONE에 복귀시킨다.

맵 변경 승인/전이 흐름



사전 거절 -> 전이 차단 -> 취소 수렴

1-2. 맵 변경 실패 및 차단 처리 기준과 세션 상태 복구

▶ 승인 이후 실패 케이스를 유형별로 분리해 차단하고, 모든 실패를 CancelMapChange와 NONE 복귀로 수렴시켜 세션·룸 상태 불일치가 남지 않게 정리한다.

실패 및 차단 처리 기준

실패 유형을 사전 거절, 전이 차단, 취소 수렴으로 나눠 차단 지점을 앞당기고, 실패 후 상태 복구 경로를 고정한다.

중복 전이 요청일 경우

요청 무시

- **검증 포인트:** IsMapChanging == true 상태인지 확인한다.
- **차단 시점:** BEGIN 재발급 전에 중복 요청을 즉시 차단한다.
- **필요 이유:** 동일 세션의 pending/토큰 중복 생성을 막는다.

-> 진행 중 전이를 유지하고 신규 BEGIN을 발급하지 않는다.

비정상 맵 이동일 경우

사전 거절

- **검증 포인트:** IsValidMapId/IsWorldMapId/MapConfig 통과 여부를 확인한다.
- **차단 시점:** 요청 승인 이전에 목적지 유효성을 먼저 판정한다.
- **필요 이유:** 잘못된 맵/채널 요청이 전이 단계로 유입되는 것을 막는다.

-> BEGIN 토큰을 발급하지 않고 요청을 종료한다.

위조/오래된 ACK일 경우

요청 무시

- **검증 포인트:** TryConsumeMapChangeAck에서 토큰 일치와 대기 상태를 함께 확인한다.
- **차단 시점:** ACK 소비 단계에서 위조/재전송 ACK를 걸러낸다.
- **필요 이유:** 승인되지 않은 전이 실행과 상태 역전을 방지한다.

-> SWITCHING 진입을 차단하고 전이를 실행하지 않는다.

ACK 미수신(타임아웃)일 경우

취소 수렴

- **검증 포인트:** WAITING_ACK 상태에서 ACK 대기 시간이 임계값을 넘었는지 확인한다.
- **차단 시점:** ACK 소비 단계에 진입하기 전에 타임아웃 요청을 정리한다.
- **필요 이유:** 끊긴 클라이언트나 유실 ACK로 인한 장기 대기 상태를 방지한다.

-> CancelMapChange를 호출하고 상태를 NONE으로 복귀시킨다.

전이 중 세션/룸 불일치일 경우

취소 수렴

- **검증 포인트:** playerId == 0 여부와 룸 경계 검증 결과를 확인한다.
- **차단 시점:** 전이 실행 직전/중간에 세션 유효성을 재검증한다.
- **필요 이유:** 유령 세션, 중복 입장, 잘못된 룸 잔류를 예방한다.

-> CancelMapChange로 정리하고 상태를 NONE으로 복귀시킨다

목적지 룸 준비 실패일 경우

취소 수렴

- **검증 포인트:** GetOrCreateLobby/Room 호출 성공 여부를 확인한다.
- **차단 시점:** EnterMapChange 이전에 목적지 준비 상태를 점검한다.
- **필요 이유:** 부분 전이 상태를 남기지 않고 실패 수렴 경로를 유지한다.

-> CancelMapChange를 호출해 부분 전이 없이 종료한다.

세션 상태 복구 보드

착수 조건

BEGIN 발급 이후 ACK 대기 상태에서는 신규 BEGIN을 차단하고 기존 pending 전이 정보를 유지한다.

SWITCHING 보호

ACK 검증 통과 후 Leave -> EnterMapChange 실행 중에는 중간 입력을 차단해 전이 직렬성을 유지한다.

NONE 수렴 고정

실패는 CancelMapChange, 성공은 EndMapChange로 종료하고 최종 상태를 항상 NONE으로 맞춘다.

2. 파티 변경 직렬화와 예외 차단으로 정합성을 유지하는 서버 처리

초대 -> 수락 -> 멤버 변경 -> 던전 연동 예외 처리까지 단일 경로

▶ 파티 변경은 PartyActor에서 직렬 처리하고, 던전 전이 구간은 상태 잠금으로 경합을 차단한다.

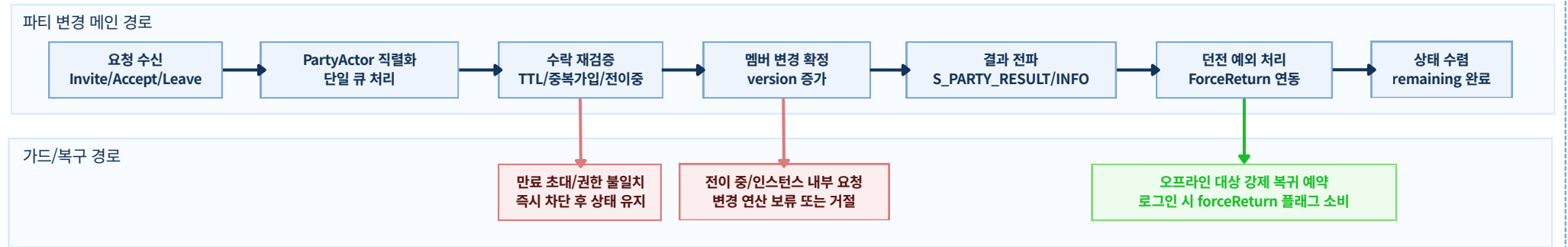
이 구조를 사용한 이유

- 파티 변경을 단일 액터에서 직렬화하고 던전 전이·오프라인·권한 불일치 예외를 같은 경로에서 처리해, 동시 요청과 실패 상황에서도 중간 상태가 남지 않게 하기 위해서다.
- 버전 기준 전파와 후속 복구(ForceReturn)를 함께 묶어, 클라이언트 표시 상태와 서버 실제 상태를 같은 기준으로 일관되게 맞추기 위해서다.

기본 처리 경계와 실행 순서

- 모든 변경 연산(Create/Invite/Accept/Leave/Kick/Disband)은 PartyActor::Push 단일 경로로 직렬화한다.
- 조회/연산은 PartyActor 스레드에서 수행하고, 실제 송신은 Session 스레드에서 처리한다.
- IsMapChanging() 가드 대상 요청은 전이 중 조작을 사전 차단한다.

파티 정합성 제어 흐름



1) 초대 수명주기와 수락 검증

- 초대장은 targetId 기준 단일 pending 으로 관리하며, 재초대 시 기존 pending을 교체한다.
- 유효시간은 GetTickCount64 + 60초이며 만료 즉시 폐기한다.
- 거절 처리에서는 pending만 제거하고 파티 본문 상태는 유지한다.

2) 멤버 변경 규칙과 버전 동기화

- 변경 연산 성공 시 version을 증가시켜 동기화 기준으로 사용한다.
- 리더 탈퇴 시 남은 멤버 중 최소 ID로 리더 권한을 위임한다.
- BroadcastPartyInfo와 S_PARTY_RESULT.version을 동일한 버전 기준으로 유지한다.

3) 던전 연계 예외 처리

- 던전 내부 탈퇴/강퇴/해산은 PartyActor가 InstanceActor 정리(EjectMember/Close*)를 순차 실행한다.
- 정리 완료 후 ForceReturnToWorld로 대상 세션의 월드 복귀를 강제한다.

실패/예외 차단 매트릭스

실패/예외 상황	서버 검증/가드	처리 결과
만료 초대/유효하지 않은 수락	pending 미존재 또는 TTL 만료	수락 거절, pending 폐기
중복 가입 요청	이미 파티 소속 또는 대상 상태 불일치	멤버 변경 미적용
전이/인스턴스 중 변경 요청	inDungeonTransition 또는 instanceId != 0	요청 차단, 기존 상태 유지
권한 없는 강퇴/해산	리더 권한 재검증 실패	실패 응답, 상태 변경 없음
조회 중 세션/룸 누락	세션 없음, 룸 없음, GameRoom 변환 실패	remaining 감소 후 최종 응답 수렴
오프라인 대상 던전 이탈	즉시 복귀 불가(세션 없음)	MarkForceReturn 기록 후 로그인 시 복귀

3. 입장 승인부터 종료 정리까지 인스턴스 던전 수명주기 확정 경로

입장 승인 -> 생성/재사용 -> 플레이 -> 종료 전이 -> 최종 정리

▶ 인스턴스 던전은 입장부터 종료 정리까지를 상태 잠금 기준으로 순차 확정해, 동시 요청 구간의 중복 생성과 미정리 상태를 방지한다.

1) 생성/입장 확정 경로

1. 입장 요청 수신 후 TryBeginDungeonTransition(ENTERING)으로 전이를 잠근다.
2. CreateOrGetForParty로 인스턴스를 생성하거나 기존 인스턴스를 재사용한다.
3. SetPartyInstance와 EndDungeonTransition(IN_DUNGEON)으로 파티 상태를 확정한다.
4. 멤버별 MapChangeBegin을 시작해 인스턴스 맵으로 전이한다.

2) 종료/정리 확정 경로

1. 퇴장 요청은 TryBeginDungeonTransition(EXITING)으로 종료 전이를 잠근다.
2. CloseForParty 성공 시 파티/플레이어 역인덱스와 인스턴스 맵을 함께 정리한다.
3. ClearPartyInstance와 EndDungeonTransition(NONE)으로 기본 상태로 복귀한다.
4. MarkClosing(true) 이후 조건 충족 시 PurgeInstanceRooms로 최종 정리한다.

인스턴스 던전 수명주기 흐름

메인 처리 경로



차단/예외 수렴 경로

중복 입장 또는 전이 중 요청
입장 거절, 기존 상태 유지

종료 처리 실패
상태 롤백(IN_DUNGEON) 후 실패 응답

오프라인 잔존/타임아웃
Close 후 빈 방 + 유예 시간 충족 시 PurgeInstanceRooms

종료 조건 및 예외 처리 기준표

상황	서버 처리 기준	결과
입장 중복 요청	inDungeonTransition 또는 기존 instancelId	입장 거절(중복 생성 방지)
던전 외부 퇴장 요청	instancelId == 0	실패 응답 후 종료
종료 처리 실패	CloseForParty 실패	상태 롤백(IN_DUNGEON)
마지막 멤버 오프라인	OnMemberOffline -> empty	인스턴스 종료 + 닫힘 마킹
제한시간 초과	CollectExpired(now)	종료 후 닫힘 마킹
빈 방 유예 미충족	ShouldPurge 불충족	인스턴스 종료 + 닫힘 마킹

상태/매핑 정리 기준

구분	입력	정리 결과
파티 기준 매핑	partyId -> instancelId	생성/종료 시점에 동기 갱신
플레이어 역인덱스	playerId -> instancelId	퇴장/강퇴/오프라인에서 제거
파티 상태	ENTERING / IN_DUNGEON / EXITING / NONE	전이 완료 상태를 단일 기준으로 유지
룸 정리 신호	MarkClosing(true)	PurgeInstanceRooms 대상 편입

- **최종 정리(Purge) 조건:** 인스턴스 룸, 닫힘 상태, 인원 0명, 유예 시간(10초) 경과를 모두 충족해야 한다.
- **핵심 원칙:** 상태 전이와 매핑 정리를 분리하지 않고 같은 트랜잭션 경계에서 처리한다.

Section2 Part E. [거래 원자성 보장과 실패 수렴 검증]

- 거래 요청은 상태별 허용 동작으로 제어해 동시 조작과 중간 개입을 차단
- 최종 확정은 사전 계획과 단일 DB 트랜잭션, 응답 매칭 검증으로 원자성 확보
- 성공/실패 케이스를 로그·응답 기준으로 반복 검증해 결과 일관성 확인

이번 파트에서 확인할 내용

✓ 1. 거래 요청의 상태 전이 검증과 실패 수렴 처리 기준

거래 요청은 Invited -> Active -> Locked -> Committing 순서로만 진행되며, 각 상태에서 허용된 입력만 처리한다.
검증 실패, 사용자 취소, 맵 전이, 접속 종료, 내부 오류는 공통 취소 경로로 수렴시켜 중간 반영 없이 종료한다.

✓ 2. 거래 확정 원자성을 위한 2단계 커밋 검증·반영 절차

확정 전에는 commitPlan과 requestId를 생성해 반영 대상과 요청 단위를 먼저 고정한다.
DB 처리 결과를 받은 뒤에는 state == Committing, requestId 일치, 계획 존재 여부를 함께 확인해 통과 건만 최종 반영하고 실패는 내부 취소로 정리한다.

1. 거래 요청의 상태 전이 검증과 실패 수렴 처리 기준

Invited -> Active -> Locked -> Committing

▶ 거래 요청은 Invited -> Active -> Locked -> Committing 순서로만 전이되며, 실패나 충돌은 모두 공통 취소 경로로 정리된다.

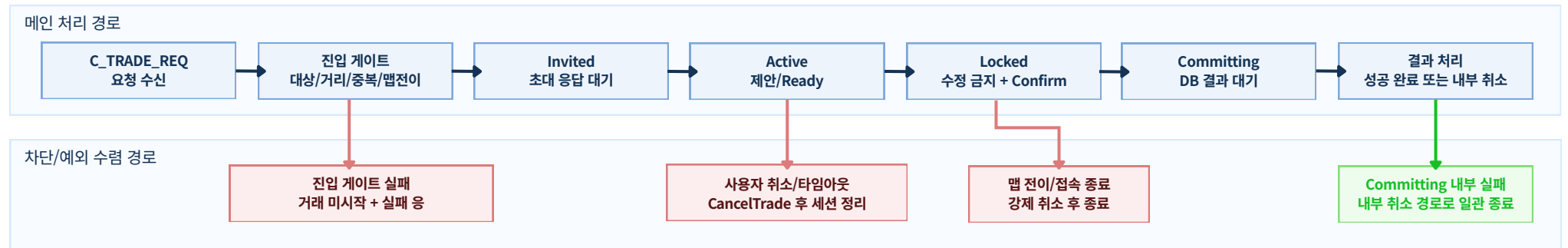
교환 방식을 이렇게 선택한 이유

초기 시도	문제점	현재 방식
요청마다 즉시 반영	양측 입력이 엇갈리면 중간 상태가 노출됨	Active -> Locked -> Committing 단계로 확정 시점을 분리
취소를 모든 시점에 허용	DB 반영 중 취소가 들어오면 결과 불일치 위험	Committing 구간에서는 사용자 취소를 차단
예외를 후처리로 복구	맵 전이/종료 시 거래 잔존 가능	맵 전이·접속 종료·타임아웃을 강제 취소 경로로 통합

상태별 책임과 허용 동작

상태	상태 의미	허용 동작	허용 동작
Invited	초대 응답 대기	수락/거절 응답	아이템·골드 제안, Confirm
Active	제안 조정 구간	아이템/골드 제안, Ready 토글	Confirm(잠금 전), 커밋
Locked	제안 확정 구간	Confirm 입력	Offer/Gold 변경
Committing	DB 결과 대기	결과 수신 후 완료/내부 취소	사용자 취소, 제안 변경

인스턴스 단점 수명주기 흐름



전이 게이트와 실패 처리

전이	진입 조건	허용 동작
None -> Invited	대상/거리/중복 거래/맵전이 상태 검증 통과	실패 응답 후 미진입
Invited -> Active	수락 응답 + 양측 플레이어 유효	거절/유실 시 세션 취소
Active -> Locked	양측 Ready == true	조건 미충족 시 Active 유지
Locked -> Committing	양측 Confirm == true	조건 미충족 시 Locked 유지
Committing -> End	DB 성공 또는 내부 실패 확정	성공 완료 또는 내부 취소

강제 취소 트리거

트리거	처리 기준	결과
사용자 취소	state != Committing에서만 허용	CancelTrade 후 종료
거래 타임아웃	kTradeTimeoutMs 초과, 단 Committing 제외	TRADE_CANCEL_TIMEOUT
맵 전이 시작	맵 이동 처리 진입 시 활성 거래 존재	TRADE_CANCEL_MAP_CHANGE
접속 종료/룸 이탈	Leave 경로에서 활성 거래 존재	TRADE_CANCEL_DISCONNECT
내부 오류	플레이어 누락, 상태 불일치, DB 실패 등	TRADE_CANCEL_INTERNAL

2. 거래 확정 원자성을 위한 2단계 커밋 검증·반영 절차

사전 검증 계획 수립 -> DB 단일 트랜잭션 -> 응답 매칭 후 최종 반영

▶ 거래 확정 단계에서는 한쪽만 반영되는 부분 성공을 차단해야 한다. commitPlan, requestId, state == Committing 조건이 동시에 맞을 때만 거래 결과를 최종 반영한다.

1) 왜 2단계 커밋으로 바꿨는가 (시행착오)

시도 단계	당시 방식	실제 문제
초기 구현	사용자 입력마다 메모리 상태를 먼저 반영하고 마지막에 DB 저장	중간 실패 시 한쪽만 반영된 상태가 남고 복구 분기가 늘어남
1차 개선	DB 처리 후 결과 응답만 보고 즉시 적용	지연된 이전 응답(stale)이 뒤늦게 들어와 오래된 결과를 덮어쓸 수 있음
2차 개선	아이템/골드 연산을 쿼리 단위로 분리 실행	일부 쿼리만 성공하면 자산 정합성이 깨지고 원복이 어려움
최종 구조	Phase 1 계획 수립 -> Phase 2 단일 트랜잭션 -> 응답 매칭 후 반영	실패 케이스가 여러 형태로 들어옴 (검증 실패, DB 실패, 응답 불일치)

4) 실패 수렴 매트릭스

실패 지점	서버 처리	보장 결과
Phase 1 사전 검증 실패	DB 요청 생략 + CancelTrade(INTERNAL)	중간 반영 없음
DB 세션 미존재	commitPlan 정리 후 실패 반환	in-flight 잔존 방지
DB 트랜잭션 실패	ROLLBACK + fail 응답	반영 원자성 유지
requestId 불일치	stale 응답 무시	오래된 응답 재적용 방지
적용 대상 누락/상태 불일치	내부 취소 경로 실행	부분 반영 차단

2) 단계별 실행·검증 책임 (GameServer / DBAgent)

Phase 1 GameServer

계획 수립

BuildTradeCommitPlan으로 골드/아이템/슬롯을 스냅샷 검증하고 commitPlan, requestId를 생성한다.

반영 기준: 검증 실패 시 DB 요청 없이 CancelTrade(INTERNAL)로 종료한다.

Phase 2 DBAgent

DB 확정

BEGIN TRAN에서 DELETE/UPSERT/UPDATE를 한 트랜잭션 경계로 처리하고 결과를 S2S_RES_TRADE_COMMIT으로 응답한다.

반영 기준: 검증 실패 시 DB 요청 없이 CancelTrade(INTERNAL)로 종료한다.

Guard GameServer

최종 반영 전 확인

응답 수신 후 state == Committing, requestId 일치, commitPlan 존재를 먼저 확인한다.

반영 기준: 불일치(stale 포함)는 응답 무시 또는 내부 취소로 처리한다.

Apply GameServer

응답 적용/정리

검증 통과 + 성공 응답에서만 메모리/Redis/클라이언트 delta를 반영하고 거래 세션을 정리한다.

반영 기준: 부분 반영 없이 성공 확정 또는 실패 취소의 단일 종료만 허용한다.

거래 확정 2단계 커밋 흐름

Phase 1: 확정 계획 생성 (GameServer)

정상 경로



실패 경로

사전 검증 실패
DB 요청 없이 CancelTrade(INTERNAL)

Phase 2: DB 확정과 응답 적용 (DBAgent -> GameServer)

정상 경로



실패/가드 경로

DB 실패 응답
CancelTrade(INTERNAL)로 수렴

state/requestId/commitPlan 불일치
stale 응답 무시 또는 내부 취소

Section3 Part A. [실행 구조와 동시성 제어]

- Actor 경계와 JobQueue 직렬화로 상태 반영 순서를 고정
- 직렬화 실행 예시와 운영 보안을 코드 근거와 함께 제시
- Hybrid 스레드 모델로 지연 전파를 줄이고 병목 위치를 분리

이번 파트에서 확인할 내용

✓ 1. Actor 경계와 JobQueue 직렬화로 고정한 서버 상태 반영 경로

동시 입력 환경에서 상태 변경 경로를 Actor 큐로 제한해 요청 처리 순서가 뒤섞이지 않도록 고정한다.
수신 스레드와 상태 반영 스레드를 분리해 경합 지점을 줄인다.

✓ 2. Actor + JobQueue 직렬화 처리 예시와 운영 보안

GameRoom 기준 요청 1건이 수신부터 반영까지 어떤 순서로 실행되는지 함수 단위로 추적하고,
운영 중 확인된 직렬화 한계를 코드 가드로 보완한 기준을 정리한다.

✓ 3. Hybrid 스레드 모델로 지연 전파를 줄인 실행 구조

네트워크 처리, 로직 실행, 주기 업데이트를 분리해 I/O 급증이 전체 지연으로 번지지 않게 구성한다.
병목 구간을 영역별로 나눠 확인 가능한 구조를 설명한다.

1. Actor 경계와 JobQueue 직렬화로 고정한 서버 상태 반영 경로

Session/Lobby/Game Room을 Actor 경계로 분리하고, 상태 반영은 각 Actor 큐의 순차 실행으로 제한해 처리 순서를 고정한다.

▶ 이동·스킬·거래 요청이 겹쳐도 최종 상태 변경은 하나의 순차 경로에서만 처리되도록 했다. 요청 수신과 상태 반영을 분리해, 동시 입력 구간에서도 처리 순서가 섞이지 않게 했다.

왜 필요한가

동시 입력 환경에서 상태가 엇갈릴 수 있는 지점을 먼저 정리하고, Actor + JobQueue 기준으로 상태 변경 흐름을 일관되게 관리했다.

실제 운영에서 확인한 문제	영향	설계 방향
입력 도착 순서와 처리 순서가 어긋남	좌표·전투 판정 기준이 요청마다 달라질 수 있음	요청 수신 경로와 상태 변경 경로를 나누고, 최종 반영은 Actor 큐 내부에서만 처리
잠금 범위를 넓혀 동시성 문제에 대응	기능이 늘수록 충돌 지점과 디버깅 비용이 함께 늘어남	잠금 범위를 다시 설계하기보다 메시지 전달 방식으로 확장
세션 종료와 룸 작업이 동시에 발생	세션 매핑이 남거나 룸 정리가 누락될 위험	종료 처리도 같은 큐 순서에 합류시켜 정리 기준을 일원화
기능 추가 때마다 동기화 규칙이 분산	회귀 위험과 유지보수 비용 증가	Actor별 상태 변경 책임을 나눠 변경 영향 범위를 축소

요청이 겹칠 때의 처리 기준

겹치는 요청 상황	적용한 기준	결과
이동 + 스킬 동시 입력	룸 Actor 큐에서 순차 실행	상태 반영 순서 일관 유지
거래 + 맵 전이 동시 처리	상태 전이 가드 + 룸 내부 반영	중복 반영/누락 방지
세션 종료 + 잔여 작업 처리	LeaveById 단일 정리 경로	후처리 누락 감소

Actor별 쓰기 책임 분리

- 상태 변경은 패킷 수신 스레드에서 바로 처리하지 않고, 상태를 소유한 Actor 큐에서만 반영되도록 설계했다.
- PlayerSession/LobbyRoom/GameRoom/PartyActor/InstanceActor가 맡는 상태 변경 범위를 나눠, 동시 요청이 들어와도 반영 순서를 일관되게 유지했다.

Actor	소유 상태	주요 역할
PlayerSession Actor	접속/인증/전이 플래그	패킷 수신 후 PostRoom 위임, 전이 가드 검사
LobbyRoom Actor	로비 입장/매칭 대기 상태	로비 기준 검증 후 게임룸 연결 경로 결정
GameRoom Actor	좌표/전투/인벤토리	Push된 작업을 순차 실행해 최종 상태 반영
PartyActor	파티 상태/해산/강퇴	파티 관련 변경 요청을 단일 큐에서 순차 처리
InstanceActor	인스턴스 생성/만료/정리	인스턴스 수명주기 변경을 단일 큐에서 순차 처리

이 구조에서 지키는 실행 원칙

- 패킷 수신 스레드는 상태를 직접 수정하지 않고 Actor 큐로 전달만 한다.
- Actor 경계를 넘는 변경은 메시지 전달로만 연결해 동시 수정 충돌 가능성을 줄인다.
- 연결 종료 정리도 같은 큐 순서에 포함해 후처리 누락을 줄인다.
- 기능 추가 시 잠금 재설계보다 Actor 메시지 경로 확장으로 대응한다.

2. Actor + JobQueue 직렬화 처리 예시와 운영 보완

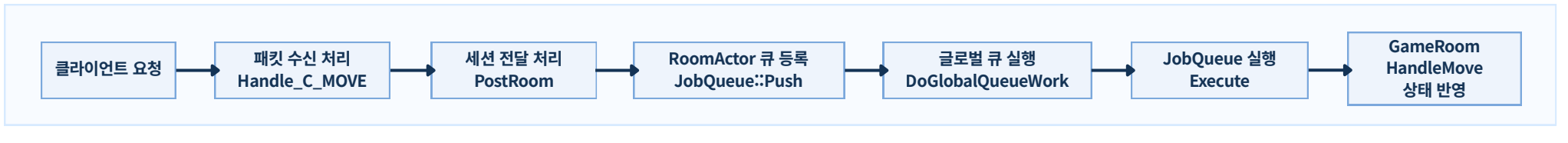
요청이 처리되는 흐름과 운영 중 보완한 기준을 나눠 보여주고, 직렬화 구조가 실제 코드에서 어떻게 동작하는지 설명한다.

▶ GameRoom 요청 1건이 수신부터 반영까지 어떤 순서로 실행되는지 따라가고, 운영 중 드러난 직렬화 한계는 코드 가드로 보완했다.

운영 중 확인한 한계와 보완 방법

한계 상황	보완 방법	적용 효과
큐 적체 지연 요청이 몰리면 처리 대기 시간이 길어진다.	_posted와 ObserveJobQueueDepth/Wait/Exec로 중복 스케줄링과 큐 적체를 함께 관리한다.	큐가 몰리는 구간을 조기에 확인하고 작업 대기를 관리할 수 있다.
전이 중 작업 유입 맵전이 요청과 일반 요청이 섞이면 상태 순서가 뒤바뀔 수 있다.	Handle_C_MOVE/PostRoom에서 IsMapChanging() 재검증으로 전이 중 요청을 차단한다.	전이 구간의 상태 오염과 순서 뒤바뀜을 줄인다.
접속 종료와 작업 겹침 접속 종료와 잔여 작업이 동시에 발생하면 정리 누락 위험이 있다.	PushJob(&GameRoom::LeaveById)와 OnMemberOffline를 같은 정리 흐름으로 묶어 누락 가능성을 줄인다.	잔여 매핑/고아 상태 발생 빈도를 낮춘다.

요청 1건이 상태에 반영되는 흐름(GameRoom 기준)



흐름도 단계 해설(GameRoom 예시)

1. 수신 단계와 전이 확인

흐름도 구간: 클라이언트 요청 → 패킷 수신 처리

핵심 처리: 세션/플레이어 식별을 확인하고 전이 중 요청을 먼저 차단한다.

3. 워커 스레드 순차 실행

흐름도 구간: 글로벌 큐 실행 → JobQueue 실행

핵심 처리: 등록된 작업을 큐 순서대로 실행해 요청 간 적용 순서를 고정한다.

2. Actor 전달 및 큐 등록

흐름도 구간: 세션 전달 처리 → RoomActor 큐 등록

핵심 처리: 수신 스레드에서 상태를 직접 수정하지 않고 작업만 위임한다.

4. GameRoom 최종 반영

흐름도 구간: GameRoom 상태 반영

핵심 처리: 검증을 통과한 요청만 최종 상태에 반영하고 후속 전파를 이어간다.

수신·전달·실행·반영을 하나의 직렬 경로로 묶어, 동시 입력에서도 상태 반영 순서가 뒤집히지 않게 유지한다.

3. Hybrid 스레드 모델로 자연 전파를 줄인 실행 구조

네트워크 처리, 로직 실행, 주기 업데이트를 분리해 입력이 몰리는 상황에서도 자연이 전체로 번지지 않게 구성했다.

▶ Hybrid 스레드 모델은 네트워크 처리(완료 통지 소비), 로직 실행(JobQueue 처리), 주기 업데이트(Main Tick)를 분리해 각 실행 영역의 역할을 고정한 구조다.

설계 이유

자연 확산 차단과 원인 추적

단일 루프에서는 I/O 급증이 로직 지연으로 바로 번졌고 원인 구분도 어려웠다. 실행 영역을 나눠 I/O 구간 문제인지 로직 구간 문제인지 빠르게 확인하도록 설계했다.

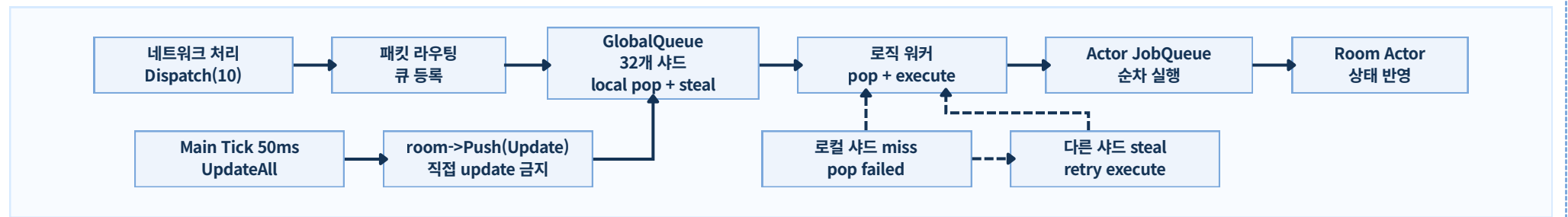
부하 패턴별 튜닝 가능

서버 코어 수에 맞춰 네트워크/로직 스레드 비율을 독립적으로 조정해, 피크 구간과 평시 구간에 맞춘 대응이 가능하도록 했다.

주기 업데이트 안정성 확보

Main Tick은 50ms 주기를 유지하고, 룸 업데이트는 Actor 큐를 통해 반영하도록 분리해 업데이트 타이밍 흔들림을 줄였다.

3개 실행 영역의 흐름



스레드 수 분할 기준

01

- 기준값 `threadCount = hardware_concurrency``, 최소 2로 보정
- 네트워크 스레드 `networkThreadCount = threadCount / 2``
- 로직 스레드 `logicThreadCount = threadCount - networkThreadCount``

로직 작업 분배 방식

02

- 32개 샤드 분할 GlobalQueue를 shard 단위로 나눠 특정 큐에 작업이 몰리는 현상을 완화했다.
- 로컬 우선 처리 워커는 `LThreadId % shardCount``에 해당하는 샤드를 먼저 pop한다.
- 부족 시 steal 로컬 큐가 비면 다른 샤드를 순회하며 steal해 유휴 시간을 줄인다.

런타임 동작 순서

03

1. 서버 시작 시 실행 영역별 스레드 수를 계산하고 워커를 생성한다.
2. 네트워크 스레드가 `Dispatch(10)`` 루프로 완료 통지를 처리한다.
3. Main Tick 스레드가 50ms 주기로 `UpdateAll``을 호출해 업데이트 작업을 등록한다.
4. 로직 스레드가 GlobalQueue pop + `JobQueue::Execute``를 수행한다.

실행 영역별 역할

04

- 네트워크 영역 완료 통지 소비와 라우팅만 수행하고 상태는 직접 수정하지 않는다.
- 로직 영역 Actor JobQueue를 실행해 실제 상태 반영을 처리한다.
- Main Tick 영역 `room->Update()`` 직접 호출 대신 `room->Push(...)``로 작업만 적재한다.

Section3 Part B. [패킷/IO 신뢰 경로와 완료 처리]

- 패킷 경계 확정과 공통 검증 체인으로 진입 품질 고정
- IOCP 완료 통지 기반 분기, 재등록, 객체 수명 규칙 정리
- Recv·Send·Disconnect 처리와 종료 수렴 경로를 일관되게 통합

이번 파트에서 확인할 내용

✓ 1. 패킷 진입 통제와 무결성 검증을 공통 경로로 고정한 구조

패킷 경계 확인, 허용 ID 분기, CRC/Seq/XOR/Parse 검증을 하나의 공통 경로로 묶어 기능이 늘어나도 동일한 검증 품질을 유지하도록 설계한다.

✓ 2. IOCP 선택 배경과 완료 통지 기반 처리 흐름 설계

완료 통지 분기 기준과 Accept/Recv/Send 재등록 규칙을 한 흐름으로 고정해 누락 없는 I/O 루프를 유지한다. 객체 참조 해제 시점까지 함께 고정해 안정성을 확보한다.

✓ 3. 완료 통지 이후 Recv·Send·Disconnect 처리와 종료 수렴 경로

완료 통지 이후 경로별 처리 순서를 분리해 어디서 재등록하고 어디서 종료로 수렴하는지 명확히 보여준다. 예외 상황에서도 같은 정리 순서가 유지되도록 만든 기준을 설명한다.

✓ 4. 메모리 풀과 송신 버퍼 재사용으로 실행 비용 변동을 줄인 구조

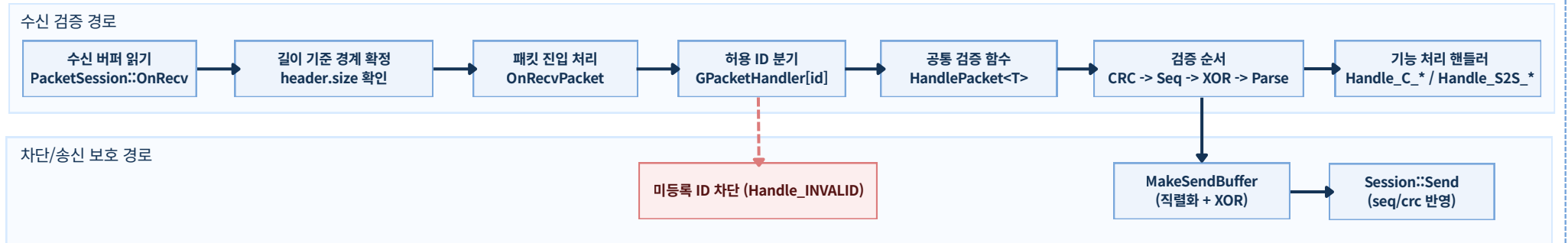
객체 할당과 send buffer 생성 비용을 재사용 경로로 묶어, 피크 구간에서도 공통 런타임 비용이 먼저 흔들리지 않도록 만든다.

1. 패킷 진입 통제와 무결성 검증을 공통 경로로 고정한 구조

개별 기능 코드마다 검증 품질이 달라지지 않도록, 수신부터 전송까지 같은 검증 순서를 공통 경로로 고정했다.

▶ 패킷 경계 확정, 허용 ID 분기, 무결성 검증, 송신 헤더 보호를 하나의 경로로 묶어 신규 기능이 추가돼도 같은 규칙으로 안전하게 처리되도록 설계했다.

패킷 공통 처리 흐름



수신 데이터에서 패킷 경계를 어떻게 확인하는가

- **경계 확인** `header.size` 기준으로 패킷 길이를 먼저 확인한다.
- **조각 보관** 헤더/본문이 덜 온 데이터는 다음 수신까지 버퍼에 유지한다.
- **완성본만 전달** 경계가 확인된 데이터만 `OnRecvPacket`으로 넘긴다.
- **미완성 데이터 차단** 완성 전 데이터는 핸들러 단계로 진입하지 않는다.

들어오는 패킷을 어떤 순서로 검증하는가

- 1단계** 경계가 확인된 패킷만 `OnRecvPacket`으로 진입시킨다.
- 2단계** 허용 목록에 있는 ID인지 먼저 확인한다.
- 3단계** `CRC + Seq`로 무결성과 순서를 검증한다.
- 4단계** `XOR + Parse` 성공 시에만 실제 핸들러를 호출한다.

허용되지 않은 패킷을 어떻게 막는가

- **기본 차단** 모든 ID를 `Handle\_INVALID`로 먼저 설정한다.
- **허용 ID 등록** 실제로 허용할 ID만 `Init()`에서 연결한다.
- **미등록 차단** 등록되지 않은 ID는 기본 차단 경로로 빠진다.
- **누락 즉시 확인** 신규 ID를 등록하지 않으면 즉시 차단되어 누락이 드러난다.

나가는 패킷을 어떻게 보호하는가

- **직렬화 + XOR** `MakeSendBuffer`에서 payload를 구성하고 XOR을 적용한다.
- **전송 직전 보호** `Session::Send`에서 `seq/crc`를 넣어 최종 패킷을 완성한다.
- **역할 분리** 기능 코드는 메시지 구성, 전송 코드는 보호 필드 작성을 담당한다.
- **공통 규칙 유지** 모든 송신 경로가 동일한 보호 정책을 사용한다.

검증 실패 시 실제 처리 기준

패킷 미완성 헤더/본문이 덜 도착한 경우	→	`header.size` 기준으로 완성 전 데이터는 버퍼에 유지하고, 검증/핸들러 단계로 넘기지 않는다.
미등록 패킷 ID	→	허용 목록에 없는 ID는 `Handle_INVALID`로 분기해 기능 처리 핸들러 진입을 차단한다.
CRC/Seq 검증 실패 무결성/순서 불일치	→	공통 검증 단계에서 즉시 중단하고 실제 처리 함수는 호출하지 않는다.
XOR/Parse 실패 복호화·파싱 불가	→	검증 단계에서 중단해 핸들러를 호출하지 않으며, 상태 변경 없이 요청을 종료한다.

2. IOCP 선택 배경과 완료 통지 기반 처리 흐름 설계

완료 통지 이후 어떤 작업을 다시 등록하고 어떤 경우 종료하는지, 처리 기준을 하나의 흐름으로 맞춘다.

▶ 완료 통지를 받는 지점(GQCS)을 시작점으로, 이벤트 분기 기준, 재등록 규칙, 객체 참조 해제 순서를 한 흐름 안에서 설명한다.

IOCP를 선택한 이유

대상 플랫폼 일치

배포 환경이 Windows라 Winsock Overlapped + IOCP 경로를 그대로 활용하는 편이 운영 리스크가 낮았다.

완료 통지 분기 기준 통일

GQCS 한 지점에서 성공, WAIT_TIMEOUT, 오류를 같은 기준으로 처리해 장애 재현과 추적 경로를 단순화할 수 있었다.

객체 참조 수명 관리가 명확함

event owner 참조를 완료 통지 처리와 함께 관리해, 객체 생존 보장과 참조 해제 시점을 코드로 명확히 고정하기 쉬웠다.

IOCP를 직접 구현한 이유

재등록 타이밍 고정

Accept/Recv/Send 재등록 시점을 엔진 코드에서 직접 고정해 완료 통지 누락 구간을 줄일 수 있었다.

수명 규칙 강제

이벤트별 owner 획득/해제 규칙을 코드로 강제해 완료 지연 구간의 해제된 객체 접근 (use-after-free) 위험을 낮췄다.

종료 기준 명시

CloseAccept 가드, _sendRegistered 게이트, Disconnect 중복 차단 조건을 코드로 고정해 종료 동작이 흔들리지 않도록 했다.

IOCP 실행 경로 미리보기 ▼

1. 완료 통지 수신

GQCS에서 완료 통지를 받고 모든 처리를 같은 시작점에서 진행한다.

2. 이벤트 타입 분기

Accept/Recv/Send/Disconnect 중 어떤 완료인지 확인해 다음 처리 함수를 결정한다.

3. 재등록 또는 종료 처리

계속 받아야 하는 작업은 재등록하고, 종료는 owner 참조 해제 규칙에 맞춰 정리한다.

Accept 등록 유지와 GQCS 분기 처리

Accept 등록 유지

- 초기 등록 StartAccept에서 최대 세션 수만큼 AcceptEvent를 미리 등록한다.
- 연결 검증 후 활성화 SetUpdateAcceptSocket/getpeername 성공 시 ProcessConnect로 세션을 활성화한다.
- 검증 실패 처리 위 검증 단계가 실패해도 즉시 RegisterAccept로 되돌려 Accept 처리가 비지 않게 유지한다.
- 종료 보호 CloseAccept가 _socket=INVALID_SOCKET으로 바꿔 종료 중 재등록 루프를 차단한다.

GQCS 결과별 처리

- 성공 완료 owner->Dispatch(event, bytes)로 바로 전달한다.
- WAIT_TIMEOUT 이벤트 미수신으로 처리하고 false를 반환해 다음 tick으로 넘어간다.
- 기타 오류 OVERLAPPED가 유효하면 동일하게 owner->Dispatch(event, bytes)로 넘겨 객체별 정리 경로를 타게 한다.

객체 참조 수명과 세션 활성화 규칙

구간	코드 규칙	코드 규칙
Register*	`owner=shared_from_this()`	완료 통지 시점까지 객체 생존 보장
Process*	`owner=nullptr`	완료 처리 직후 참조를 해제해 수명 정리
ProcessConnect	`AddSession -> OnConnected -> RegisterRecv`	Accept된 세션을 서비스 활성화 상태로 전환

- Send 동시 실행 제한** _sendRegistered.exchange(true)로 동시에 하나의 WSASend만 진행 (in-flight)되도록 제한한다.
- 실패 시 복구** WSASend가 WSA_IO_PENDING 외 오류면 _sendRegistered=false로 복구해 다음 등록이 가능하도록 만든다.
- Disconnect 중복 차단** _connected.exchange(false)로 종료 등록 경로를 1회로 고정한다.

3. 완료 통지 이후 Recv·Send·Disconnect 처리와 종료 수렴 경로

완료 통지 이후 어떤 경우 재등록하고 어떤 경우 종료하는지 분리해, 처리 순서가 끊기지 않도록 구성했다.

▶ 완료 통지 이후 Recv/Send를 언제 다시 등록하고, 어떤 조건에서 Disconnect로 종료하는지 실행 기준을 한 흐름으로 보여준다.

Recv 완료 처리와 재등록 기준

1. `numOfBytes == 0` 이면 상대가 연결을 닫은 것으로 보고 즉시 `Disconnect` 로 종료한다.
1. 정상 수신이면 받은 바이트를 반영하고 읽기 위치(`cursor`)를 전진한 뒤 `Clean` 으로 버퍼를 정리한다.
2. 버퍼 경계 검증이 실패하면 중간 상태를 남기지 않고 `Disconnect` 로 종료한다.
3. 정상 처리 후 `RegisterRecv` 를 재호출해 수신 루프를 유지한다.
4. 루프 재진입은 완료 통지마다 동일한 `owner->Dispatch` 경로로 처리된다.

Send 처리와 Disconnect 종료 기준

1. `Send` 진입 시 `\_sendRegistered.exchange(true)` 를 통과한 1회 호출만 `RegisterSend` 를 등록한다.
2. `RegisterSend` 는 송신 큐를 배치로 묶어 `WSASend` (scatter-gather)로 전달한다.
3. `WSASend` 가 `WSA\_IO\_PENDING` 외 오류를 반환하면 send buffer 참조를 해제하고 `\_sendRegistered=false` 로 복구한다.
4. `ProcessSend` 는 큐 잔량이 있으면 재등록하고, 비면 `\_sendRegistered=false` 로 내려 다음 등록을 허용한다.
5. `Disconnect` 는 `\_connected.exchange(false)` 로 중복 종료를 차단하고 완료 시 `OnDisconnected -> ReleaseSession` 으로 마무리한다.

적용 결과

재등록 연속성

- Recv 처리 후 즉시 재등록이 이어져 수신 루프가 끊기지 않는다.
- Send 큐 잔량 조건으로 재등록 여부가 결정돼 루프 상태가 예측 가능해진다.

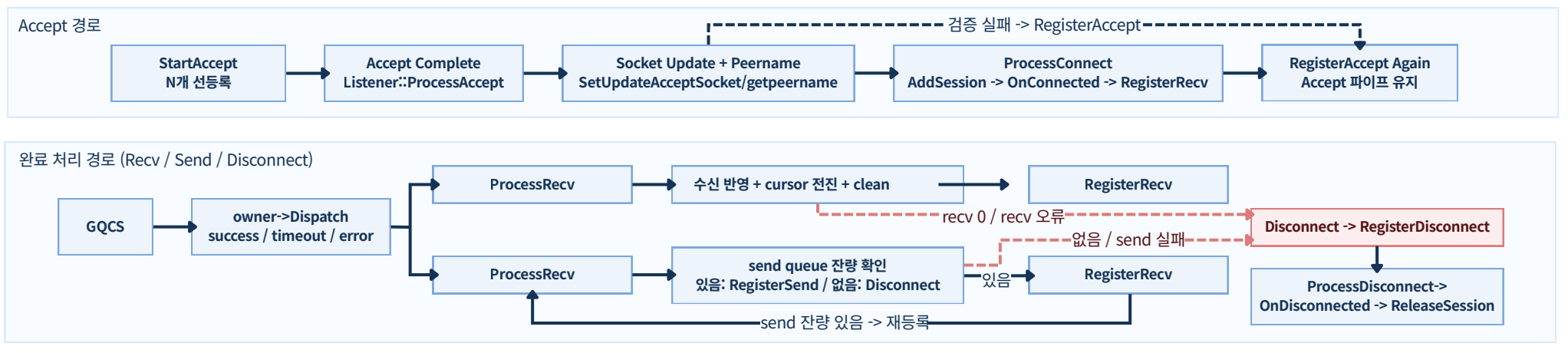
오류 종료 일관성

- Recv 0, send 오류, 버퍼 검증 실패가 모두 `Disconnect` 경로로 수렴한다.
- 종료 경로가 단일화돼 예외 상황에서도 정리 순서가 흔들리지 않는다.

운영 안정성

- 단일 in-flight `WSASend` 규칙으로 중복 등록과 큐 상태 혼선을 줄였다.
- 완료 통지 지연 구간에서도 객체 참조 해제 순서를 일관되게 유지한다.

IOCP 완료 통지 파이프라인



4. 메모리 풀과 송신 버퍼 재사용으로 실행 비용 변동을 줄인 구조

객체 할당과 송신 버퍼 생성을 재사용 경로로 묶어, 부하가 몰릴 때도 공통 실행 비용이 급격히 흔들리지 않도록 했다.

▶ 완료 통지 이후 Recv/Send를 언제 다시 등록하고, 어떤 조건에서 Disconnect로 종료하는지 실행 기준을 한 흐름으로 보여준다.

왜 이렇게 설계했나

반복 비용은 여러 기능 경로에서 함께 누적된다

이동, 전투, 브로드캐스트는 기능은 다르지만 결국 같은 할당 경로와 송신 버퍼 경로를 반복 사용한다. 그래서 특정 기능 하나보다 공통 경로를 먼저 안정화하는 편이 효과가 크다.

직렬화 구조만으로는 비용 변동을 막을 수 없다

Actor/JobQueue로 처리 순서를 고정해도, 반복 할당과 버퍼 생성 비용이 커지면 대기 시간은 다시 흔들릴 수 있다. 그래서 실행 순서뿐 아니라 메모리 경로도 함께 정리해야 했다.

송신 경로는 버퍼 수명 관리까지 포함해 설계해야 한다

패킷을 보호해서 보내는 것만으로는 충분하지 않았다. 비용이 WSAend 완료 전까지 버퍼가 안전하게 유지돼야 하므로, 송신 버퍼 재사용과 수명 관리 규칙까지 함께 설계했다.

1. MemoryPool + ObjectPool

짧게 생성되고 바로 정리되는 객체는 일반 할당 대신 재사용 풀에서 관리했다.

대상

- Projectile, Monster, Job처럼 수명이 짧고 반복 생성되는 객체가 한 시점에 몰린다.
- 일반 힙에 맡기면 생성 횟수만큼 비용 변동과 파편화 부담이 함께 커질 수 있다.

적용 방식

- Memory는 크기별 pool table로 소형·중형 할당을 대응하는 풀에 바로 연결한다.
- ObjectPool<T>는 Pop/Push, MakeShared로 생성과 반납 규칙을 타입 단위로 고정한다.

의미

- 반복 비용이 예측 가능한 풀 경로로 정리되어 피크 구간에도 새 할당 의존도를 낮춘다.
- 객체 수가 급증하는 순간에도 공통 실행 비용의 변동 폭을 줄일 수 있다.

2. SendBufferChunk 재사용

패킷마다 새 버퍼를 만들지 않고, 재사용 가능한 청크 안에서 필요한 범위만 잘라 썼다.

대상

- 송신량이 늘어날수록 버퍼 생성, 복사, 임시 객체 비용이 조용히 누적된다.
- 다건 전송 구간에서는 패킷 수만큼 버퍼 관리 비용이 다시 시작되는 구조가 특히 불리하다.

적용 방식

- SendBufferChunk는 6000 bytes 고정 크기 버퍼를 재사용하고, SendBuffer는 청크 내부 범위만 슬라이스로 참조한다.
- SendBufferManager::Open은 LSendBufferChunk를 우선 사용하고, 부족할 때만 글로벌 풀에서 새 청크를 보충한다.

의미

- 송신 경로의 메모리 단위가 패킷 수보다 청크 재사용 중심으로 바뀌어 재할당 빈도가 줄어든다.
- SendBuffer가 owner 참조로 청크 수명을 함께 보장해 비용이 WSAend 완료 전까지 메모리를 안정적으로 유지한다.

적용 효과

반복 비용 안정화

생성·송신 비용이 요청 수만큼 다시 시작되지 않고 재사용 경로에 묶인다.

실행 구조와 연결

직렬화 실행과 완료 통지 처리 같은 상위 구조가 재사용 경로와 결합될 때 실제 피크 구간의 흔들림이 줄어든다.

운영 관점

성능 항상 자체보다, 비용이 예측 가능한 공통 경로로 정리된다는 점에 의미가 있다.

Section3 Part C. [입장 게이트와 로딩 동기화]

- 로비 경유 구조로 입장 전 준비 상태를 먼저 확정
- 병렬 로딩 응답 확인과 실패 복구를 같은 게이트 규칙으로 통합
- 로딩 완료 직후 데이터 반영 기준을 일관된 순서로 고정

이번 파트에서 확인할 내용

✔ 로비 경유 입장 구조를 선택한 이유와 입장 안정화 기준

직접 월드 진입 대신 로비 중간 단계를 두어 로딩 완료 판정과 실패 복구를 한 경로에서 관리한다.
입장 초기 처리와 기본값 보정 기준을 함께 제시한다.

✔ 병렬 로딩 응답 확인과 게이트 기반 월드 입장 확정·복구 흐름

비동기 응답 라우팅과 무효 응답 차단 기준을 고정하고, stat/items/quick 조건을 모두 만족한 경우에만 월드 진입을 확정한다. 실패 시 복구 경로도 같은 규칙으로 수렴한다.

✔ 로딩 완료 후 아이템·스탯·퀵슬롯 상태 일관성 확정 경로

로딩 콜백 이후 데이터 반영, 규칙 정리·재계산, Redis 기준값 갱신, 클라이언트 동기화 순서를 고정해
입장 첫 프레임부터 전투 수치와 UI 기준이 어긋나지 않게 유지한다.

1. 로비를 거치는 입장 구조를 선택한 이유와 입장 안정화 기준

로그인 입장과 맵 전이를 같은 로비 경로로 모아, 로딩 완료 전 진입 차단과 실패 복구 기준을 통일했다.

▶ 로비는 단순 대기 공간이 아니라, 비동기 응답을 모아 입장 가능 여부를 판정하는 조정 지점이다. 스탯·인벤·퀵슬롯이 모두 준비되기 전에는 월드 상태에 최종 반영하지 않는다.

로비를 중간 단계로 둔 이유

로딩 완료 전 진입 차단

스탯/인벤/퀵슬롯이 모두 준비되기 전에는 월드로 보내지 않아 반쪽 상태 입장을 막는다.

실패 시 복구 지점 확보

플레이어를 먼저 로비가 보유하고, 월드 이관 실패 시 `Adopt`로 회수해 유실을 방지한다.

응답 처리 경로 단일화

S2S 응답을 로비 큐에서만 처리해 상태 갱신과 입장 판단이 같은 실행 경로에서 이루어지게 했다.

채널별 라우팅 분리

`GetOrCreateLobby(channelId)`로 채널별 로비를 분리해 잘못된 채널 응답이 다른 월드 반영으로 퍼지지 않게 했다.

직접 진입 대신 로비 경유를 선택한 이유

직접 월드 진입

실패 대응 난이도

실패 지점마다 되돌림 경로를 따로 관리해야 해 복구가 복잡하다.

상태 일치성

로딩 완료 전 일부 상태가 먼저 반영될 위험이 있다.

문제 추적성

응답 반영 지점이 분산돼 원인 추적 범위가 넓어진다.

로비 중간 단계

실패 대응 난이도

실패 지점마다 되돌림 경로를 따로 관리해야 해 복구가 복잡하다.

상태 일치성

로딩 완료 전 일부 상태가 먼저 반영될 위험이 있다.

문제 추적성

응답 반영 지점이 분산돼 원인 추적 범위가 넓어진다.

입장 요청 초기 처리와 상태 보정 ▼

입장 요청 초기 처리

- 세션 식별 고정 `Handle\_C\_ENTER\_GAME`에서 토큰 검증 후 `playerId`를 세션에 바인딩한다.
- 응답 매칭 키 저장 `PendingEnter(channel/map)`를 기록해 비동기 응답이 어느 입장 요청의 결과인지 식별한다.
- 필수 데이터 병렬 로딩 `LOAD\_PLAYER\_DATA`, `ITEMS\_LOAD`, `QUICKSLOT\_LOAD`를 동시에 요청해 준비 시간을 줄인다.

입장 조건 보정

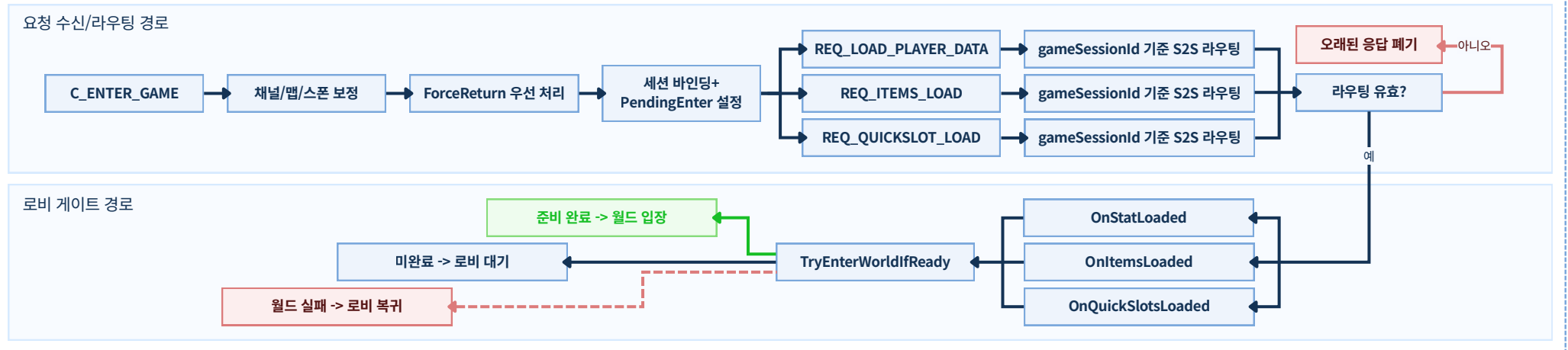
- 채널 기본값 보정 `channelId <= 0`이면 기본 채널로 교정한다.
- 맵/스폰 안전 교정 월드 외/잘못된 맵 요청은 기본 월드맵 + 기본 스폰으로 교정한다.
- 강제 복귀 처리 `ForceReturn` 대상은 로그인 직후 월드 스폰으로 복귀시켜 불완전 전이를 정리한다.

2. 병렬 로딩 응답 확인과 게이트 기반 월드 입장 확정·복구 흐름

병렬 로딩 응답이 들어오는 동안 오래된 응답을 걸러내고, 준비 완료 후에만 월드 진입을 확정하도록 실행 경로를 분리했다.

▶ 응답을 세션/채널 기준으로 확인한 뒤 유효한 것만 전달한다. 게이트 조건 충족 시 월드 합류를 확정하고, 실패 시 로비 소유권으로 되돌려 재시도 가능 상태를 유지한다.

병렬 로딩 합류 흐름



입장 게이트와 실패 복구 기준

1) 입장 준비 완료 확인

언제 확인하나 `stat/items/quick` 3개 로딩이 모두 완료됐을 때
어떻게 처리하나 하나라도 미완료면 로비 대기를 유지하고, 모두 완료 시 다음 단계로 진행한다.

2) 진입 예약 해제 확인

언제 확인하나 월드 입장을 확정하기 직전 `PendingEnter`를 비울 때
어떻게 처리하나 예약값을 먼저 해제해 늦은 응답·재요청으로 인한 이중 진입을 막는다.

3) 월드 입장 실패 복구

언제 확인하나 월드 입장 또는 방 확보가 실패했을 때
어떻게 처리하나 Lobby `Adopt`로 소유권을 되돌려 재시도 가능한 상태로 복구한다.

4) 중복 요청 차단

언제 확인하나 `MapChanging`이 true이거나 이미 입장 준비가 끝난 상태일 때
어떻게 처리하나 기존 입장 경로를 유지하고 새 요청은 무시해 처리 경로를 하나로 고정한다.

비동기 라우팅과 무효 응답 차단 기준

1) 라우팅 기준

- 세션 식별 S2S 응답의 `gameSessionId`로 대상 세션을 찾는다.
- 대상 로비 결정 `GetPendingChannelId\_AnyThread()` 값으로 채널별 로비를 선택한다.
- 반영 경로 고정 네트워크 스레드에서 상태를 직접 바꾸지 않고 `lobby->Push(...)`로 로비 액터 큐에 전달한다.

2) 무효 응답 차단 조건

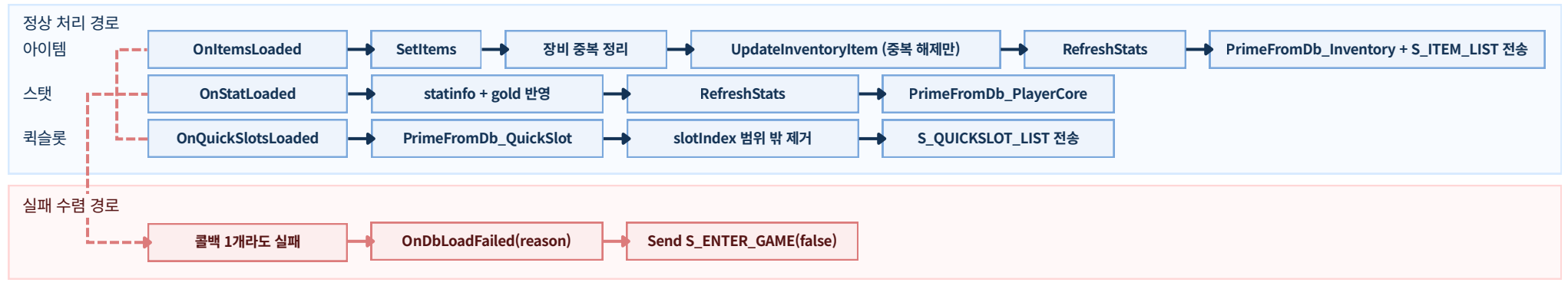
- 연결 종료 세션 `FindBySessionId`에 실패한 응답은 즉시 무시한다.
- 기본 조건 불충족 `playerId==0`, `pending` 채널 값 ≤ 0 , `lobby` 없음 이면 처리하지 않는다.
- 오래된 응답 현재 `PendingEnter`와 맞지 않으면 로비 액터로 전달하지 않는다.

3. 로딩 완료 후 아이템·스탯·퀵슬롯 상태 일관성 확정 경로

로비 게이트를 통과하기 직전, DB 응답 원본을 그대로 쓰지 않고 서버 규칙으로 보정해 플레이 상태를 일관되게 맞춘다.

▶ 입장 직후 아이템·스탯·퀵슬롯 반영 기준이 다르면 첫 프레임부터 전투 수치와 UI가 어긋난다. 그래서 로딩 콜백 순서를 고정해 반영 기준을 통일했다.

로딩 완료 처리 흐름도



정상 반영 단계

1) 인벤토리 반영과 장비 정리

- 1) 인벤토리 반영과 장비 정리
- SetItems로 인벤토리 원본을 메모리에 반영한다.
- 장비 슬롯 정책 기준으로 중복 장치를 해제한다.
- 중복 해제가 필요한 항목만 UpdateInventoryItem(...)로 반영한다.
- RefreshStats 후 S_ITEM_LIST를 전송한다.

2) 스탯 반영과 재계산

- statinfo와 gold를 플레이어 상태에 반영한다.
- 장비 보정 포함 파생 스탯을 다시 계산한다.
- PrimeFromDb_PlayerCore로 Redis 기준값을 갱신한다.

3) 퀵슬롯 정리와 UI 확정

- PrimeFromDb_QuickSlot로 Redis 기준값을 먼저 갱신한다.
- slotIndex < 0 또는 slotIndex ≥ QS_MAX 항목은 제외한다.
- 유효 슬롯만 S_QUICKSLOT_LIST로 전송해 UI 초기 상태를 확정한다

실패 수렴·품질 검증

1) 로딩 실패 공통 수렴

- OnItemsLoaded / OnStatLoaded / OnQuickSlotsLoaded 중 1개라도 실패하면 OnDbLoadFailed(playerId, reason)를 호출한다.
- S_ENTER_GAME(false)를 전송해 로딩 실패를 즉시 확정한다.
- 콜백 종류와 무관하게 실패 결과가 같은 경로로 처리되도록 고정한다.

2) 데이터 일관성 체크 기준

- 중복 장착 데이터는 로딩 직후 서버 정책으로 즉시 정규화한다.
- 퀵슬롯은 인덱스 검증을 통과한 항목만 동기화한다.
- 파생 스탯 계산은 아이템·스탯 반영 이후에만 수행한다.

3) 검증 확인 결과

- 중복 장착 상태 입력에서 로딩 단계 자동 해제가 수행되는지 확인했다.
- 범위 밖 퀵슬롯 인덱스가 전송 패킷에서 제외되는지 확인했다.
- 3개 콜백 중 1개 실패 시 실패 응답 경로가 동일하게 수렴하는지 확인했다.

Section3 Part D. [맵 전이 검증과 종료 수렴]

- 상태 확인과 토큰 ACK 검증으로 맵 전이 요청 실행 진입을 통제
- 전이 완료와 연결 종료 충돌 상황에서도 정리 순서를 고정
- 실패 처리 기준을 단일 경로로 수렴해 상태 일관성 유지

이번 파트에서 확인할 내용

✔ 맵 전이 요청을 상태 확인과 ACK 검증으로 안전하게 처리하는 방식

`REQ -> BEGIN(token) -> ACK(token)` 순서를 통과한 요청만 전이 실행 단계로 넘기고, 중복 요청·불일치 ACK·무효 컨텍스트는 사전 단계에서 차단하는 기준을 설명한다.

✔ 전이 완료·연결 종료 충돌 시 상태 일관성 유지 구조

맵 전이 완료 처리와 연결 종료 정리를 동일한 순서로 고정해 충돌 상황에서도 상태가 꼬이지 않게 만든다. 실패 처리 기준과 예방 효과까지 함께 정리해 운영 관점에서 확인 가능하게 구성한다.

1. 맵 전이 요청을 상태 확인과 ACK 검증으로 안전하게 처리하는 방식

룸 이동은 비용이 큰 작업이라, 전이 실행 전에 세션 FSM에서 먼저 검증을 끝낸다.

▶ 맵 전이는 좌표·채널·룸 소유권이 함께 바뀌는 구간이라, `REQ` -> `BEGIN(token)` -> `ACK(token)`을 통과한 요청만 전이 단계로 보내고 나머지는 사전 검증에서 걸러지도록 했다.

왜 상태 단계와 토큰 ACK 검증이 필요한가

- 사전 검증 없이 전이를 실행하면 실패 시 되돌림 범위가 커지고 장애 대응이 느려진다.
- 재전송·중복 입력·지연 ACK가 섞이면 단순 플래그만으로 정상 ACK와 오래된 ACK를 구분하기 어렵다.
- 세션 상태를 단계로 고정해, 검증을 통과하기 전에는 전이 실행 단계로 넘어가지 않게 했다.

차단 대상과 해결 방법

| 중복 전이 요청

확인

`WAITING\_ACK` / `SWITCHING` 상태에서 추가 요청 여부를 본다.

처리

`WAITING\_ACK` / `SWITCHING` 상태에서 추가 요청 여부를 본다.

| 오래된 ACK 재도착

확인

`TryConsumeMapChangeAck`에서 상태+토큰 동시 일치 여부를 본다.

처리

불일치 ACK는 즉시 무시해 전이를 진행시키지 않는다.

| 세션이 이미 종료된 상태의 ACK

확인

ACK 시점에 `playerId == 0`인지 본다.

처리

실행 경로로 넘기지 않고 `CancelMapChange`로 정리한다.

| 유효하지 않은 룸 상태의 ACK

확인

현재 룸이 유효하고 `GameRoom`인지 본다.

처리

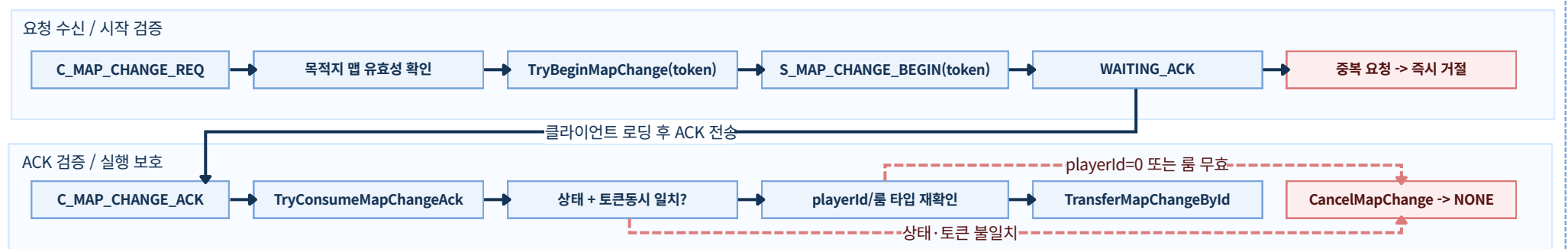
전이를 중단하고 `CancelMapChange`로 정리한다.

실패/예외 처리 기준: begin 실패는 즉시 차단하고, 상태·토큰 불일치 ACK는 무시하며, `playerId == 0` 또는 룸 무효 상태는 `CancelMapChange` 단일 경로로 정리한다.

입력 상태별 판정 기준표

상태	허용 작업	차단/정리
NONE	`TryBeginMapChange` 1회만 허용, 토큰/목적지 저장	중복 begin 요청은 거절하고 상태 유지
WAITING_ACK	상태+토큰 동시 일치 ACK만 소비	오래된 ACK/불일치 ACK는 무시
SWITCHING	완료(`EndMapChange`) 또는 취소(`CancelMapChange`)만 허용	신규 Begin/ACK 소비 차단
Cancel 처리	토큰/목적지/상태를 즉시 초기화	다음 전이를 `NONE`에서 재시작

맵 변경 요청 확인 흐름 (실행 전 검증 단계)



2. 전이 완료·연결 종료 충돌 시 상태 일관성 유지 구조

성공 처리와 종료 처리가 동시에 들어와도
세션·룸·저장 상태가 같은 순서로 마무리되도록 구성했다.

▶ 맵 전이 완료 처리와 연결 종료 처리는 같은 객체를 서로 다른 스레드에서 다룬다. 핵심은 충돌 상황에서도 정리 순서가 뒤섞이지 않도록 실행 순서를 고정하는 것이다.

맵 전이 완료 처리 순서

1 이전 룸 분리

이전 룸의 플레이어/시아/그리드 목록에서 먼저 제거해 이중 소속을 막는다.

2 목적지 정보 확정

channel/map/instance를 먼저 고정해 다음 단계 기준을 맞춘다.

3 로비 소유권 유지

`Lobby Adopt`로 실패 시 복구 가능한 임시 소유권 구간을 유지한다.

4 새 룸 진입 실행

`EnterMapChange`로 대상 룸 진입을 수행한다.

5 완료 응답 전송

`S\_MAP\_CHANGE\_END`를 전송해 클라이언트 화면 기준을 새 룸으로 맞춘다.

6 상태 종료 처리

`EndMapChange`로 전이 상태를 `NONE`으로 복귀시켜 다음 요청을 독립 처리한다.

연결 종료 정리 순서

1 세션 라우팅 차단

`GameSessionManager::Remove`로 새 패킷 유입을 먼저 차단한다.

2 전이 상태 초기화

`CancelMapChange`로 세션 상태를 `NONE`으로 되돌린다.

3 저장 반영 요청

`Dirty`, `FlushNow` 순서로 종료 직전 변경분 저장을 먼저 요청한다.

4 세션 결합 해제

`Unbind/ClearPlayerId`로 세션-플레이어 연결을 끊어 이전 세션 참조를 막는다.

5 공통 퇴장 경로 실행

`LeaveById`로 룸 자료구조 정리를 일관되게 수행한다.

6 인스턴스 오프라인 반영

필요 시 `OnMemberOffline`까지 이어 빈 방 종료 연계를 수행한다.

실패 처리 기준과 예방 효과

| 세션/룸 조회 실패

처리 기준 `TransferMapChangeById` 단계에서 세션 또는 룸 확보에 실패하면 `CancelMapChange`로 즉시 정리한다.

예방 효과 전이 상태가 중간 단계에 멈춘 채 남는 문제를 막는다.

| 새 룸 등록 실패

처리 기준 `EnterMapChange` 중 `EnterRegister` 실패 시 전이를 중단하고 `CancelMapChange`를 호출한다.

예방 효과 전이 도중 상태 불일치가 누적되지 않고 즉시 복구한다.

| 연결 종료 직후 이전 세션 참조

처리 기준 `Remove` -> `Unbind/ClearPlayerId` -> `LeaveById` 순서로 정리해 세션/룸 참조를 끊는다.

예방 효과 `FindByPlayerId`가 이전 세션을 반환하거나 유령 객체가 남는 문제를 줄인다.

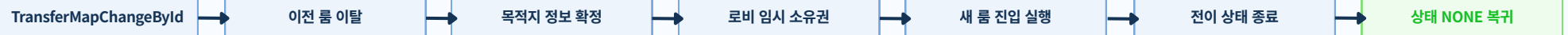
| 인스턴스 종료 신호 누락

처리 기준 마지막 멤버 이탈 시 `OnMemberOffline`까지 연계해 종료 신호를 보장한다.

예방 효과 빈 방 종료 플래그 누락과 정리 지연을 줄인다.

전이 완료·연결 종료 충돌 시 처리 흐름

전이 완료 처리



연결 종료 정리



전이·종료 동시 진입

이전 세션 참조/유령 객체 방지

Section4 Part A. [로그인 인증과 입장 안정화]

- 토큰 검증을 기준으로 로그인 인증과 게임 입장 경계를 분리
- 세션 식별 확정 시점을 고정해 비인가 진입과 중복 연결 충돌을 차단
- 입장 실패, 조회 오류, 재접속 충돌을 같은 기준으로 정리

이번 파트에서 확인할 내용

✔ 1. 인증과 게임 입장을 분리해 토큰 검증 후 세션을 연결하는 구조

로그인 성공과 실제 게임 입장을 분리하고, GameServer에서 Redis 토큰을 다시 검증한 뒤에만 세션 식별자를 확정한다. 인증 경계와 입장 경계를 같은 기준으로 맞춘 이유를 정리한다.

✔ 2. 게임 입장 예외와 재접속 충돌을 같은 기준으로 정리한 구조

토큰 조회 실패, 이름 키 조회 실패, 동일 사용자 재접속 충돌을 유형별로 나눠 처리 결과를 고정한다. 실패 원인에 따라 종료와 재시도 기준이 흔들리지 않도록 만든 방식을 설명한다.

1. 인증과 게임 입장을 분리해 토큰 검증 후 세션을 연결하는 구조

입장 요청은 Redis 토큰을 통한 경우세만세션을 연결한다.
실패 요청은 즉시 종료해 충돌을 초기에 막는다.

▶ 로그인 인증과 게임 입장을 분리하고, 토큰 검증을 통과한 요청에만 세션을 연결하도록 만든 이유와 처리 흐름을 정리한다.

설계 이유

왜 로그인 성공과 게임 입장을 분리했나

로그인 응답만으로 입장을 열면 유효하지 않은 요청도 게임 진입 단계까지 올라올 수 있어, GameServer에서 토큰 재검증을 필수로 두었다.

권한 없는 접속이 게임 입장 단계까지 넘어오는 경로를 초기에 차단한다.

왜 토큰 확인과 세션 연결 결정을 나눴나

토큰 검증 실패와 재접속 충돌은 원인과 처리 방식이 다르기 때문에 한 단계에서 함께 처리하면 예외 흐름이 복잡해진다. 그래서 토큰 확인과 세션 연결 결정을 분리했다.

보안 확인과 운영 적합성을 같은 흐름 안에서 분리해 관리할 수 있다.

입장 처리 흐름

경로 A · 로그인 검증과 토큰 발급

1. 로그인 요청
C_LOGIN 수신

2. 계정 검증
DBAgent에서 로그인 계정을 검증

4. Redis 저장
token -> playerId,
token:name 저장 (TTL 300초)

3. 토큰 발급
S2S_RES_LOGIN 성공 응답
이후 토큰 생성

경로 B · 입장 검증과 세션 연결

5. 입장 요청
C_ENTER_GAME(token) 수신

6. 토큰 조회와 처리 분기
토큰 없음 -> Disconnect(Invalid Token)
토큰 유효 -> 세션 연결 단계 진행

8. 세션 연결 완료
재접속 -> 최신 세션 기준으로 연결 정보 갱신
정상 입장 -> 연결 완료

7. 세션 연결
BindPlayerId -> SetPlayerId

LoginServer 인증 토큰 발급·저장

로그인 성공 이후 토큰 생성

DB 로그인 검증이 끝난 요청에서만 입장 토큰을 발급한다.

`token -> playerId` 저장

토큰과 사용자 식별자 매핑을 Redis에 300초 기준으로 저장한다

`token:name` 동시 저장

입장 초기 사용자 정보 조회를 위해 이름 키를 같은 수명으로 함께 저장한다.

짧은 유효시간 유지

오래된 토큰 재사용 가능성을 낮추기 위해 토큰 수명을 짧게 제한한다.

GameServer 입장 검증과 세션 연결

입장 요청에서 토큰 조회

Handle_C_ENTER_GAME에서 Redis 토큰을 조회해 playerId를 확인한다.

무효 토큰 즉시 차단

유효하지 않은 토큰은 즉시 Disconnect 처리해 입장 경계를 지킨다

유효 토큰만 세션 연결

BindPlayerId -> SetPlayerId_ActorOnly 순서로 세션을 연결한다.

재접속 시 최신 세션 반영

GameSessionManager가 동일 playerId 충돌을 최신 세션 기준으로 갱신한다.

2. 게임 입장 예외와 재접속 충돌을 같은 기준으로 정리한 구조

토큰 조회 실패, 이름 키 조회 실패, 재접속 충돌을 유형별로 나누고, 각 상황을 같은 원칙으로 처리하도록 구성했다.

▶ 입장 과정의 예외를 유형별로 나누고, 차단과 종료 기준을 일관되게 유지하도록 정리했다.

재접속 처리와 키 수명을 함께 본 이유

왜 토큰 키와 이름 키의 만료 시간을 같게 맞췄나

token과 token:name 만료 시점이 다르면 입장 순간에 사용자 정보 누락이 생길 수 있어, 같은 TTL로 조회 구간을 맞췄다.

기대 효과: 입장 초기 정보 누락을 줄이고 동기화 지연을 완화한다.

왜 재접속은 최신 연결을 기준으로 정리하나

같은 사용자의 연결이 겹치면 이전 연결 참조가 남아 처리 충돌이 생길 수 있어, 최신 연결 기준으로 세션 인덱스를 갱신한다.

기대 효과: 중복 연결 충돌과 이전 세션 참조 오류를 줄인다.

토큰 발급과 저장 원칙

토큰 생성 형식 고정

Token_{playerId}_{timestamp}_{random} 조합으로 토큰을 생성한다.

인증/이름 키 만료 시간 통일

인증 키와 이름 키를 동일 TTL(300초)로 운용해 수명주기를 맞춘다.

이름 정보는 별도 키로 저장

token:name:{token}를 분리 저장해 입장 초기 이름 동기화를 DB 재조회 없이 처리한다.

토큰 사용 가능 시간 제한

짧은 TTL 정책으로 재전송/탈취 토큰의 사용 가능 시간을 제한한다.

재접속 처리 원칙

세션 연결 조건

세션 연결(BindPlayerId)은 토큰 검증을 통한 요청에서만 수행한다.

동일 사용자 재접속 충돌 처리

GameSessionManager에서 동일 사용자 충돌을 감지하면 최신 연결 기준으로 세션 인덱스를 갱신한다.

세션 갱신 순서 고정

갱신 순서를 고정해 이전 세션 참조가 재사용되지 않도록 한다.

애매한 연결 상태 차단

입장 단계 실패는 세션 연결 이전에서 차단해 중간 연결 상태가 남지 않도록 한다.

5) 적용 전/후 비교

상황	처리 방식	처리 결과
토큰 없음 (`token miss`)	요청을 즉시 `Disconnect(Invalid Token)`으로 종료	권한 없는 접속 차단
토큰 유효 + 정상 사용자 식별자	`BindPlayerId` -> `SetPlayerId_ActorOnly` 순서로 세션 연결 완료	정상 입장 기준 고정
이름 키 조회 실패	기본 이름 보정 경로로 입장 진행	입장 연속성 유지
Redis 조회 타임아웃/오류	세션 연결 없이 입장을 중단하고 재시도 유도	불완전한 세션 연결 방지
동일 사용자 재접속 충돌	후접속 세션 우선으로 인덱스 갱신	이전 세션 참조 축소

Section4 Part B. [저장 구조와 데이터 정합성]

- Redis Write-Back과 AutoCommit으로 잦은 변경분을 먼저 흡수
- DBAgent 중심 저장 경로로 실시간 로직과 DB 반영 구간을 분리
- 부팅 시 기준 데이터를 나눠 불러와 초기 상태와 운영 기준을 맞춤

이번 파트에서 확인할 내용

✔ Redis Write-Back과 AutoCommit 기반 저장 구조

Prime, Dirty, In-Flight 기준을 나눠 변경분을 Redis에 먼저 반영하고, DB 저장은 AutoCommit 워커가 모아 처리한다. 저장 요청이 실시간 로직에 직접 번지지 않도록 만든 기준을 정리한다.

✔ 실시간 처리와 DB 저장을 분리한 DBAgent 저장 구조

게임 로직과 실제 DB 반영을 같은 경로에 두지 않고, DBAgent를 통해 저장 요청을 위임한다. 저장 처리 실패와 재시도, 트랜잭션 경계를 어디서 관리하는지 함께 설명한다.

✔ DB 템플릿과 JSON 월드 데이터를 나눠 불러와 기준을 맞춤 구조

DB 템플릿과 JSON 월드 데이터를 분리해 불러오고, 시작 시점에 필요한 기준값을 먼저 정리한다. 서버가 재시작돼도 같은 초기 기준에서 시작하도록 맞춤 구조를 보여준다.

Redis Write-Back과 AutoCommit 기반 저장 구조

접속 시 기존 상태를 먼저 정리하고, 변경분은 Redis에 반영한다.
DB저장은 AutoCommit 워커가 영역별로 처리한다.

▶ 코어, 인벤토리, 퀵슬롯을 나눠 저장하고 같은 사용자의 저장이 동시에 겹치지 않도록 막아, 변경이 많은 구간에서도 저장 경로가 흔들리지 않게 했다.

왜 이 구조를 선택했는가

자주 바뀌는 데이터는 먼저 Redis에 반영해 DB 지연이 게임 처리까지 번지지 않게 했다.

코어/인벤토리/퀵슬롯을 분리 저장해 한 영역 실패가 전체 저장 실패로 번지는 범위를 줄였다.

저장 요청 경로를 분리해 `AutoCommitService`와 DB 세션의 결합도를 낮췄다.

`\_inflightCount`로 같은 사용자의 저장 요청이 같은 라운드에서 겹치지 않도록 했다.

`RequestFlushNow`를 통해 종료 직전 변경분도 다음 주기를 기다리지 않고 반영할 수 있게 했다.

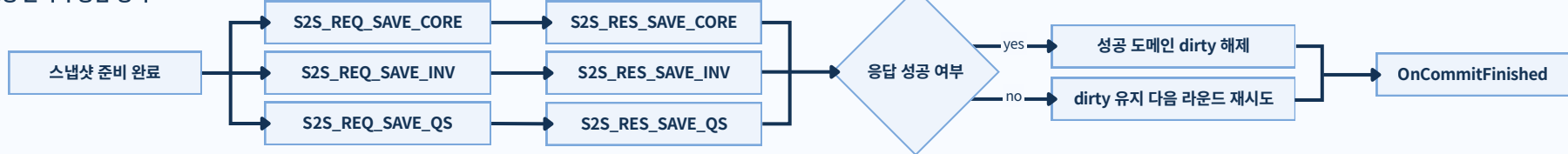
접속 시작 시 이전 세션의 변경 표시와 삭제 기록을 먼저 정리해 재접속 직후 상태 오염을 방지했다.

AutoCommit 워커 파이프라인

1) 시작 상태 정리·변경분 수집·중복 저장 차단



2) 저장 요청 분기와 응답 정리



입장 요청 초기 처리

- 접속 시점에 `p:{pid}:core/inv/qs` 값을 다시 읽어 시작 기준 데이터를 먼저 맞춘다.
- 이전 세션의 `dirty:\*`, `p:{pid}:invdel` 잔여값을 먼저 비워 시작 오염을 막는다.
- Prime 이후 변경분만 집계되도록 저장 시작 시점을 분리한다.

변경분을 모으는 방식

- 런타임 변경은 Redis Hash에 즉시 반영하고 `dirty:player/inv/qs`에 `pid`를 기록한다.
- 저장 대상은 `dirty:\*` + `\_flushNow`를 합쳐 계산해 저장 대상을 같은 방식으로 고른다.
- Set 특성으로 같은 `pid` 중복을 자동 제거해 불필요한 저장 요청 확장을 막는다.

같은 사용자 저장이 겹치지 않도록 막는 기준

- `\_inflightCount[pid]`가 있으면 해당 PID는 이번 라운드에서 제외한다.
- 저장 요청 수만큼 in-flight를 늘리고, 응답마다 줄여 같은 사용자 저장이 겹치지 않게 한다.
- in-flight가 해제된 PID만 다음 라운드에 다시 진입시켜 중복 저장을 막는다.

저장 응답 후 정리 방식

- 성공 응답을 받은 도메인만 dirty를 해제해 해당 저장 경로를 마무리한다.
- 인벤토리 저장 성공 시 `p:{pid}:invdel`도 함께 정리해 삭제 상태를 맞춘다.
- 실패 응답 도메인은 dirty를 유지해 다음 라운드 재시도로 넘긴다.
- `OnCommitFinished`에서 in-flight를 감소시켜 다음 라운드에서 다시 저장할 수 있게 한다.

2. 실시간 처리와 DB 저장을 분리한 DBAgent 저장 구조

GameServer는 실시간 처리에 집중하고, DB 저장은 DBAgent가 전담한다..
DBAgent는 같은 기준으로 저장 결과를 처리한다.

▶ 실시간 처리 경로와 DB 저장 경로를 분리해 지연 전파를 줄이고, 저장 항목별 처리 방식을 나눠 일부만 저장되는 상황 없이 일관되게 마무리되도록 구성했다.

왜 DB 저장 경로를 분리했는가

실시간 처리와 저장 일관성을 함께 지키기 위해, DB 저장을 별도 경로로 분리하고 트랜잭션 처리 방식을 나눴다.

운영 중 확인한 문제	대응 방식
게임 로직 스레드가 DB 지연을 직접 맞는 구조	DBAgent 분리로 실시간 처리와 DB 저장 경로를 분리
연결 생성/종료 오버헤드 누적	Connection Pool 기반으로 연결 재사용
과도한 저장 요청 구간에서 폭주	Pool Pop 대기로 백프레셔를 걸어 요청 속도 조절
반복 쿼리 경로의 비용/실수 위험	Prepare - Bind - Execute - Unbind 순서를 공통화
Inventory/QuickSlot/Trade 다중 쿼리 정합성	영역별 트랜잭션 경계와 ROLLBACK 처리 방식을 분리

GameServer 역할

게임 로직 실행, 저장 대상 데이터 구성, S2S 저장 요청 전송

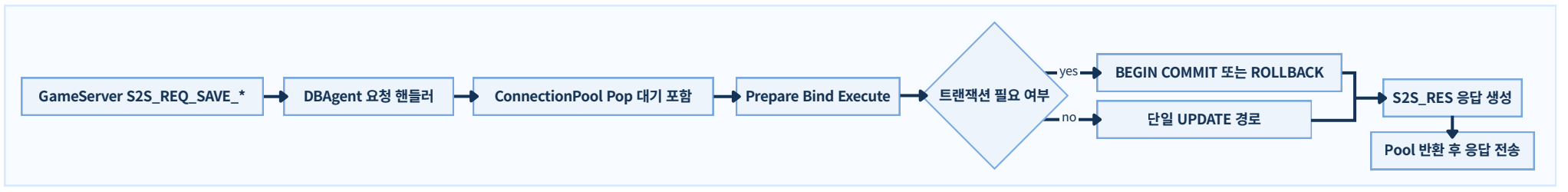
DBAgent 역할

Pool 대여/반납, SQL 실행, 트랜잭션 커밋/롤백 처리

SQL Server 역할

최종 저장 반영, 무결성 제약 검사, 저장 결과 반환

DB 저장 처리 흐름 (요약)



| 저장 항목별 처리 방식

코 어 저장

처리 범위: 단일 UPDATE

실패 시 처리: `success=false` 반환

인 벤 토 리 저장

처리 범위: `BEGIN/COMMIT`

실패 시 처리: `ROLLBACK TRAN`

퀵 슬롯 저장

처리 범위: `BEGIN/COMMIT`

실패 시 처리: `ROLLBACK TRAN`

거 래 확 정 저장

처리 범위: 단일 트랜잭션

실패 시 처리: `ROLLBACK TRAN` + `TRADE\_FAIL\_INTERNAL`

| 저장 처리 4단계

1

연결 확보

- Pop/Push로 DB 연결 대여/반납
- 풀 고갈 시 대기 후 순차 처리

2

쿼리 실행

- Prepare - Bind - Execute - Unbind 고정
- 작업별 SQL 실행 경로를 공통화

3

저장 확정

- 코어는 단일 UPDATE로 처리
- Inv/QS/Trade는 트랜잭션과 ROLLBACK 적용

4

결과 응답

- 성공/실패를 S2S 응답으로 반환
- 거래 실패는 failcode + context id 포함

3. DB 템플릿과 JSON 월드 데이터를 나눠 불러와 기준을 맞춘 구조

서버 시작시 DB 템플릿과 JSON 월드를 나눠 불러오고, 입장 전에 서버 기준을 먼저 맞춘다.

▶ 성격이 다른 데이터를 경로별로 나눠 불러오면 문제 위치를 빠르게 좁힐 수 있고, 첫 입장 전에 게임 기준값을 안정적으로 맞출 수 있다.

왜 이렇게 설계 했는가

데이터 출처를 나눠 관리 템플릿은 DB, 월드 데이터는 JSON으로 분리해 변경 원인과 반영 경로를 명확히 구분했다.	실패 영향 범위 최소화 한쪽 로드 실패가 전체 초기화 실패로 번지지 않도록 실패 지점을 분리하고 차단/보정 방식도 미리 정해 두었다.	입장 전 기준값 먼저 확정 게임 시작 전에 맵/스폰 규칙과 전투 템플릿 기준을 먼저 맞춰 첫 입장부터 판정 기준이 흔들리지 않게 했다.
---	--	---

DB 템플릿 로드 경로

DB 연결 직후 템플릿 데이터를 받아 서버 기준값을 먼저 맞춘다.

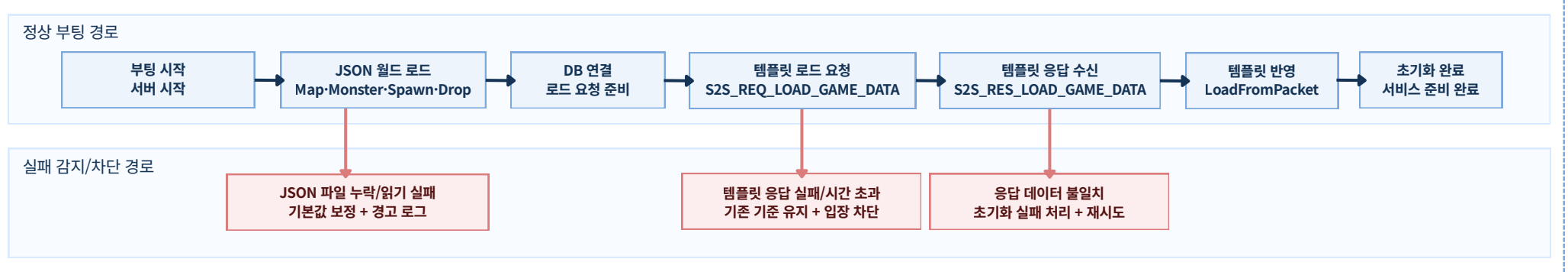
핵심 동작	적용 이유
1. DBAgent에 `S2S_REQ_LOAD_GAME_DATA` 로드 요청을 보낸다. 2. 조회 결과를 `S2S_RES_LOAD_GAME_DATA` 응답으로 받는다. 3. `LoadFromPacket`에서 기존 기준 데이터를 비운다. 4. Stat/Item/Skill을 다시 채워 시작 기준값을 맞춘다.	- 부분 수정 대신 전체 재적재로 템플릿 불일치 가능성을 줄인다. - DB 단일 관리 경로로 운영 기준과 밸런스 기준을 일관되게 유지한다. - 입장 전 적재 완료를 보장해 전투 계산 기준을 안정화한다.

JSON 월드 로드 경로

Map/Monster/Spawn/Drop 파일을 순서대로 로드하고, 잘못된 값은 보정 규칙으로 정리한다.

핵심 동작	적용 이유
1. JSON 4종 파일을 순서대로 로드한다. 2. 파일 누락/읽기 실패 시 로그를 남기고 기본 보정 경로로 이어간다. 3. SpawnTables에서 중복 `spawnId`를 제거하고 `maxAlive`/`respawnSec`를 보정한다. 4. DropTables에서 `noDropWeight`, `min/max` 범위를 정규화한다.	- DB 템플릿과 분리 운영해 문제 위치(DB/파일)를 빠르게 좁힌다. - 파일 기반 버전 관리로 변경 추적과 협업 운영이 쉽다. - 입장 전에 맵 검증/스폰 좌표 계산 기준을 확정한다.

부팅 동기화 경로 (정상/실패 분기)



Section5. [운영 관측 & 성능 검증]

- Prometheus/Grafana 기반 모니터링 구성 위에서 여러 서버 지표를 같은 시간축으로 비교했다.
- AOI 비교, 저장 방식 비교, 장시간 복합 부하 검증을 각각 독립된 묶음으로 나눠 정리했다.
- 구조 차이가 실제 운영 지표와 안정성으로 어떻게 이어지는지 그래프와 해석으로 연결했다.

이번 파트에서 확인할 내용

✔ Prometheus/Grafana 기반 모니터링 구성

- GameServer와 DBAgent 지표를 같은 기준으로 비교할 수 있도록, 수집부터 대시보드 구성까지의 흐름을 먼저 정리했다.

✔ 집중 혼합 부하 시나리오로 AOI 적용 효과 비교

Hot Room Mix 조건에서 전파 대상 수, 세션 송신량, 작업 대기 시간이 어떻게 달라지는지 같은 기준으로 비교했다.

✔ 저장 요청 부하 시나리오로 write-back 저장 방식 비교

quickslot 변경이 몰릴 때 저장 전송 빈도, RTT, DB 경로 지표가 어떻게 달라지는지 묶어 봤다.

✔ 장시간 복합 부하로 900명 동시 접속 안정성 검증

메모리, 작업 대기, 스레드 사용률, 저장 응답 시간을 함께 보며 장시간 유지 구간의 안정성을 정리했다.

1. Prometheus/Grafana 기반 운영 관측 체계

GameServer와 DBAgent를 같은 시간축에서 비교할 수 있도록 수집, 저장, 시각화를 하나의 관측 흐름으로 구성했다.

▶ 운영 관측의 핵심은 화면 구성보다, 게임 로직·큐·DB 경로 중 어느 구간에서 지연이 시작되는지 같은 기준으로 구분해 읽을 수 있게 만드는 데 있다.

Prometheus/Grafana를 선택한 이유

비교 가능성

GameServer와 DBAgent를 같은 시간축으로 비교할 수 있다

Prometheus는 두 서버의 지표를 동일 주기로 수집하고, Grafana는 지연·처리량·실패를 한 화면에 정렬해 보여준다. 병목의 시작 지점을 서비스 경계까지 포함해 읽기 쉬운 구성이다.

구조 적합성

기존 서버 구조에 무리 없이 연결할 수 있다

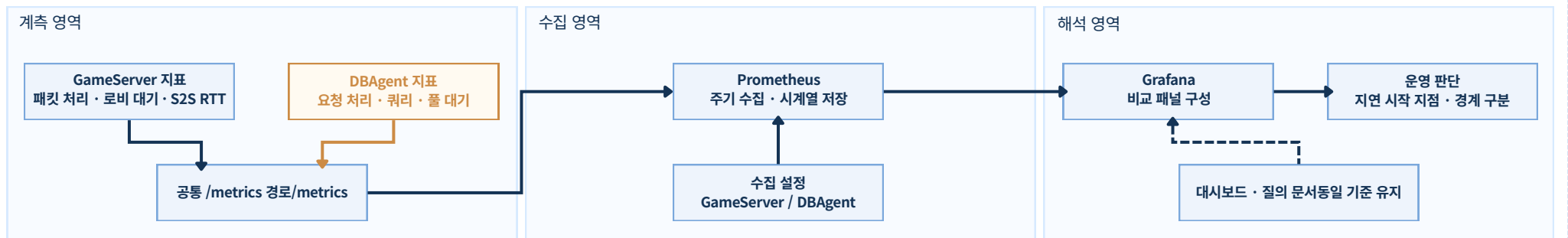
공용 계층은 지표 노출 경로를 유지하고, 각 서버는 필요한 지표만 추가하면 된다. 계층 범위가 넓어져도 수집 방식과 해석 기준을 안정적으로 유지하기 좋다.

재현성

대시보드와 질의를 함께 남겨 동일한 기준을 유지할 수 있다

scrape 설정, 대시보드, 질의 문서를 함께 보관하면 이후 측정 결과를 다시 확인할 때도 같은 기준을 그대로 적용할 수 있다. 결과 해석의 일관성을 유지하기에도 유리하다.

3개 관측 영역 흐름



주요 관측 지표

게임

게임 처리 흐름

패킷 처리 시간, 로비 대기, 서버 간 왕복 시간을 함께 보면 게임 로직과 서버 간 연동 중 어느 구간에서 지연이 시작되는지 구분할 수 있다.

DB

DB 처리 경로

요청 처리 시간, 실제 질의 시간, 커넥션 대기를 함께 보면 DB 병목과 풀 경쟁을 나눠서 볼 수 있다.

큐

큐와 워커 상태

대기 시간과 실행 시간을 나눠 보면 처리 로직의 비용인지, 앞단 적체의 영향인지 분리해 해석할 수 있다.

서비스

접속 상태와 네트워크 변화

CCU, 인게임 인원, 세션 송수신량은 지표 변화가 실제 접속 규모와 어떤 관계를 가지는지 해석하는 기준이 된다.

이 체계로 확인할 수 있는 운영 판단

지연은 어느 구간에서 먼저 시작되는가

패킷 처리, 큐 대기, 서버 간 왕복 시간, DB 대기를 나란히 비교하면 선행 지표를 확인할 수 있다.

문제는 단일 서버 내부인가, 서버 간 연동 구간인가

GameServer와 DBAgent의 변화를 같은 시간축으로 보면 어느 쪽에서 변화가 먼저 시작되는지 구분할 수 있다.

부하 증가는 실제 이용자 규모와 어떻게 연결되는가

CCU, 인게임 인원, 세션 I/O를 함께 보면 특정 지표 상승이 접속 증가와 동행하는지 해석할 수 있다.

Section5 Part A. [AOI 적용 효과 검증]

- Hot Room Mix 조건에서 AOI 미적용 기준안과 적용안을 같은 입력으로 비교했다.
- 전파 대상 수, 세션 송신량, 작업 대기 시간 변화를 한 흐름으로 읽을 수 있게 묶었다.
- 설계 의도, 대표 그래프, 결과 해석 순으로 AOI 채택 근거를 정리했다.

이번 파트에서 확인할 내용

✓ 1. 집중 혼합 부하 시나리오 (Hot Room Mix)를 통한 AOI 적용 효과 검증

왜 플레이어가 몰린 혼합 부하 조건을 AOI 비교 시나리오로 잡았는지부터 정리했다. 이동, 스킬, 스폰/디스폰이 한 구간에 겹치는 상황에서는 AOI 적용 여부에 따른 전파 대상 수 차이가 가장 분명하게 드러난다.

✓ 2. 집중 혼합 부하 시나리오 그래프 및 비교

Broadcast Recipients p95, Session TX Throughput, JobQueue Wait p95를 같은 시간 구간에서 나란히 비교했다. 이를 통해 전파 범위 변화가 실제 송신량과 작업 대기 시간 변화로 어떻게 이어지는지 한 흐름으로 읽을 수 있도록 구성했다.

✓ 3. AOI 적용은 전파 범위, 송신량, 큐 대기를 함께 낮췄다

대표 측정 구간에서는 AOI 적용안이 전파 대상 수를 먼저 줄였고, 그 차이가 세션 송신량과 작업 대기 시간 감소로 이어졌다. 결과적으로 AOI는 단순한 계산 최적화가 아니라, 플레이어가 몰리는 구간의 부하 구조 자체를 바꾸는 방식임을 보여준다.

1. 집중 혼합 부하 시나리오 (Hot Room Mix)를 통한 AOI 적용 효과 검증

같은 혼합 플레이 조건에서
AOI 적용 전후의 구조 차이를 비교했다.

- ▶ 집중 혼합 부하 시나리오는 최대 접속자 수 자체를 보여주기보다, recipient fan-out, session tx, jobqueue wait가 어떤 흐름으로 연결되는지를 동일한 시간축에서 설명하기 위한 비교 시나리오다.

집중 혼합 부하 시나리오를 비교 시나리오로 선택한 이유

AOI 적용 여부에 따른 차이가 가장 분명하게 나타나는 조건이다 플레이어와 이벤트가 한 구간에 몰리면 이동, 스킬, 스폰/디스폰이 동시에 겹친다. 이때 AOI 유무에 따른 전파 대상 수 차이가 즉시 드러나므로, 구조적 차이를 비교하기에 적합하다.	전파 범위 차이가 운영 지표 변화로 이어지는 흐름을 보여주기에 좋다 수신 대상 수 증가는 세션 송신량 증가로 이어지고, 그 변화는 다시 룸 직렬 처리 대기로 확산된다. 따라서 이 시나리오는 구조 차이가 운영 지표로 어떻게 연결되는지 설명하기에 적합하다.	AOI 계산 비용을 포함한 전체 구조 효과를 비교할 수 있다 AOI 자체에는 계산 비용이 필요하다. 그러나 그 비용보다 fan-out 축소로 줄어드는 송신량과 큐 대기가 더 크다면, AOI는 전체 시스템 관점에서 충분한 구조적 이점을 가진다.
--	--	--

기본 부하 설정과 비교 기준

입력 조건은 동일하게 두고, AOI 적용 여부에 따른 구조 차이만 비교할 수 있도록 정리했다.

부하 구성 이동, 스킬, 스폰/디스폰이 함께 발생하는 혼합 입력을 사용해 실제 전투 구간과 유사한 흐름을 구성했다.	비교 방식 동일한 입력 조건에서 AOI 적용 구조와 미적용 구조만 다르게 두어, 전파 방식 차이가 그대로 드러나도록 구성했다.
집계 구간 초기 증가 구간보다 ramp-up 이후 steady-state 구간을 기준으로 주요 지표를 비교했다.	해석 기준 최대 CCU보다 전파 대상 수, 송신량, 룸 직렬 처리 대기가 어떤 흐름으로 연결되는지를 중심으로 해석했다.
비교안 B / 적용안 AOI 적용 구조 visible set diff, zone broadcast, spawn/despawn delta를 사용해 필요한 대상에게만 전파한다. 계산 비용이 추가되지만 전체 fan-out을 줄이는 방향의 설계다.	비교안 A / 기준안 AOI 미적용 구조 같은 룸을 단일 관심 영역처럼 보고, 이동과 전투 관련 전파를 룸 전체 기준으로 보낸다. 이는 제품 최종 계약이 아니라 AOI 부재 시 비용을 설명하기 위한 비교용 기준안이다.

주요 관찰 지표

- 지표 1** **Broadcast Recipients p95**는 AOI 적용 여부에 따라 전파 대상 수가 어떻게 달라지는지 보여준다. 이는 Hot Room 비교에서 가장 직접적인 구조 차이이다.
- 지표 2** **Session TX Throughput**는 전파 범위 차이가 실제 네트워크 송신량 차이로 이어졌는지를 보여준다. fan-out 축소 효과가 운영 지표로 나타나는 구간이다.
- 지표 3** **JobQueue Wait p95**는 송신량 변화가 룸 직렬 처리 대기 시간에 어떤 차이를 만드는지 보여준다. 결과적으로 병목이 어떤 양상으로 전개되는지를 설명하는 지표다.

2. 집중 혼합 부하 시나리오 그래프 및 비교

▶ AOI 적용 여부에 따라 전파 대상 수, 실제 송신량, 직렬 처리 대기가 어떻게 달라지는지 같은 시나리오 기준으로 비교했다.

세 그래프를 따로 보기보다
fan-out, tx, queue wait 순서로 함께 읽도록 배치했다.

대표 차트 1. Broadcast Recipients p95(전파 대상 수)

AOI가 줄여주는 비용의 본질은 recipient fan-out 축소다.



대표 차트 2. Session TX Throughput(세션 송신량)

fan-out 차이가 실제 네트워크 송신량 차이로 이어지는지 확인한다.



대표 차트 3. JobQueue Wait p95(작업 대기 시간)

송신량 증가는 결국 룸 직렬 처리 대기 증가로 이어진다.



3. AOI 적용은 전파 범위, 송신량, 큐 대기를 함께 낮췄다

대표 지표를 종합해
AOI 채택의 근거를 정리했다.

- ▶ 약 100 CCU 대표 측정 구간에서 Broadcast Recipients p95, Session TX, JobQueue Wait p95가 모두 같은 방향으로 낮아졌다.
이는 AOI가 단순한 계산 최적화가 아니라, 이벤트가 집중되는 구간에서 전파 범위를 줄여 병목의 전개 양상을 바꾸는 핵심 구조임을 보여준다.

대표 지표 요약

약 100 CCU 대표 측정 구간 기준이다. 절대 수치보다 동일 조건에서 어떤 지표가 얼마나 달라졌는지를 보는 데 의미가 있다.

Metric	Baseline	Final	Delta	Meaning
Broadcast Recipients p95	125	7.9	-93.7%	전파 대상 수 자체가 크게 줄었다.
Session TX	163K B/s	8.9K B/s	-94.6%	fan-out 차이가 송신량 폭증 여부를 갈랐다.
JobQueue Wait p95	69.0 ms	18.8 ms	-72.8%	룸 직렬 처리 대기가 눈에 띄게 완화됐다.
Worker Busy Ratio	3.28%	0.51%	-84.6%	같은 입력을 훨씬 낮은 실행 부담으로 처리했다.

지표 비교

Broadcast Recipients p95

Baseline에서는 p95가 125까지 증가한 반면,
AOI 적용안은 7.9 수준으로 유지됐다. 동일한 입력 조건에서도
전파 대상 수가 크게 달라졌음을 보여준다.

지표 비교

Session TX Throughput

Baseline은 163K B/s, AOI 적용안은 8.9K B/s 수준이었다.
전파 범위 차이가 실제 네트워크 송신량 차이로 직접 이어졌음을
보여준다.

지표 비교

JobQueue Wait p95

Baseline은 69.0 ms, AOI 적용안은 18.8 ms였다.
송신량이 높아진 조건에서는 같은 룸 직렬 처리 경로의 대기 시간
도 함께 길어졌다.

해석

전파 범위 차이가 가장 먼저 드러났다

같은 혼합 입력에서도 AOI 적용 여부에 따라 전파 대상 수가 크게
달라졌다. 이벤트가 집중되는 구간의 비용 출발점이 개별 핸들러
보다 전파 범위에 있음을 보여주는 결과다.

해석

전파 범위 축소는 송신량 절감으로 이어졌다

AOI로 줄어든 recipient 수는 곧바로 Session TX 감소로 연결됐
다. 즉 AOI의 효과는 로직 내부의 미세한 처리 시간보다 송신 경로
에서 더 분명하게 확인된다.

해석

Queue wait 변화가 병목 양상을 설명했다

부하 증가 구간에서 차이는 packet handle 자체보다 room
queue wait에서 더 크게 나타났다. 이는 집중 부하 구간 최적화
의 핵심이 전파 범위 제어에 있음을 뒷받침한다.

종합 해석

- 구조** AOI는 “보이는 대상만 전파한다”는 계약을 통해 fan-out을 먼저 줄였다.
그 효과가 Session TX와 room queue wait 완화로 이어지면서 최종 비용 차이를 만들었다.
- 운영** 운영 관점에서 AOI 적용안은 이벤트 집중 구간의 변동 폭을 더 낮게 유지했다.
같은 입력에서도 더 낮은 송신량과 더 짧은 queue wait를 유지하므로, 병목이 급격히 확대되는 시점을 뒤로 미르는 구조적 이점이 있다.
- 한계** 이 결과는 단일 PC 환경의 대표 측정 구간 기준이며, baseline은 naive baseline이다.
따라서 절대 성능 규모를 제시하기보다, AOI 채택의 타당성을 설명하는 구조 비교 근거로 해석하는 것이 적절하다.

Section5 Part B. [저장 방식 비교]

- quickslot 변경 부하에서 즉시 저장과 write-back 구조를 같은 조건으로 비교했다.
- 저장 전송 빈도, Game -> DBAgent RTT, DB 내부 지표를 함께 보며 차이를 읽었다.
- 설계 의도, 대표 그래프, 결과 해석 순으로 write-back 구조의 차이를 정리했다.

이번 파트에서 확인할 내용

✔ 1. 저장 요청 부하 시나리오 (Persistence Drain)로 write-back 저장 방식 비교

찾은 quickslot 변경이 반복될 때, 즉시 저장 방식과 write-back 방식이 어떤 차이를 만드는지 비교했다. 변경 이벤트가 그대로 DB 경로로 전달되는 구조와, 중간에서 모아 전달하는 구조를 같은 입력 조건에서 나란히 확인했다.

✔ 2. 저장 요청 부하 시나리오 그래프 및 비교

저장 전송 빈도, Game -> DBAgent RTT p95, DB Query / Pool Wait p95를 함께 비교했다. 이를 통해 저장 방식 차이가 앞단 전달 빈도에서 먼저 드러나는지, 아니면 DB 내부 지표까지 이어지는지 한 흐름으로 읽을 수 있도록 구성했다.

✔ 3. Write-back은 찾은 저장 요청을 먼저 흡수했다

같은 입력 부하에서도 write-back 적용안은 저장 요청을 바로 DB 경로로 넘기지 않고 중간에서 모아 처리했다. 그 결과 실제 저장 전송 빈도와 RTT가 함께 낮아졌고, 저장 요청이 실시간 처리 흐름을 직접 흔들지 않도록 만드는 구조임을 보여줬다.

1. 저장 요청 부하 시나리오 (Persistence Drain)로 write-back 저장 방식 비교

같은 quickslot 변경 부하에서
즉시 저장과 write-back 방식을 비교했다.

- ▶ 이 시나리오는 quickslot 변경이 자주 발생할 때, 저장 요청이 바로 DB 경로로 쌓이지 않으면 Redis dirty set + AutoCommit + DBAgent가 중간에서 한 번 흡수하는지를 비교하기 위한 것이다.

저장 요청 부하 시나리오를 선택한 이유

찾은 변경이 저장 경로 부담으로 바로 이어질 수 있다

quickslot 변경은 짧은 간격으로 반복될 수 있다. 이때 매번 즉시 저장하면 게임 플레이와 무관하게 저장 경로 자체가 계속 부담을 받게 된다. 그래서 이 시나리오가 구조 차이를 보기 좋은 조건이 됐다.

Write-back의 핵심 차이를 가장 직접적으로 확인할 수 있다

write-back의 핵심은 변경 횟수만큼 저장 요청을 바로 보내지 않는 데 있다. 그래서 변경 횟수와 실제 저장 전송 횟수를 나눠서 보는 것이 중요했다.

지연의 원인이 어디에서 생기는지도 함께 볼 수 있다

Game->DBAgent RTT, DB Query, DB Pool Wait를 함께 보면, 지연이 DB 내부에서 생긴 것인지 아니면 그 이전 단계인 저장 방식 차이에서 먼저 생긴 것인지 더 쉽게 구분할 수 있다.

기본 부하 설정과 비교 기준

입력 조건은 동일하게 두고, AOI 적용 여부에 따른 구조 차이만 비교할 수 있도록 정리했다.

시나리오

quickslot 저장 이벤트를 반복적으로 발생시키는 전용 부하 조건

관찰 축

저장 전송 빈도, RTT, DB 내부 지표 변화

입력

두 조건에 같은 quickslot 변경 부하 적용

해석

즉시 저장과 write-back 방식 차이 확인

B안 / 적용안

Write-back 구조

quickslot 변경은 dirty set에 모아두고, AutoCommit이 일정 주기로 DBAgent에 전달한다. 찾은 저장 요청이 바로 DB 경로로 쌓이지 않게 하는 방식이다.

A안 / 기준안

즉시 저장 구조

quickslot 변경이 생길 때마다 저장 요청을 바로 DBAgent/DB 경로로 보낸다. write-back이 없을 때 저장 부담이 어떻게 커지는지 보기 위한 기준안이다.

주요 관찰 지표

- 저장 빈도** DB-bound Save Traffic은 변경 이벤트가 실제 저장 요청으로 얼마나 이어지는지 보여준다. write-back의 완충 효과를 가장 직접적으로 볼 수 있는 지표다.
- RTT** Game -> DBAgent RTT p95는 저장 방식 차이가 게임 서버 기준 지연으로 얼마나 이어지는지 보여준다.
- DB 내부** DB Query / Pool Wait p95는 지연 차이가 DB 실행이나 커넥션 대기 때문인지 확인하기 위한 지표다.

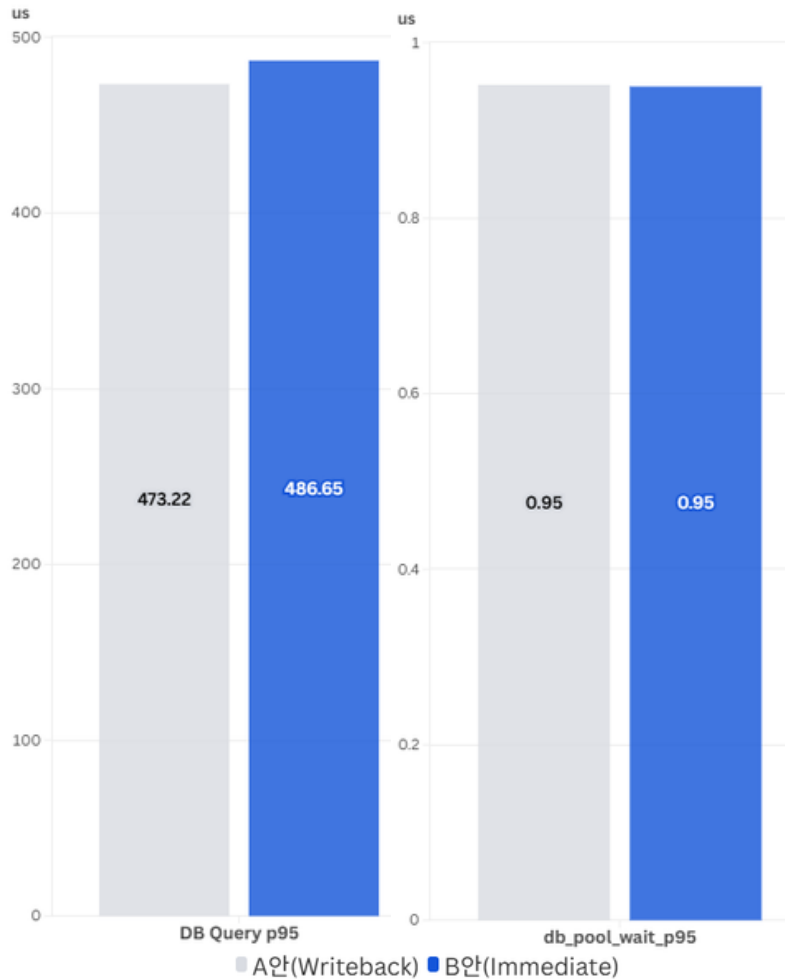
2. 저장 요청 부하 시나리오 (Persistence Drain) 그래프 및 비교

저장 빈도 함수와 DB 경로 지연을 같은 흐름 안에서 함께 읽도록 배치했다.

- ▶ 이 페이지는 immediate_quickslot baseline과 write-back final을 비교하는 대표 차트 자리다.
먼저 저장 트래픽 함수 여부를 읽고, 그 다음 RTT와 DB 지연 지표를 확인하는 순서가 중요하다.

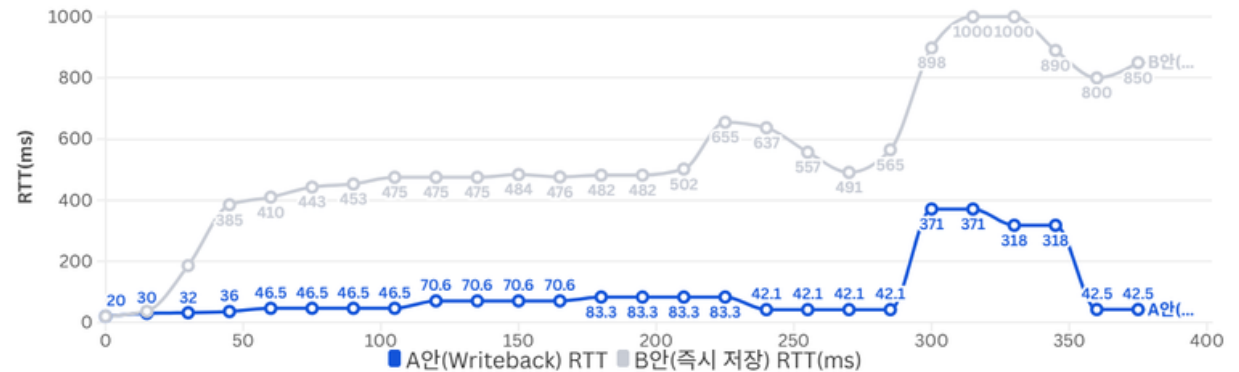
대표 차트 1. DB-bound Save Traffic

write-back이 mutation 빈도를 그대로 DB 요청 빈도로 보내지 않는지 먼저 확인한다.



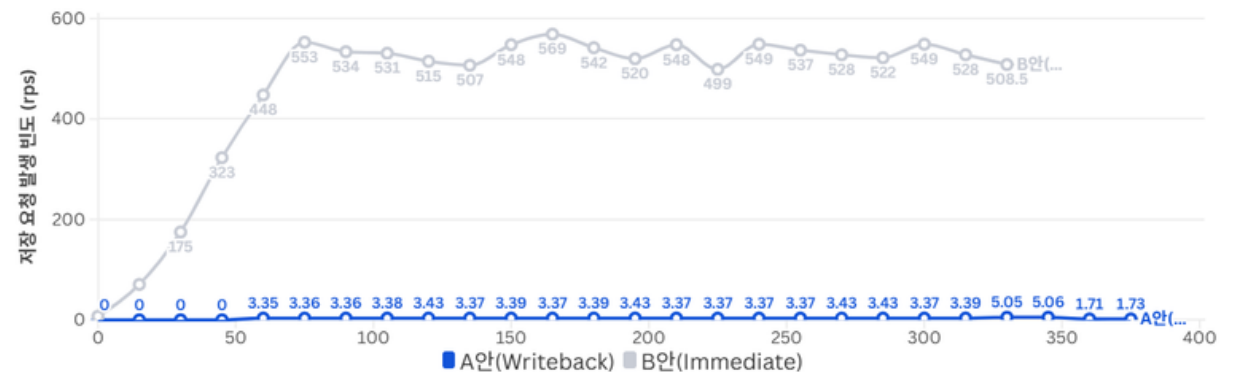
대표 차트 2. Game -> DBAgent RTT p95

게임 서버 입장에서 저장 RTT가 즉시 저장에서 먼저 커지는지 본다.



대표 차트 3. DB Query / Pool Wait p95

저장 압력이 실제 DB 실행 구간의 지연으로도 드러나는지 확인한다.



3. Write-back은 잦은 저장 요청을 먼저 흡수했다

대표 지표를 바탕으로
write-back 적용 결과를 정리했다.

- ▶ 같은 입력 부하에서 write-back 적용안은 잦은 quickslot 변경이 바로 저장 요청으로 이어지지 않도록 만들었고, 그 결과 Game -> DBAgent RTT p95도 더 낮게 유지됐다.

대표 지표 요약

같은 입력이 저장 경로에서 어떤 차이로 이어졌는지 정리했다. 핵심은 저장 요청 수와 지연 지표가 어떻게 달라졌는지다.

항목	기준안	적용안	변화	해석
quickslot 입력 빈도	579	579	~0%	입력 부하는 사실상 동일했다.
저장 경로 전달 빈도	529.5 rps	3.37 rps	-99.4%	write-back이 저장 요청을 모아 보내면서 부담을 크게 줄였다.
Game -> DBAgent RTT p95	482 ms	70.6 ms	-85.4%	지연 차이는 DB 내부보다 저장 방식 차이에서 먼저 드러났다.
DB Query / Pool Wait p95	0.486 ms / 0.95 us	0.476 ms / 0.95 us	minor	DB 내부 지표 차이는 크지 않았다.

비교 결과

실제 저장 요청 빈도

기준안에서는 **529.5 rps** 수준의 quickslot 변경이 거의 그대로 저장 경로로 전달됐다. write-back 적용안에서는 같은 입력이 약 **3.37 rps** 수준의 AutoCommit 전송으로 정리됐다.

비교 결과

Game -> DBAgent RTT p95

Game -> DBAgent RTT p95는 기준안 **482 ms**, 적용안 **70.6 ms**로 나타났다. 같은 입력 조건에서도 저장 방식에 따라 지연이 크게 달라졌다.

비교 결과

DB Query / Pool Wait p95

DB Query p95는 **0.486 ms**와 **0.476 ms**였고, DB Pool Wait p95도 두 조건 모두 약 **0.95 us** 수준이었다. DB 내부 지표 차이는 크지 않았다.

의미

Write-back은 저장 요청을 앞단에서 줄였다

가장 큰 차이는 실제 저장 요청 수였다. write-back은 변경을 모아 전달하면서, 이벤트 수가 그대로 저장 요청 수가 되지 않도록 만들었다.

의미

지연 차이는 저장 방식 차이에서 먼저 드러났다

DB Query와 Pool Wait가 비슷한 수준이었는데도 RTT는 크게 차이 났다. 이번 구간에서는 DB 내부 처리보다 저장 요청이 전달되는 방식이 먼저 영향을 준 것으로 볼 수 있다.

의미

DB 내부 병목은 아직 크게 드러나지 않았다

이번 결과만으로 DB 실행 구간이나 커넥션 대기가 주요 병목이라고 보기는 어렵다. 더 높은 부하에서는 이 지표들이 언제부터 벌어지는지도 추가로 확인할 수 있다.

종합 해석

이번 결과는 write-back이 잦은 quickslot 변경을 바로 DB 경로로 넘기지 않고, 중간에서 모아 처리한다는 점을 보여준다. RTT가 함께 낮아진 것도 이 차이와 연결된다. 반면 DB Query와 Pool Wait는 거의 비슷했기 때문에, 이번 구간의 지연 차이는 DB 내부 처리보다 저장 요청이 바로 전달되는 방식에서 먼저 생긴 것으로 볼 수 있다.

Section5 Part C. [장시간 복합 부하 검증]

- 이동, 스킬, 하트비트, quickslot 저장 요청이 함께 반복되는 mixed load를 유지했다.
- 900명 동시 접속 유지 구간에서 메모리, 작업 대기, 스레드 사용률, 저장 응답을 함께 확인했다.
- 테스트 설계, 핵심 그래프, 결과 정리 순으로 장시간 안정성 근거를 묶어 정리했다.

이번 파트에서 확인할 내용

✓ 1. 장시간 복합 부하를 통한 900명 동시 접속 안정성 검증

이동, 스킬, 하트비트, quickslot 저장 요청이 함께 반복되는 조건에서 900명 동시 접속을 60분 동안 유지하도록 설계했다. 실제 플레이와 저장 요청이 함께 걸리는 상황에서 서버 안정성을 확인하는 데 초점을 맞췄다.

✓ 2. 장시간 복합 부하 핵심 그래프 및 비교

메모리 사용량, JobQueue Wait p95, Worker Busy %, Save Quickslot RTT p95를 같은 유지 구간 기준으로 정리했다. 장시간 부하에서 메모리, 작업 대기, 처리 여유, 저장 응답이 어떤 흐름으로 유지되는지 함께 읽을 수 있도록 구성했다.

✓ 3. 장시간 복합 부하에서도 900명 동시 접속을 안정적으로 유지했다

유지 구간 동안 메모리 사용량은 일정 수준에 머물렀고, 작업 대기와 스레드 사용률도 급격히 무너지지 않았다. 저장 요청이 함께 발생하는 상황에서도 저장 응답 시간이 크게 흔들리지 않아, 장시간 복합 부하에서도 안정적으로 유지됐음을 보여줬다.

1. 장시간 복합 부하를 통한 900명 동시 접속 안정성 검증

이동, 스킵, 저장 요청이 함께 발생하는 상황에서 서버가 안정적으로 유지되는지 검증했다.

- ▶ 장시간 복합 부하 테스트는 이동, 스킵 전파, 하트비트, quickslot 저장 요청이 함께 반복되는 상황을 일정 시간 유지하면서 서버 안정성을 확인하는 테스트다.

이 시나리오를 따로 만든 이유

실제 서비스 부하는 기능별 테스트만으로 설명하기 어렵다

AOI나 저장 경로를 따로 검증한 결과만으로는 서버 전체 안정성을 설명하기 어렵다. 그래서 실제 플레이와 저장 요청이 함께 발생하는 조건을 별도로 검증했다.

중요한 건 순간 최고치보다 오래 버티는 힘이었다

짧은 순간의 최고치보다, 일정 수준을 오래 유지할 때 대기열과 메모리, 응답 시간이 어떻게 변하는지가 더 중요했다. 그래서 60분 유지를 핵심 조건으로 잡았다.

실제 플레이 흐름에 가까운 입력을 함께 보고 싶었다

이동, 스킵, 하트비트, quickslot 저장 요청을 함께 걸어야 실제 운영에서 서버가 감당해야 하는 부하를 더 현실적으로 설명할 수 있었다.

기본 부하 설정과 비교 기준

부하 구성 이동, 스킵, 하트비트, quickslot 저장 요청이 함께 발생하는 혼합 입력	집계 구간 단계별 증가 구간보다 900명 도달 후 60분 유지 구간을 기준으로 주요 지표를 확인
채널 구성 3개 채널에 분산해 채널당 약 300명 수준으로 부하를 유지	해석 기준 순간 최고치보다 메모리, 작업 대기, 스레드 사용률, 저장 응답이 안정적으로 유지되는지를 중심으로 해석
부하 증가	부하는 150 → 300 → 450 → 600 → 750 → 900 순서로 올렸다. 각 단계 간격은 180초로 두고, 마지막 단계에서만 60분 동안 유지하도록 설계했다.
사용자 구성	전투·이동 사용자 70%, 저장 사용자 20%, 유틸 사용자 10%으로 구성했다. 실제 플레이 전파와 저장 부하가 동시에 걸리도록 만든 분배다.
입력 패턴	전투·이동 사용자는 초당 이동 6회, 스킵 1회, 하트비트 1회로 동작했고, 저장 사용자는 초당 quickslot 변경 1회와 하트비트 1회로 저장 경로에 부하를 만들었다.

주요 관찰 지표

- 지표 1**○ 메모리 사용량은 60분 유지 구간에서 사용량이 계속 증가하지 않고, 일정 수준에서 머무는지 확인하는 지표다.
- 지표 2**○ JobQueue Wait p95(작업 대기 시간)는 복합 부하가 계속 걸리는 동안 대기 시간이 누적되는지, 아니면 관리 가능한 범위에 머무는지 보여준다.
- 지표 3**○ Worker Busy %(스레드 사용률)는 내부 처리 여유가 얼마나 남아 있는지 확인하기 위한 지표다.
- 지표 4**○ Save Quickslot RTT p95(저장 응답 시간)는 저장 요청이 함께 발생하는 상황에서도 저장 경로가 안정적으로 유지되는지 보여준다.

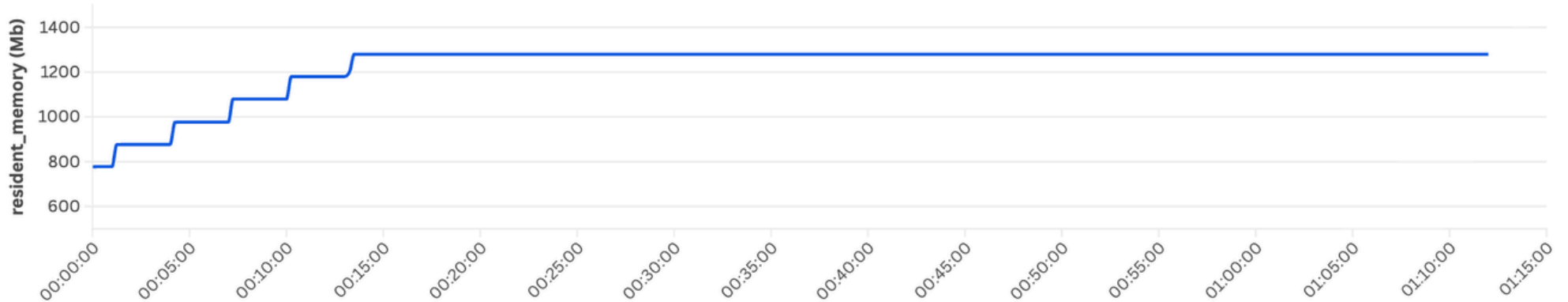
2-1. 장시간 복합 부하 핵심 그래프 (1)

이 구간에서는 순간 최고치보다, 시간이 지나도 누적 증가나 대기 악화가 계속 이어지는지가 더 중요했다.

▶ 60분 유지 구간에서 메모리 사용량과 JobQueue Wait p95를 함께 확인해 장시간 복합 부하 동안 시스템이 안정적으로 유지되는지 점검했다.

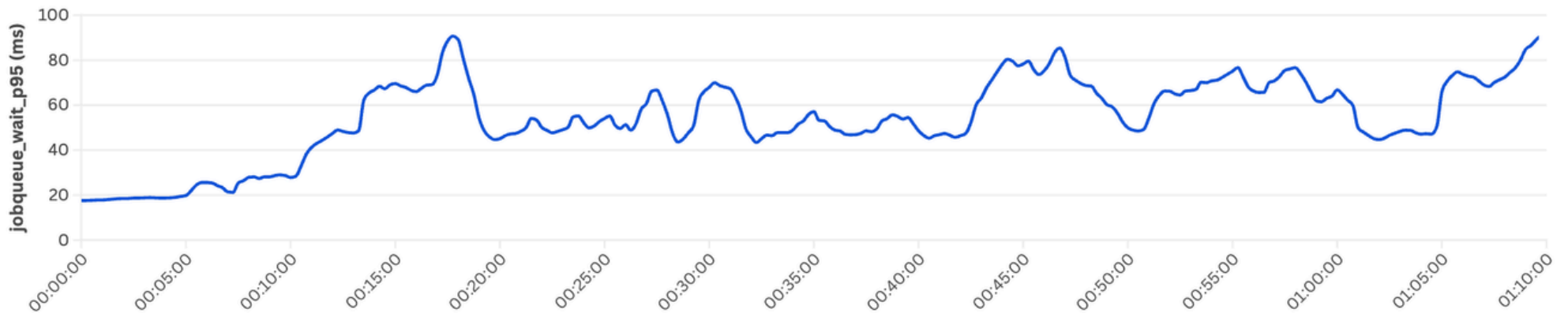
그래프 1. 메모리 사용량

유지 구간에서 메모리 사용량이 계속 증가하는지, 아니면 일정 수준에서 안정되는지 본다.



그래프 2. JobQueue Wait p95

유지 시간이 길어질수록 작업 대기가 계속 쌓이는지, 아니면 관리 가능한 범위에 머무는지 본다.



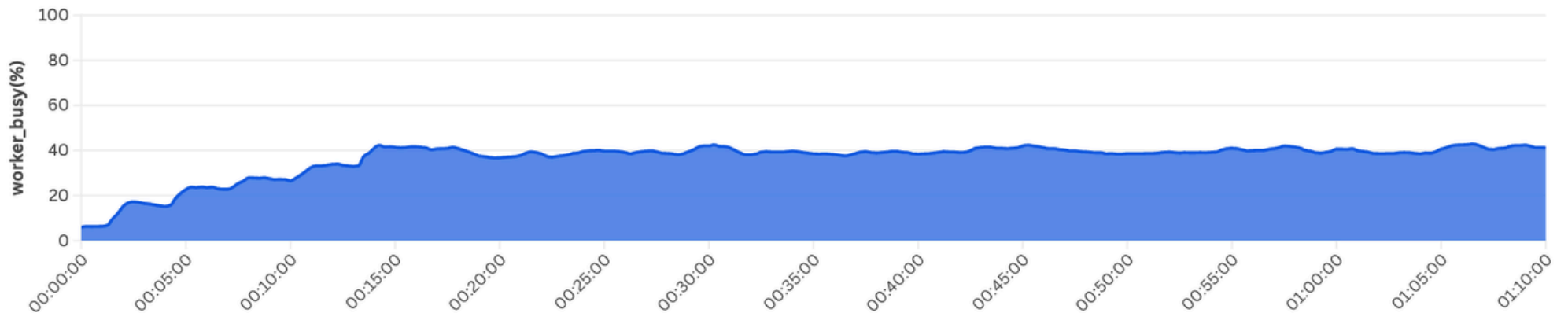
2-2. 장시간 복합 부하 핵심 그래프 (2)

장시간 복합 부하가 계속 걸리는 동안에도 작업 스레드 사용률과 저장 응답 시간이 관리 가능한 범위에 머무는지 확인했다.

▶ 60분 유지 구간에서 Worker Busy %와 Save Quickslot RTT p95를 함께 확인해, 내부 처리 여유와 저장 경로 안정성이 유지되는지 점검했다.

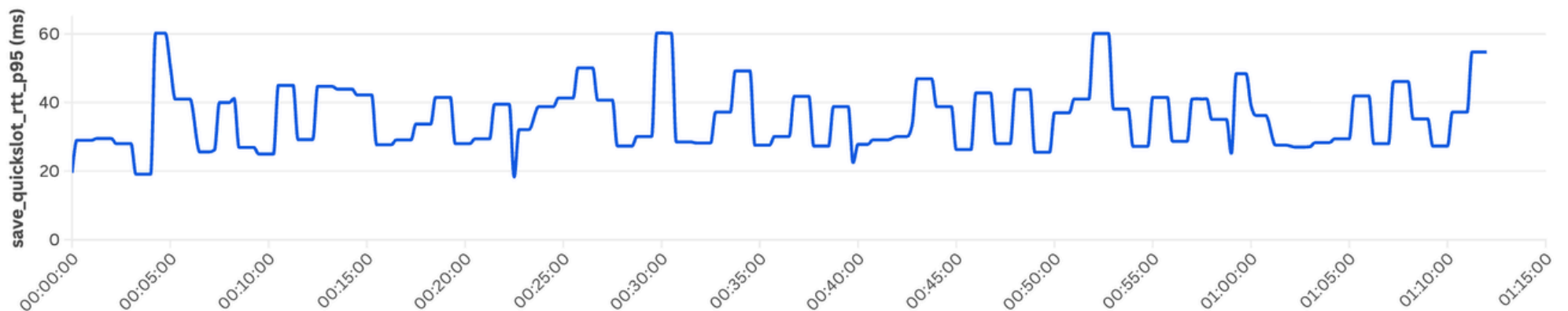
그래프 3. Worker Busy %

작업 스레드 사용률이 유지 구간에서 포화에 가까워지는지, 아니면 처리 여유가 남아 있는지 본다.



그래프 4. Save Quickslot RTT p95

저장 요청이 함께 발생하는 상황에서도 저장 응답 시간이 크게 흔들리지 않는지 본다.



3. 장시간 복합 부하에서도 900명 동시 접속을 안정적으로 유지했다

대표 지표를 바탕으로
장시간 복합 부하 결과를 정리했다.

▶ 같은 장시간 복합 부하에서 네 지표를 같은 기준으로 비교했다.

이를 통해 메모리 사용량, 작업 대기, 스레드 사용률, 저장 응답 시간 사이의 흐름을 함께 확인할 수 있었다.

대표 지표 요약

메모리
메모리 사용량

60분 평균 1280 Mb → 일정 수준에서 유지

대기

JobQueue Wait p95

유지 구간 최대 90.7 ms → 급격한 상승 없이 유지

스레드

Worker Busy %

유지 구간 최대 42.9% → 포화 없이 유지

저장용량

Save Quickslot RTT p95

유지 구간 최대 60.3 ms → 큰 흔들림 없이 유지

이 결과를 어떻게 해석할 수 있는가

관찰

메모리는 유지 구간에서 안정적으로 머물렀다

장시간 구간에서 메모리 사용량이 계속 증가하는 흐름이 아니라, 일정 수준에서 유지되는 흐름을 보였다.

관찰

작업 대기과 스레드 사용률도 감당 가능한 범위였다

JobQueue Wait p95와 Worker Busy %를 함께 보면, 시간이 갈수록 처리 여유가 급격히 사라지는 흐름은 아니었다.

관찰

저장 요청이 있는 상황에서도 전체 흐름은 유지됐다

Save Quickslot RTT p95를 함께 보면, 저장 요청이 동반된 상황에서도 응답 시간이 크게 흔들리지 않았다.

해석

장시간 운용에서 가장 먼저 확인할 위험 신호가 크지 않았다

메모리 누적 증가가 보이지 않았다는 점은 장시간 복합 부하에서도 서버가 급격히 불안정해지지 않았다는 근거가 된다.

해석

내부 처리 측면에서도 장시간 유지가 가능했다

작업 대기과 스레드 사용률이 함께 안정적이었다는 점은, 내부 처리 구조가 장시간 복합 부하를 감당했다는 뜻으로 해석할 수 있다.

해석

이 결과는 실제 운영형 부하에 가까운 검증이었다

단순 접속 유지가 아니라 저장 요청까지 포함한 상태에서도 안정성을 보였다는 점에서, 실제 운영 상황을 설명하는 근거로 사용할 수 있다.

“최종 정리”

이 테스트는 이동과 스킬 전파, 저장 요청이 함께 발생하는 상황에서 동시 접속자 900명을 60분 동안 안정적으로 유지할 수 있음을 보여준다.

메모리 사용량, 작업 대기, 스레드 사용률, 저장 응답 시간 모두를 함께 보면, 이 결과는 단순한 접속 수치가 아니라 실제 운영 안정성을 설명하는 근거로 활용할 수 있다.

Section 6 [트러블슈팅]

- 이 섹션은 기능을 얼마나 많이 만들었는가보다, 문제를 어떻게 해석하고 해결했는가를 중심으로 정리한다.
- 각 사례는 문제를 발견한 뒤 대안을 비교하고, 왜 그 해결책을 선택했는지까지 설명하는 흐름으로 구성한다.
- 뒤의 5개 페이지는 각각 하나의 트러블슈팅을 다루며 공통적으로 문제, 판단, 해결의 연결을 보여준다.

이번 파트에서 확인할 내용

✓ 1. 락 기반 구조의 데드락 위험과 Actor/JobQueue 전환

락 규칙을 더 복잡하게 늘리는 대신, 룸 단위 상태 변경을 직렬화하는 구조가 장기적으로 더 안전하다고 판단했다.

✓ 2. 투사체 중복 히트 방지

판정 주기를 줄여 문제를 숨기기보다, 이미 맞힌 대상을 서버 런타임 상태로 기록해 전투 판정을 명확하게 유지하는 쪽을 택했다.

✓ 3. 거래 중 인벤토리 조작으로 인한 정합성 붕괴 방지

단순 차단 한 줄보다 거래 상태머신과 DB 원자 커밋을 함께 설계해야 복사와 증발을 동시에 막을 수 있다고 봤다.

✓ 4. AOI 대량 스폰 동기화와 배치/스냅샷 처리

많이 보내는 것보다 안정적으로 나눠 보내는 것이 중요하다고 판단해, 갱신 주기 조절과 스냅샷 경계 표현을 같이 적용했다.

✓ 5. Redis Dirty Set 기반 자동 저장과 중복 커밋 방지

많이 보내는 것보다 안정적으로 나눠 보내는 것이 중요하다고 판단해, 갱신 주기 조절과 스냅샷 경계 표현을 같이 적용했다.

1. 락 기반 구조의 데드락 위험과 Actor/JobQueue 전환

락을 더 잘 거는 방법보다, 락에 덜 의존하는 구조를 택한 사례

▶ 락 규칙을 더 복잡하게 늘리기보다, 룸 단위 직렬화가 더 안전한 구조라고 판단했다.

문제 상황

룸, 파티, 인스턴스처럼 공유 상태가 많은 기능이 늘어나면서 여러 스레드가 같은 데이터에 접근하는 일이 잦아졌다.

처음에는 락으로 일관성을 맞추는 방식이 자연스러웠지만, 기능이 늘수록 락 획득 순서를 사람이 계속 기억해야 하는 구조가 됐다.

나는 특정 버그 한 건보다, 기능이 더 붙을수록 사고 가능성이 커지는 구조 자체가 더 큰 문제라고 봤다.

데드락은 단순 성능 저하가 아니라, 룸 진행과 관련 흐름 전체를 멈출 수 있는 문제였다. 재현이 어려운 만큼 운영 단계에서 터지면 분석 비용도 매우 커진다.

처음 고민한 대안

- 1 락 획득 순서를 더 엄격하게 문서화한다**
바로 적용하기는 쉽지만, 시간이 지날수록 결국 사람 기억과 리뷰 품질에 크게 의존하게 된다.
- 2 락 범위를 더 잘게 나눠 경합을 줄인다**
단기 튜닝에는 도움이 되지만, 유지보수자가 더 많은 순서와 예외를 떠안게 되는 점이 부담이었다.
- 3 공유 상태 변경이 많은 영역을 직렬화한다**
동시에 바뀌지 않게 구조를 바꾸면, 데드락을 막는 규칙보다 훨씬 단순한 설명으로 시스템을 유지할 수 있다고 봤다.

최종 선택

"락으로 복잡한 공유 상태를 지키는 구조"에서 "기본적으로 동시에 바꾸지 않는 구조"로 중심을 옮겼다.

룸 단위 핵심 상태 변경은 Actor와 JobQueue를 통해 순차적으로 처리하고, 락은 꼭 필요한 경계 보호에만 남기는 방향을 선택했다.

왜 이 방법을 골랐는가

락 규칙 강화는 가능하지만, 규모가 커질수록 다시 깨질 여지가 크다고 봤다.

락 세분화는 단기적으로는 좋아 보여도 장기적으로는 더 많은 순서와 예외를 기억해야 한다.

액터 기반 직렬화는 "한 번에 한 흐름만 상태를 바꾼다"는 단순한 규칙으로 사고 범위를 줄일 수 있었다.

액터 기반 직렬화는 "한 번에 한 흐름만 상태를 바꾼다"는 단순한 규칙으로 사고 범위를 줄일 수 있었다.

해결 방식과 결과

- 룸 내부의 핵심 상태 변경은 큐에 작업을 넣고 순차적으로 실행하는 흐름으로 정리했다.
- 개발 단계에서는 락 획득 흐름을 추적해 위험 징후를 조기에 확인할 수 있게 했다.
- 결과적으로 락 규칙을 계속 늘리는 구조보다, 기능이 늘어나도 설명 가능한 구조로 바뀌었다.

2. 투사체 중복 히트 방지

▶ 전투 결과가 흔들리지 않도록, 투사체 판정 구조를 다시 잡았다.

문제 상황

투사체는 한 번 생성된 뒤 여러 틱에 걸쳐 충돌 판정을 수행하기 때문에, 같은 대상을 연속 틱에서 다시 판정할 여지가 있었다.

여기에 폴링 재사용까지 섞이면 이전 투사체의 상태가 남아, 의도하지 않은 중복 데미지나 오판정으로 이어질 가능성도 생긴다.

나는 이 문제를 단순히 피격이 한 번 더 들어가는 버그가 아니라, 서버가 전투 결과를 얼마나 일관되게 설명할 수 있는가의 문제로 봤다.

전투 결과가 흔들리면 플레이어는 서버 판정을 믿기 어려워진다. 특히 투사체는 여러 대상과 겹치기 쉬워서, 예외가 누적되면 밸런스 문제와 체감 불신으로 바로 이어질 수 있었다.

처음 고민한 대안

- 충돌 판정 주기를 낮춰 중복 가능성을 줄인다**
겉으로는 문제가 줄어들 수 있지만, 빠른 전투 감각이 둔해지고 근본 원인도 그대로 남는 방식이었다.
- 같은 대상에 짧은 무적 시간을 주는 후처리를 넣는다**
특정 상황은 막을 수 있어도 다른 스킬의 다단 히트 설계와 충돌할 여지가 있어 보편적인 해결책은 아니라고 봤다.
- 투사체가 이미 맞힌 대상을 직접 기억하도록 만든다**
원인과 대응이 가장 정확하게 연결되고, 전투 감각을 건드리지 않으면서도 판정을 명확하게 통제할 수 있는 방향이었다.

최종 선택

"덜 맞게 보정하는 것"보다 "왜 한 번만 맞는지 설명 가능한 상태를 갖는 것"을 우선했다.

투사체마다 이미 타격한 대상 목록을 서버 런타임 상태로 관리하고, 재사용 시에는 그 상태를 반드시 초기화하는 방식을 선택했다.

왜 이 방법을 골랐는가

판정 주기를 낮추는 방법은 문제를 숨길 수는 있어도 전투 감각 자체를 둔하게 만들 수 있었다.

짧은 무적 시간은 다른 스킬이나 다단 히트 설계와 충돌할 여지가 있어 예외 관리가 늘어날 수 있었다.

반면 투사체 단위 상태 관리는 원인과 해결이 정확하게 대응되고, 판정 기준도 서버 안에서 명확하게 남는다.

나는 이 문제에서 중요한 것은 덜 맞게 보정하는 것이 아니라, 서버가 왜 다시 맞지 않았는지를 설명 가능한 상태를 갖는 것이라고 봤다.

해결 방식과 결과

- 투사체가 실제로 데미지를 적용한 대상을 런타임 집합으로 기록해 동일 대상 재타격을 막았다.
- 판정 직후 바로 기록하는 흐름으로 맞춰, 연속 틱에서도 같은 대상이 다시 처리되지 않도록 했다.
- 폴링 재사용 시에는 피격 기록과 관련 카운트를 함께 초기화해 이전 생애의 상태가 남지 않도록 정리했다.

3. 거래 중 인벤토리 조작으로 인한 정합성 붕괴 방지

▶ 거래는 기능 추가보다 정합성을 지키는 구조가 먼저 필요한 영역이었다.

문제 상황

거래 도중 플레이어가 아이템을 사용하거나 장착을 바꾸거나 슬롯을 옮기면, 거래창에 보이는 상태와 실제 인벤토리 상태가 달라질 수 있었다.

이 상태에서 취소, 확정, 맵 이동, 접속 종료가 겹치면 아이템 복사나 증발 같은 문제가 생길 여지가 있었다.

나는 거래를 기능 하나로 보기보다, 인벤토리, 골드, 세션 상태, DB 저장이 함께 엮이는 흐름으로 봤고 그래서 중간 상태를 어떻게 다룰지가 더 중요하다고 판단했다.

정합성 문제는 한 번 발생하면 유저 피해로 바로 이어지고 복구 비용도 크다.
특히 거래는 두 명이 동시에 엮힌 기능이라 한쪽만 맞는 해결로는 충분하지 않았다.

처음 고민한 대안

- 1 거래 중에도 인벤토리 조작을 허용하고 마지막 단계에서만 검증한다**
사용성은 좋지만 이미 중간 과정에서 꼬인 상태를 마지막 한 번의 검증만으로 설명하고 복구하기 어렵다고 봤다.
- 2 거래 시작과 동시에 모든 인벤토리 조작을 강하게 막는다**
안전성은 높지만 사용자가 아직 교환 품목을 조정하는 단계까지 너무 일찍 잠가 버려 사용성이 거칠다고 판단했다.
- 3 거래 상태를 단계별로 나누고, 잠금 이후엔 수정 불가로 만들며 최종 반영은 원자적으로 처리한다**
사용성과 안정성을 함께 가져가면서, 어느 시점부터 상태를 확정해야 하는지도 명확하게 설명할 수 있는 방향이었다.

최종 선택

"처음부터 전부 잠그는 것"보다 "수정 가능한 구간과 확정 구간을 명확히 나누는 것"을 선택했다.

거래를 초대, 활성, 잠금, 커밋 단계로 나누고, 준비 완료 이후에는 물품 수정이 불가능하게 만들며, 최종 반영은 DB 원자 커밋으로 처리하는 구조를 택했다.

왜 이 방법을 골랐는가

마지막 단계 검증만으로는 이미 중간 과정에서 꼬인 상태를 설명하기 어렵고, 예외 상황이 겹칠수록 누수 가능성이 커진다고 봤다.

처음부터 전부 잠그는 방식은 안전하지만 사용성 측면에서 너무 거칠어, 거래 흐름 자체가 불편해질 수 있었다.

단계별 상태머신은 사용자가 조정할 수 있는 구간과 더 이상 바꾸면 안 되는 구간을 자연스럽게 나눌 수 있었다.

여기에 DB 원자 커밋을 붙여야만 메모리 상태와 영속 상태가 함께 맞고, 두 플레이어 결과도 같이 성공하거나 같이 실패하게 만들 수 있다고 판단했다.

해결 방식과 결과

- 거래 중에는 아이템 사용, 장착, 드래그 같은 인벤토리 조작을 서버에서 차단해 중간 상태가 흔들리지 않도록 했다.
- 두 플레이어가 준비를 마치면 잠금 상태로 전환해 이후 수정이 불가능하도록 만들었다.
- 최종 확정은 DB 단일 트랜잭션으로 반영하고, 접속 종료나 맵 이동처럼 흐름을 깨는 이벤트는 취소 경로로 분리해 상태를 정리했다.

많이 보내는 것보다, 안정적으로 나눠 보내는 쪽을 선택한 사례

4. AOI 대량 스폰 동기화와 배치/스냅샷 처리

▶ 많은 오브젝트가 동시에 나타나는 구간에서, 기존 방식은 순간 부하와 동기화 흔들림이 컸다.

문제 상황

맵 입장 직후나 시야가 크게 바뀌는 순간에는 플레이어, 몬스터, 투사체 정보가 한꺼번에 갱신된다.

이 데이터를 한 번에 몰아서 보내면 순간적인 패킷 부담이 커지고, 수신 시점에 따라 체감상 순서가 꼬일 수 있었다.

나는 이 문제를 단순 성능 문제가 아니라, 클라이언트가 언제 스냅샷을 받기 시작했고 언제 끝났는지를 알 수 있어야 하는 문제로 봤다.

AOI는 전투, 이동, 타겟팅, 시야 체감 등 거의 모든 플레이 경험의 바탕이 된다. 서버는 정확한 데이터만 보내는 것이 아니라, 클라이언트가 그것을 안정적으로 해석할 수 있게 전달해야 했다.

처음 고민한 대안

- 1 **시야 갱신 때 필요한 정보를 한 번에 전송한다**
구현은 단순하지만 실제 플레이 상황에서는 순간 부하와 동기화 혼란에 가장 취약한 방식이라고 판단했다.
- 2 **시야 반경 자체를 줄여 전송량을 억제한다**
부담은 줄어 들 수 있어도 근본 해결이 아니라 콘텐츠 품질과 체감 범위를 희생하는 방법에 가까웠다.
- 3 **AOI 갱신 주기를 조절하고, 스폰을 배치로 나누며, 스냅샷 경계를 패킷으로 명확히 표현한다**
성능과 동기화를 함께 잡을 수 있고, 이후 튜닝 포인트도 명확해지는 방향이라고 봤다.

최종 선택

"얼마나 많이 보내는가"보다 "클라이언트가 안정적으로 받도록 어떻게 나눠 보내는가"를 우선했다.

AOI 갱신은 조건이 충족될 때만 수행하고, 스폰 데이터는 배치 전송으로 나누며, 스냅샷 시작과 종료를 명확히 표시하는 구조를 선택했다.

왜 이 방법을 골랐는가

한 번에 보내는 방식은 구현은 단순하지만, 실제 플레이 상황에서 가장 취약한 형태라고 판단했다.

시야 반경 축소는 근본 해결이 아니라 콘텐츠 품질을 희생하는 방법에 가까웠다.

반면 배치 전송과 스냅샷 경계는 성능과 동기화를 함께 잡을 수 있고, 나중에 튜닝 포인트도 분명하게 남는다.

나는 이 문제를 얼마나 적게 보내는가보다, 클라이언트가 안정적으로 받도록 어떻게 나눠 보내는가의 문제로 봤다.

해결 방식과 결과

- AOI는 매 프레임 무조건 갱신하지 않고, 이동량이나 구역 변경이 일정 기준을 넘을 때만 갱신하도록 정리했다.
- 스폰 데이터는 종류와 수량에 따라 배치로 나눠 전송해 순간 부하를 분산했다.
- 대량 동기화가 필요한 상황에서는 스냅샷 시작과 종료를 명시해 수신 경계를 분명하게 만들었다.

5. Redis Dirty Set 기반 자동 저장과 중복 커밋 방지

▶ 변경이 생길 때마다 바로 저장하는 방식으로는 저장 부하와 중복 커밋을 감당하기 어려웠다.

문제 상황

플레이어 데이터는 주기 저장, 로그아웃 즉시 저장, 인벤토리 변경, 퀵슬롯 변경 등 여러 계기로 저장 요청이 발생한다.

이 요청들이 겹치면 같은 플레이어를 여러 번 저장하거나, 저장 도중 다시 저장 요청이 들어와 흐름이 꼬일 수 있었다.

나는 저장 문제를 단순히 언제 저장하느냐의 문제가 아니라, 무엇이 정말 바뀌었고 누가 이미 처리 중인지 서버가 알고 있어야 하는 문제로 봤다.

저장은 마지막 방어선이라서 한 번 잘못되면 그 전까지의 모든 서버 판정이 의미를 잃을 수 있다. 중복 커밋은 DB 부담을 키우고, 반대로 저장 누락은 유저 데이터를 잃게 만든다.

처음 고민한 대안

- 1 변경이 생길 때마다 즉시 저장한다**
구현은 단순하지만 변경이 몰리는 상황에서는 저장 부하가 급격히 커지고, 같은 대상을 여러 번 보내게 될 가능성도 컸다.
- 2 일정 주기마다 모든 플레이어를 통째로 저장한다**
누락 걱정은 줄일 수 있어도 실제로 바뀌지 않은 대상까지 계속 저장하게 되어 비용이 너무 컸다.
- 3 변경된 플레이어만 모으고, 이미 저장 중인 대상은 다시 보내지 않는 구조를 만든다**
정말 필요한 대상만 다룰 수 있고, 중복 저장도 명확하게 제어할 수 있는 흐름이라고 판단했다.

최종 선택

"언제 저장할지"보다 "무엇이 바뀌었고 누가 이미 저장 중인지"를 아는 구조를 먼저 만들었다.

변경된 플레이어를 Dirty Set으로 모으고, 저장 중인 플레이어는 inflight 상태로 관리해 같은 시점의 중복 저장을 막는 구조를 선택했다.

왜 이 방법을 골랐는가

변경마다 즉시 저장하는 방식은 구현은 단순하지만, 요청이 몰리면 부하가 급격히 커지고 중복 저장도 늘어날 수 있었다.

모든 플레이어를 주기 저장하는 방식은 실제로 바뀌지 않은 대상까지 반복해서 처리하게 되어 비용이 너무 컸다.

Dirty Set 방식은 정말 바뀐 대상만 다루기 때문에 효율이 좋고, inflight 제어를 붙이면 중복 커밋도 명확하게 막을 수 있었다.

나는 저장을 나중에 한 번 하면 되는 일이 아니라, 변경 감지와 반영 순서를 설계해야 하는 서버 흐름이라고 봤다.

해결 방식과 결과

- 로그인 직후처럼 DB와 동기화된 상태는 Dirty 표시를 지워 기존 상태를 명확히 했다.
- 플레이어 코어, 인벤토리, 퀵슬롯 등 변경된 범주만 Dirty Set에 기록해 실제 변경 대상만 모았다.
- 저장 주기나 즉시 저장 시점에는 Dirty Set을 기준으로 대상을 추리고, 이미 저장 중인 플레이어는 inflight 상태로 관리해 중복 전송을 막았다.